AWS Whitepaper

Games Industry Lens

Games Industry Lens: AWS Whitepaper

Table of Contents

Games Industry Lens – AWS Well-Architected Framework

Publication date: **December 9, 2025** (*Document revisions*)

The AWS Well-Architected Framework assists cloud architects build secure, high-performing, resilient, and efficient infrastructure for their applications and workloads. Based on six pillars — operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability — Well-Architected provides a consistent approach for customers and AWS Partners to evaluate architectures, remediate risks, and implement designs that deliver business value.

In this lens, we focus on how to design, deploy, and architect your games workloads in the AWS Cloud. We define components, explore common workload scenarios, and outline design principles that assist you to apply the Well-Architected Framework. We recommend that you begin designing your architecture by considering the best practices and questions from the AWS Well-Architected Framework whitepaper. This document provides supplemental best practices for games industry customers.

This lens specifies best practices that are intended to address the unique characteristics of building and operating games in the cloud based on our experience working with games industry developers and publishers around the world. It provides guidance on how to design and operate your environment so that it is cost optimized and scalable for fluctuations in global player demand. This lens also provides guidance for securing your game infrastructure and tuning performance to deliver a positive player experience.

This document is intended for those in technology roles, such as chief technology officers (CTOs), game studio technical directors, architects, developers, and operations team members. After reading this document, you will understand AWS best practices and strategies to use when designing architectures for games.

Lens availability

Custom lenses extend the best practice guidance provided by AWS Well-Architected Tool. AWS WA Tool allows you to create your own custom lenses, or to use lenses created by others that have been shared with you.

To begin reviewing your games workload, download and import the Games Industry Lens into AWS Well-Architected Tool from the public AWS Well-Architected custom lens GitHub repository.

Design principles

The AWS Well-Architected Framework identifies the following general design principles to facilitate good design in the cloud for games workloads:

- **Understand player behavior and usage patterns to evolve the game and help protect players:** To drive improvement continually to your game and effectively manage the player experience, it is important to have visibility into how players interact with game itself and with the rest of the players. This assists you understand how to improve the game, manage costs, and monitor and react to unauthorized usage activity that poses risks to the player experience.
- **Use technologies that simplify game operations and increase development speed:** Prioritize the adoption of technologies that can improve speed and reduce the operational overhead of delivering new features and improvements to players. Games are hits-driven and there are many choices for players to consider, so moving quickly and adapting to change is critical for the success of a game. Consider whether you are comfortable operating your own software or if you would prefer adopting a comparable managed service from AWS, AWS Partners, or both.
- **Optimize your architecture to improve metrics that reflect actual player experience:** As you adapt and evolve your architecture over time, think about how these improvements and changes will impact player experience. Games workloads should be able to withstand and minimize the impact of failures to block widespread disruptions to gameplay. Game features and systems that aren't critically dependent on each other should be de-coupled to reduce the impact of failures and isolate player impacting issues.
- **Design infrastructure to meet peak player concurrency and dynamically scale as needed:** Infrastructure should be designed to scale to meet player demand. Metrics, such as player session concurrency and number of logins, can be used to preemptively scale before systems become overloaded. Reactive system utilization metrics, such as CPU and memory consumption, can be used to scale after systems become overloaded. By dynamically scaling your infrastructure, you can reduce the costs of operating your game.
- **Implement runbooks to improve game operations:** Operational runbooks should be used to consistently manage recurring game operations tasks. Runbooks should exist for common game operations workflows such as investigating and responding to player reports, managing infrastructure pre-scaling activities to prepare for anticipated large-scale events like new season launches and game content releases, and to address typical game maintenance activities.

Scenarios

In this section, we cover several scenarios that are common in a game architecture. Each scenario includes the common characteristics that drive the design and an example reference architecture diagram.

Scenarios

Game hosting for real-time synchronous gameplay

Real-time synchronous gameplay allows for two or more players to participate and interact in a game simultaneously where the state of the gameplay is shared between the connected players to create as close to a real-time experience as possible. Examples of synchronous games include first-person shooters, massively multiplayer online games (MMOG), sports and action games, or an online game where two or more players must be connected to share the play experience in near real time.

Characteristics of real-time synchronous gameplay architectures include:

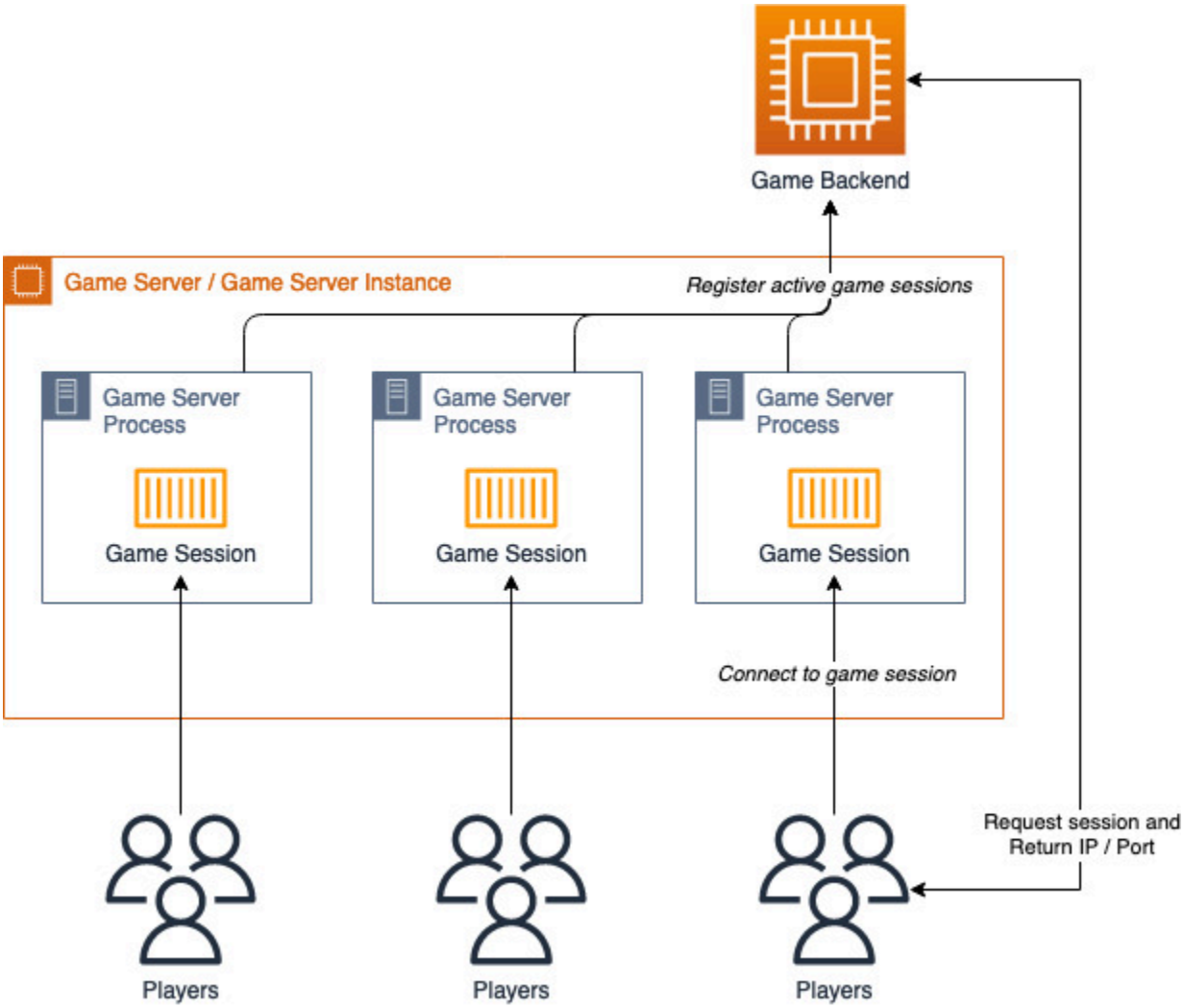
- Some games may be hosted as game sessions through game server processes that run on dedicated servers. Some games may use P2P-like architectures that employ lighter-weight Session Traversal Utilities for NAT (STUN) or Traversal Using Relays around NAT (TURN) servers. Regardless of the type of server involved, game servers are hosted in multiple data centers and AWS Regions globally.
- Game clients can join a game session either by requesting a match from a centralized matchmaking service hosted in the game backend system or by selecting a match from a predefined list of available game servers. The game client is provided with an IP address and port to connect to.
- Many synchronous games are latency sensitive, like first-person shooters and massively multiplayer online games. They will likely include algorithms like rewind and time dilation to

minimize lag effects but may also have a pre-defined latency tolerance that is carefully measured and optimized to reduce the lag experience that can sometimes occur for players in high-latency situations. This latency information is determined by instrumenting game clients to ping the available game server AWS Regions to capture metrics such as latency, network jitter, and other important metrics for the gameplay experience. These metrics are sent to a central metrics collection service in the game backend system so that live operation streams can monitor game health. During the matchmaking process, game clients provide their current latency data as one of the request parameters when requesting a match, and the matchmaking service can use that latency data as one of the variables when selecting a game server to host the player.

- Typically, the gameplay is conducted over a mix of protocols (like game servers using faster UDP-based messaging with matchmaking, authentication, and other client-server traffic using HTTPS).
- Game servers employ algorithms and design to minimize client-server traffic like transform streaming, deltas, and data compression.
- Game servers are frequent targets for malicious activities and should be protected with a DDoS protection solution like AWS Shield Advanced.

Game server processes

The following diagram illustrates a typical game server architecture. It describes the logical relationship between a game server instance and the game server processes that host game sessions.



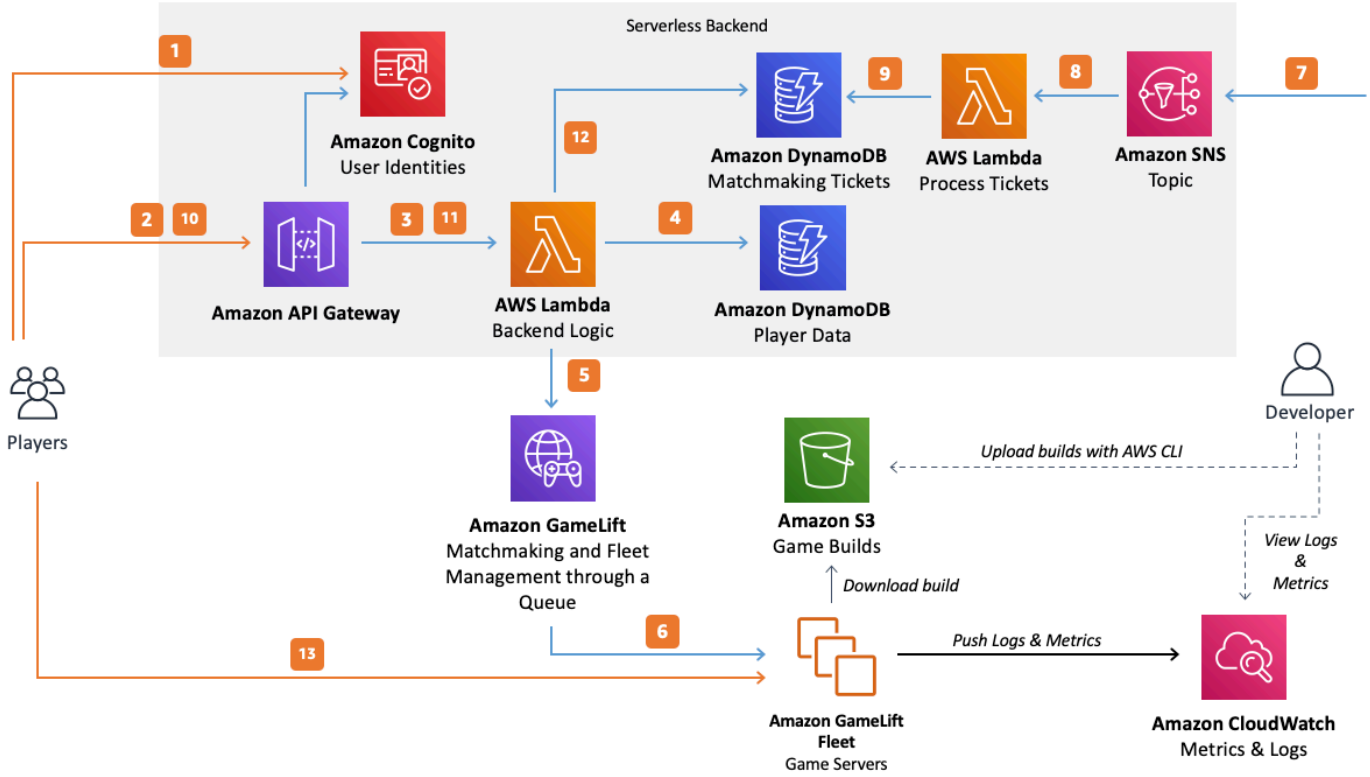
Logical game server architecture

- An Amazon EC2 instance is used as a game server, also known as a game server instance. The game server hosts one or more game server processes, and each is running a copy of your game server build. Typically, multiple game server processes are running on a game server instance to utilize compute resources efficiently and reduce costs. When a game session is active and ready to host player sessions, its status is updated with the game backend (usually a matchmaking service) so that it can begin to be used to host players.
- The game backend can provide the player with the IP address and server port that is hosting a game session so that they can connect to play.

Session-based game server hosting with serverless backend

When developing an architecture for your game, consider the features and capabilities you need and the level of operational management overhead that you are prepared to own. To provide the best balance between ease of operations and flexibility, you can build your game using managed services from cloud providers. Managed services give you the control to develop and customize your own custom game features, while also reducing your burden to deploy and manage infrastructure.

Hosting a session-based multiplayer game requires having server infrastructure to host the game server processes as well as a scalable backend for matchmaking and session management. The following reference architecture shows how Amazon GameLift managed hosting and a serverless backend can be used to manage your session-based games.



Amazon GameLift managed hosting for session-based games

The diagram describes the process of getting players into games running on GameLift managed game hosting. It includes the following steps:

1. The game client requests an Amazon Cognito identity from an Amazon Cognito identity pool. This can optionally be connected to external identity providers.

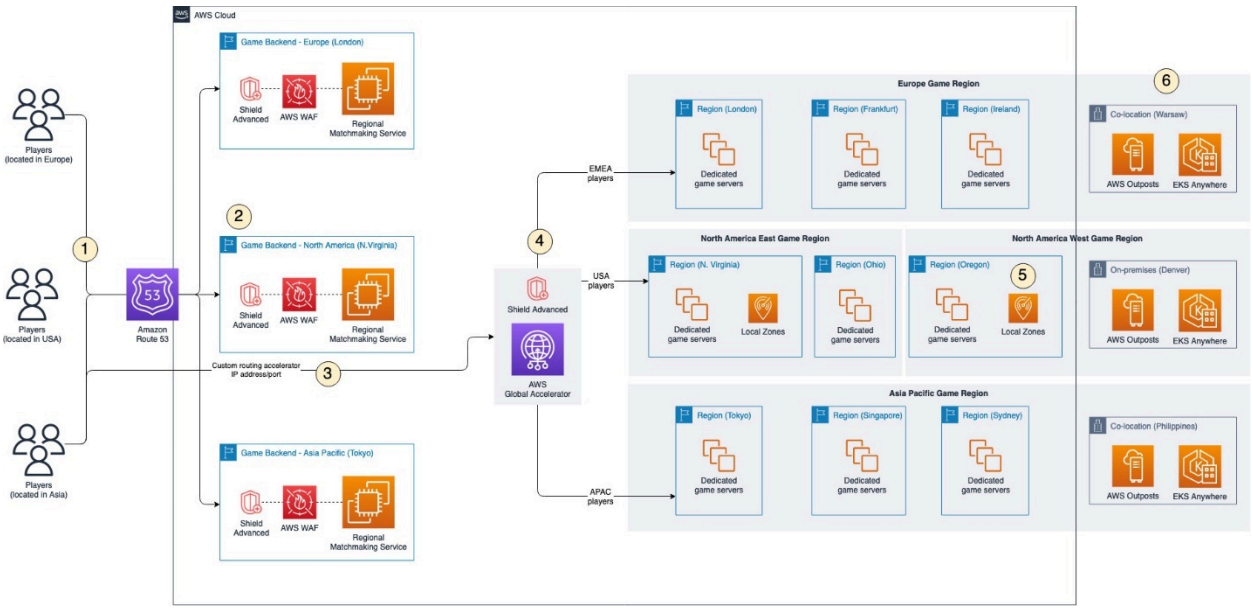
2. The game client receives temporary access credentials and requests a game session through an Amazon API Gateway by signing the request with the Amazon Cognito credentials.
3. API Gateway invokes an AWS Lambda function.
4. The Lambda function requests player data from an Amazon DynamoDB table. The Amazon Cognito identity is used to securely request the correct player data because the authenticated identity is provided in the request context data.
5. Using the correct player data for additional information (like player skill level), the Lambda function requests a match through GameLift FlexMatch matchmaking. You can define a FlexMatch matchmaking configuration with JSON-based configuration documents. The game client can generate latency metrics by pinging server endpoints in various Regions, and the latency data can be used to support latency-based matchmaking.
6. After FlexMatch matches a suitable group of players with suitable latency to a Region, it requests a game session placement through a GameLift queue. The queue contains fleets with one or more registered Region locations.
7. When the session is placed on one of the fleet's locations, an event notification is sent to an Amazon SNS topic.
8. A Lambda function will receive the Amazon SNS event and process it.
9. If the Amazon SNS message is a MatchmakingSucceeded event, the Lambda function writes the result to DynamoDB with the server port and IP address. A time-to-live (TTL) value is used to make sure that matchmaking tickets are deleted from DynamoDB when they no longer needed.
10. The game client makes a signed request to API Gateway to check the status of the matchmaking ticket on a specific interval.
11. API Gateway invokes a Lambda function that checks the matchmaking ticket status.
12. The Lambda function checks DynamoDB to determine whether the ticket has succeeded. If it has succeeded, the Lambda function sends the IP address, port, and the player session ID back to the client. If the ticket failed, the Lambda function sends a response declaring that the match is not ready.
13. The game client connects to the game server using TCP or UDP by using the port and IP address provided by the backend. It sends the player session ID to the game server, and the game server validates it using the Amazon GameLift Server SDK.

Alternatively, you can modify the preceding architecture to use API Gateway WebSockets with Amazon GameLift. In this approach, communication between the game client and your game backend service occurs using a WebSocket-based implementation. This implementation can be

used so that the game backend Lambda function initiates a server-side message to the game client over a WebSocket rather than implementing a polling model.

Multi-Region and hybrid architecture for low-latency games

This section describes a multi-Region and hybrid architecture for low-latency games.



Reducing latency with network acceleration and game servers deployed globally

1. Players in a globally available game can originate from anywhere. When a player requests a game session or match, their game client sends a request to the game backend service registered with Amazon Route 53. Route 53 latency-based routing can be used to route the player to the closest available game backend.
2. The game backend is deployed in multiple AWS Regions closest to player populations. Each game backend includes a Regional matchmaking service that finds a game session from across the game Regions. Although a player's matchmaking request is processed by a Regional matchmaking service near them, the matchmaking service is capable of routing a player to a game session in another game Region, if necessary. This action improves resiliency and performance. Additionally, each game backend service uses AWS WAF and AWS Shield Advanced to provide layer-7 web filtering and bot control, and protections against distribution denial of service (DDoS) attacks. You have many options for how you build your game backend service, such as serverless, containers, EC2 instances, or hosting the game backend service in your own data centers.

3. To improve the experience for players by reducing network latency and jitter, a custom routing accelerator is deployed using AWS Global Accelerator, which automatically optimizes the routing of traffic from the game client to the game server using the AWS global network. You configure the custom routing accelerator to map Global Accelerator listener ports to your game server's EC2 instance port. The game client connects to the Global Accelerator IP and port that serves as a proxy and deterministically routes the players to the correct game server IP and port that is hosting the game session.
4. Your game includes player-friendly logical game Regions that represent a collection of game server hosting locations that are geographically close to each other, such as North America or Asia Pacific. To reduce latency and increase geographic coverage, a combination of different game server hosting solutions can be used to improve the player experience. Prioritize the use of AWS Regions where possible, as these locations are fully featured and contain the largest capacity footprint.
5. Use AWS Local Zones to host game servers in geographic locations of under-served players where you do not have existing hosting facilities, or an AWS Region is not available.
6. Deploy AWS Outposts into your existing on-premises data centers and co-location providers to create a seamless control plane and management experience across each deployment location using fully managed racks and rack-mounted servers. AWS Outposts are also beneficial if you don't have existing server capacity in your on-premises environment. However, if you want a hybrid implementation with software running on your own existing server infrastructure, you should use Amazon EKS Anywhere, which allows you to create and run Kubernetes clusters on your own infrastructure with connectivity back to Amazon EKS. This provides a consistent console view of your Kubernetes clusters in on-premises.

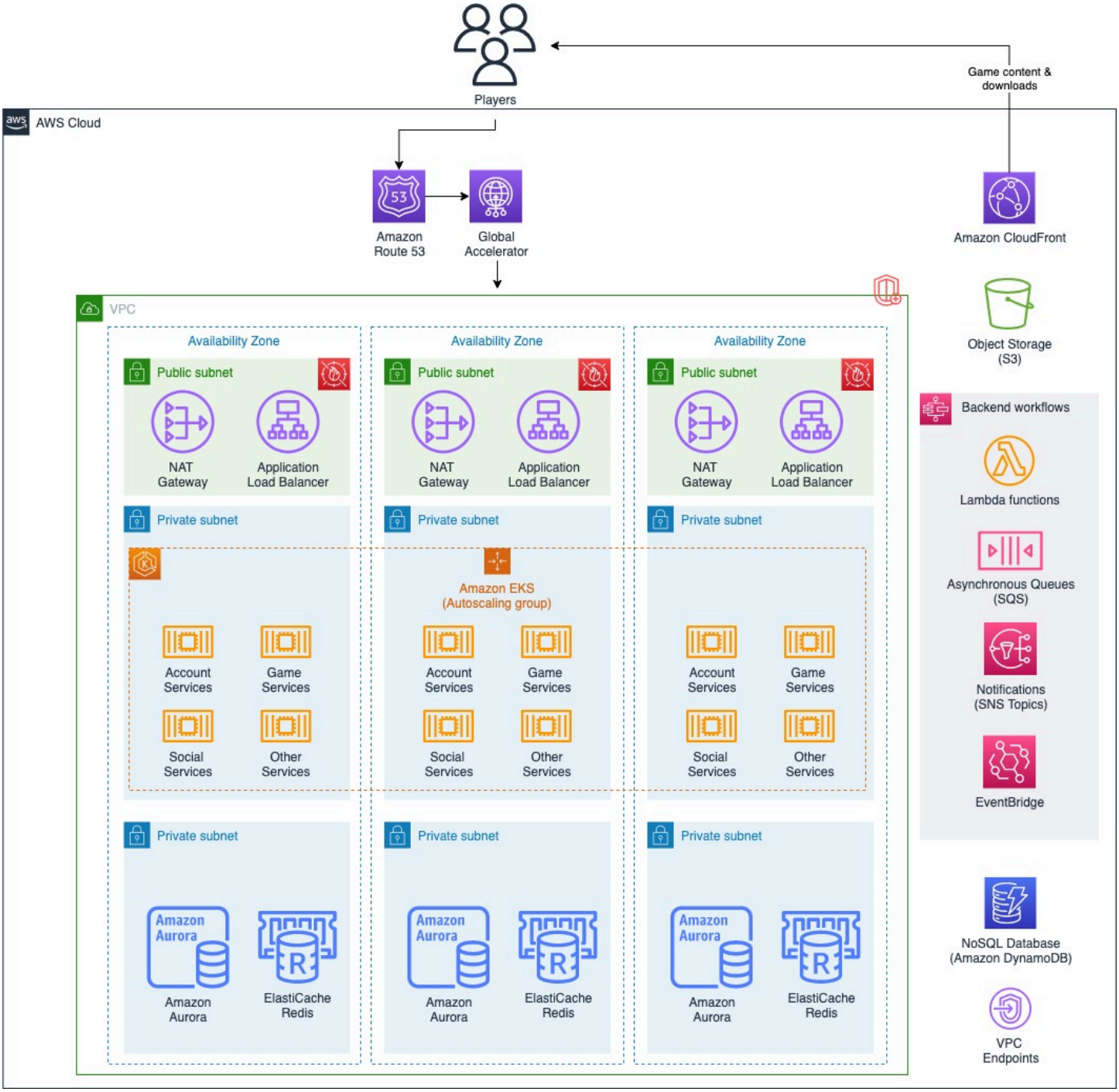
Game backends

Game backends are used to manage game and player state, as well as integrate social and system-level features into the game that support the gaming experience. Player profile management, item and inventory storage, and stats and leaderboards are examples of services hosted in game backends.

Game backends are typically built as REST APIs that are accessed by clients using HTTPS. However, other approaches are also common, such as WebSockets that provide bidirectional channels for use cases such as client notifications for in-game chat and presence. Game backends can be deployed using a variety of different deployment architectures, including using instances, containers, or a serverless architecture.

Container-based game backend architecture

This section outlines a container-based game backend architecture.



Hosting a game backend using containers

- Players access the game using game client software, which can be distributed to them through a gaming system, a digital storefront, or a direct download from a content delivery network

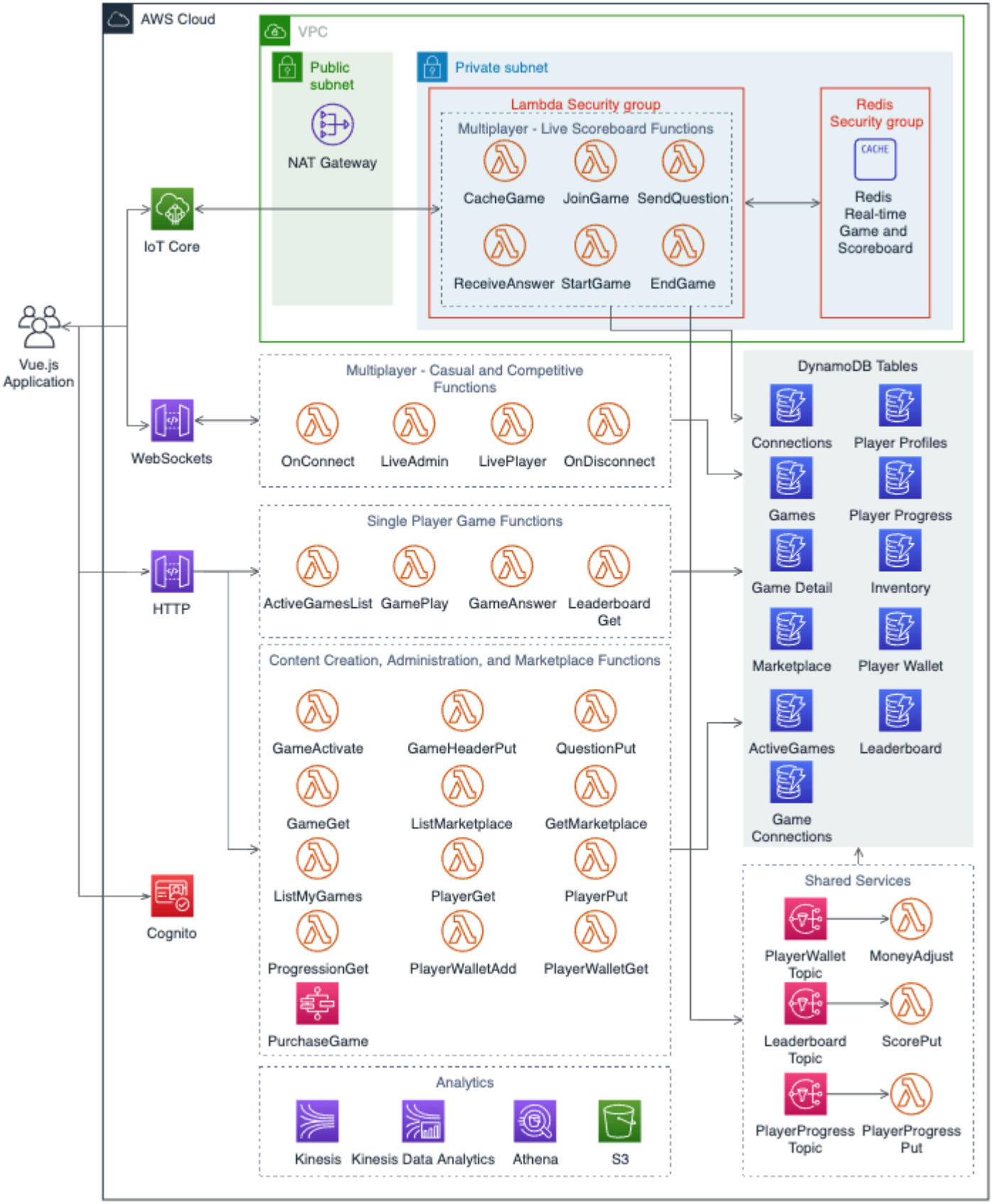
(CDN), such as Amazon CloudFront. CDNs provide caching at edge locations to accelerate the performance for users downloading content. For example, CloudFront can be used to distribute the game client software to your players as well as the game assets and other content.

- AWS Global Accelerator provides traffic acceleration and customizable controls for routing traffic from player game clients to your load balancers as well as routing traffic across Regions for multi-Region and failover purposes. The custom domain names for your game backend REST APIs are configured in Amazon Route 53 to route traffic to Global Accelerator endpoints. Not shown in the diagram, AWS Shield Advanced can provide additional DDoS mitigation for your accelerator and game backend.
- NAT Gateway and Application Load Balancer are deployed to public subnets in each of the Availability Zones used by the game backend to provide high availability with the Region. AWS WAF is deployed on the Application Load Balancer to provide layer-7 web traffic filtering.
- The game backend is hosted as a collection of individual container-based microservices deployed into an Amazon EKS cluster in private subnets spread across Availability Zones for resiliency. Automatic scaling dynamically adjusts the capacity of the services and cluster nodes based on resource utilization, which is typically correlated with player demand. Whereas Cluster Autoscaler automatically adjusts the number of nodes in the cluster, the Horizontal Pod Autoscaler automatically scales the pods deployed into the cluster.
- Game and player data is stored in backend databases and caches that are deployed into private subnets across Availability Zones with replication between primary and replica nodes. Amazon Aurora is a popular choice for use cases such as player profiles, entitlements, and in-game purchasing, which can have more complex querying requirements and may benefit from the relational data modeling capabilities of MySQL and PostgreSQL. Amazon ElastiCache is useful for building high-performance leaderboards, pub/sub messaging, and for caching frequently accessed data to reduce latency and load on databases. Amazon DynamoDB is a fully managed NoSQL data store that is ideal for unpredictable access patterns and the ability to scale to virtually unlimited throughput for use cases such as player and game state data, session data, inventory and item stores, or use cases where you want a global database with minimal overhead.
- Asynchronous processing workflows should be used to perform work that can be done in the background, such as updating leaderboards or sending friend requests. Configure your game backend to push this type of work into Amazon SQS queues to scale as your game grows or consider using Amazon SNS topics to distribute the work between many consumer application queues for parallel processing. Use AWS Lambda functions to perform the processing in an event-driven manner to reduce your compute infrastructure costs and management overhead.

For workflows that are long-lived or require task coordination with multiple steps, consider orchestrating the entire workflow using AWS Step Functions. Amazon EventBridge can be used to initiate functions to respond to AWS service and custom application events.

Serverless-based game backend architecture

Many game developers do not want to manage infrastructure, and instead prefer to build their games using technologies that allow them to focus on software. A serverless architecture is recommended in this scenario because it allows you to build and release features more quickly, and with less operational overhead. Serverless architectures are designed using cloud services that can scale dynamically based on demand without needing to set up, manage, and scale servers. The following reference architecture illustrates how to build a game using a serverless architecture.


AWS Cloud
VPC
Public subnet
NAT Gateway
Private subnet
Lambda Security group
Multiplayer - Live Scoreboard Functions
CacheGame
JoinGame
SendQuestion
ReceiveAnswer
StartGame
EndGame
Redis Security group
CACHE
Redis Real-time Game and Scoreboard
IoT Core
Vue.js Application
WebSockets
HTTP
Cognito
Multiplayer - Casual and Competitive Functions
OnConnect
LiveAdmin
LivePlayer
OnDisconnect
Single Player Game Functions
ActiveGamesList
GamePlay
GameAnswer
Leaderboard Get
Content Creation, Administration, and Marketplace Functions
GameActivate
GameHeaderPut
QuestionPut
GameGet
ListMarketplace
GetMarketplace
ListMyGames
PlayerGet
PlayerPut
ProgressionGet
PlayerWalletAdd
PlayerWalletGet
PurchaseGame
Analytics
Kinesis
Kinesis Data Analytics
Athena
S3
DynamoDB Tables
Connections
Player Profiles
Games
Player Progress
Game Detail
Inventory
Marketplace
Player Wallet
ActiveGames
Leaderboard
Game Connections
Shared Services
PlayerWallet Topic
MoneyAdjust
Leaderboard Topic
ScorePut
PlayerProgress Topic
PlayerProgress Put

Serverless-based game backend reference architecture

This reference architecture illustrates a web-based trivia game that provides single player and multiplayer features.

- **Player authentication**: Players authenticate using Amazon Cognito, which provides secure authentication with a user directory for player identity management.
- **Game logic as serverless functions**: Game features and backend business logic run as AWS Lambda functions that are initiated in response to events, which keeps costs down because you only pay when the function runs. Lambda gives you the flexibility to write each game feature as a separate microservice using a programming language of your choice. For example, you might choose to develop .NET Lambda functions if you have experience using C# to build Unity games, or you might choose to develop Node.js Lambda functions if you desire to program a frontend and backend for a web-based game both in JavaScript.
- **NoSQL Data Store for game and player data:** Use DynamoDB to store your player and game data, as it is purpose-built for storing large amounts of data from microservices. As illustrated in this architecture, it is a best practice to use separate data stores for each game feature's data storage needs, which makes it straightforward for you to monitor and manage the features independently. This also creates boundaries of separation if feature or service ownership changes within your team. In this reference architecture, DynamoDB tables are used to store data such as connection state, game details, player progress, and leaderboard information.
- **Single player gameplay:** Single player features allow players to perform actions like selecting and playing a game and viewing the leaderboard. These features are implemented as RESTful backend services hosted with an Amazon API Gateway HTTP API that invokes the appropriate Lambda function to get and set data in DynamoDB tables. When gameplay completes, the backend also sends notifications to Amazon SNS topics that initiate Lambda functions asynchronously to store the player's progress and stats.
- **Multiplayer gameplay:** Multiplayer game features require the players to be able to interact with the game for point-to-point communication, as well as broadcast and receive updates from other connected players. A WebSockets implementation is suitable for point-to-point communication in a lightweight game, such as trivia. Players can establish a WebSockets connection to Amazon API Gateway WebSockets, which manages the connection and only invokes the Lambda functions when there are messages to send or receive for a player. For use cases where one-to-many communication is required between players, AWS IoT Core provides support for messaging using WebSockets over MQTT, which allows clients to subscribe to topics and act on the messages it receives. In this architecture, WebSockets over MQTT are used to support use cases such as

broadcasting live in-game updates and asking questions to connected players. As an alternative to AWS IoT, you might choose Redis Pub/Sub for message delivery or Redis Streams if you need message retention.

- **Use VPC-enabled Lambda functions to access resources in your private subnets:** Configure VPC-enabled Lambda functions to access resources in your VPC's private subnets, such as Amazon ElastiCache, which is used to reduce query times for low-latency datasets like live leaderboards.

For more information, see Guidance for Custom Game Backend Hosting on AWS.

Cloud game development (CGD)

Cloud game development (CGD) refers to the infrastructure and tools that are required for the game development lifecycle to build, test, and develop a game. Game development is collaborative between users and the infrastructure requirements frequently change throughout the stages of development.

Many game developers are embracing globally distributed and remote development teams, which requires technology that supports this type of development. Game developers can host all or part of these environments in AWS and use the global availability of AWS Regions to place resources closer to users and manage their development environments more cost-effectively by scaling compute and storage as needed.

The environments can vary depending on game developer needs, but they typically include developer workstations for artists, designers, engineers, QA testers, contractors, and other personnel to perform their work. These environments also typically include a build farm comprised of source code repositories for users to check-in their changes and the CI/CD infrastructure for building, packaging, and testing the developed artifacts.

These game production architectures have the following characteristics:

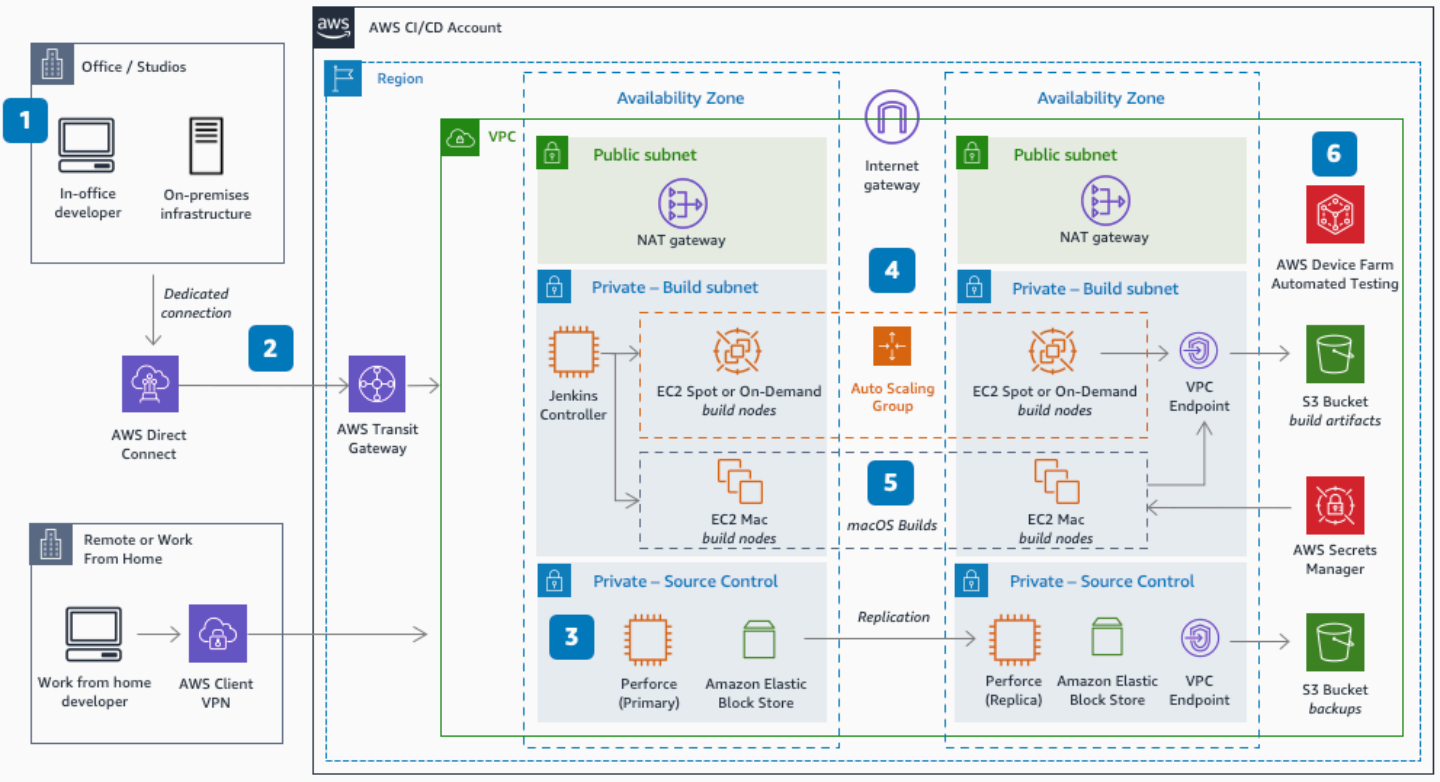
- Users should be able to access a virtual workstation through a web browser or local desktop client, such as Amazon DCV, that provides them with a low latency streaming session to access the same software and tools that they would have access to if they were working on a machine in an office or development studio. These virtual workstations, typically a cloud-based server, should allow a user to collaborate and work on their projects entirely in a cloud environment over a LAN or WAN. When users are not actively using the machines, their work should be

backed up to durable cloud storage, for example a source control repository or file system such as Amazon Elastic File System (EFS) and Amazon FSx, and their machine should be shut down to reduce costs.

- Source control repositories, such as Perforce, should be designed with high availability using replication between Availability Zones or between on-premises environments with backups stored in cloud storage such as Amazon S3. For example, a cloud-based Perforce server should include a primary commit server hosted in one Availability Zone with replication to a standby server hosted in another Availability Zone in the same Region.
- Game development build farm resources should be designed with automatic scaling so that compute resources are provisioned as they are required, and EC2 Spot Instances should be used to reduce the costs incurred when scaling out the number of servers required for builds.

Cloud game development: CI/CD

CI/CD infrastructure is important when developing games regardless of team size to improve iteration times, build reliability, efficient deployment, and better control over the development and release process to deliver a high-quality game experience to players. A game development CI/CD pipeline is typically comprised of highly available source control servers and storage, compute resources to run your builds, and software to perform automated testing, along with the proper network connectivity from your development machines. The following reference architecture demonstrates how to offload game builds from remote or on-premises game development environments to the AWS Cloud to aid developers in migrating or building new build farms.



Offload game builds to the cloud

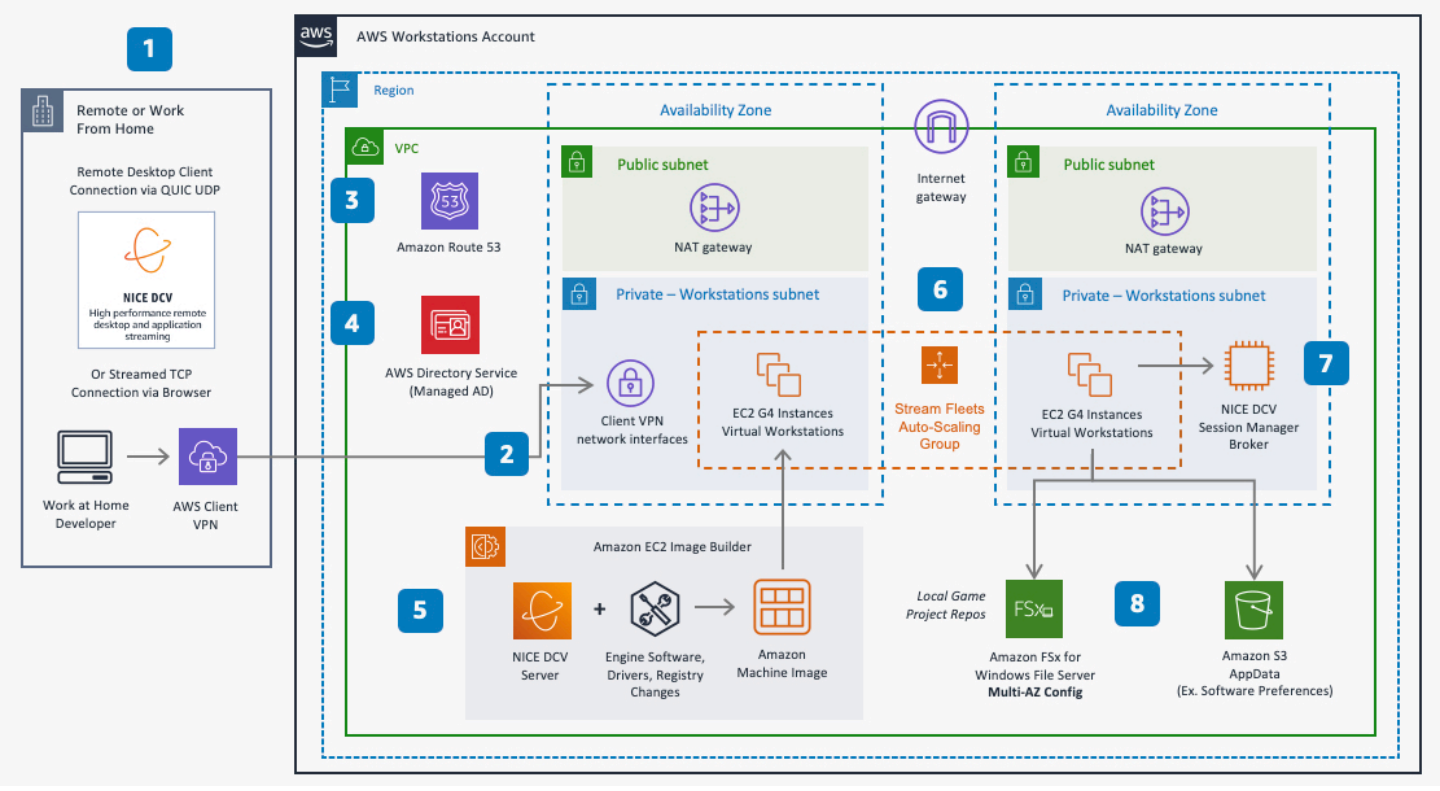
1. AWS Direct Connect provides a low latency, private, dedicated connection to AWS for in-office developers. Remote developers use zero-trust technologies like AWS Verified Access, or virtual private networks (VPN) like AWS Client VPN.
2. AWS Transit Gateway simplifies network management for connectivity between VPCs and from on-premises networks.
3. Perforce manages source and version control (CI) backed by Amazon EBS storage for quickly accessed, persistent data. Perforce Helix Core is available in the AWS Marketplace.
4. Commits start a build (CD) in Jenkins when developers push changes to Perforce tied to a branch. Perforce initiates POST a JSON payload to Jenkins. The Jenkins controller calls engine *headless* CLI commands to run and parallelize the build process across ephemeral, Docker nodes (such as Amazon EC2 Spot Instances or Amazon EC2 On-Demand Instances). Developers can increase availability by using two Jenkins controllers, one in each Availability Zone, behind a load balancer. For some game engines, developers might need additional licensing infrastructure configured in additional subnets to vend licenses for the build context each time a concurrent build is run.

5. The Xcode portion of iOS builds is offloaded to Amazon EC2 Mac instances to sign, build, and export the .IPA file, splitting the process and reducing build times. AWS Secrets Manager holds provisioning profiles, private keys, and certificates.
6. Build artifacts are delivered to Amazon S3, which sends notifications of success or failure. AWS Device Farm enables automated testing for mobile devices.

Cloud game development: Workstations

Game development often involves distributed teams working remotely from various locations, requiring access to shared infrastructure and the ability to support collaborative, geographically dispersed development. This trend towards a more distributed game development process, including the use of work-for-hire studios and remote work, necessitates the implementation of robust technologies and workflows to facilitate productivity, shared expertise, and agile development practices.

The following reference architecture demonstrates how to use AWS for hosting remote game development workstations using the Amazon DCV protocol.



Stream game development from anywhere with Amazon DCV

1. Amazon DCV is a streaming protocol that supports 4K, 60-FPS streaming. Developers using a browser connect through TCP connections, whereas desktop clients can use QUIC UDP over port 8443 for increased performance.
2. Developers use AWS Client VPN for a secure connection to network interfaces in the workstation subnets with source network address translation (SNAT).
3. Amazon Route 53 provides private DNS for the resources in the VPC as well as inbound and outbound DNS forwarding.
4. Directory Service provides managed Microsoft Active Directory to enable local game project storage mapped to individual users.
5. Workstations are created using an Amazon Machine Image (AMI) built with Image Builder. Images include Amazon DCV Server, developer software, registry changes, and drivers such as the NVIDIA gaming drivers or peripheral drivers. AWS Marketplace includes common AMIs used for workstations.
6. Fleets of workstations use graphics Amazon EC2 instance types that provide GPUs, and are scaled using EC2 Auto Scaling groups.
7. The Amazon DCV Session Manager Broker enables management of Amazon DCV sessions.
8. Local file storage of projects is hosted in Amazon FSx for Windows File Server. Developers commit to a separate CI/CD pipeline by pushing from local storage to source control.

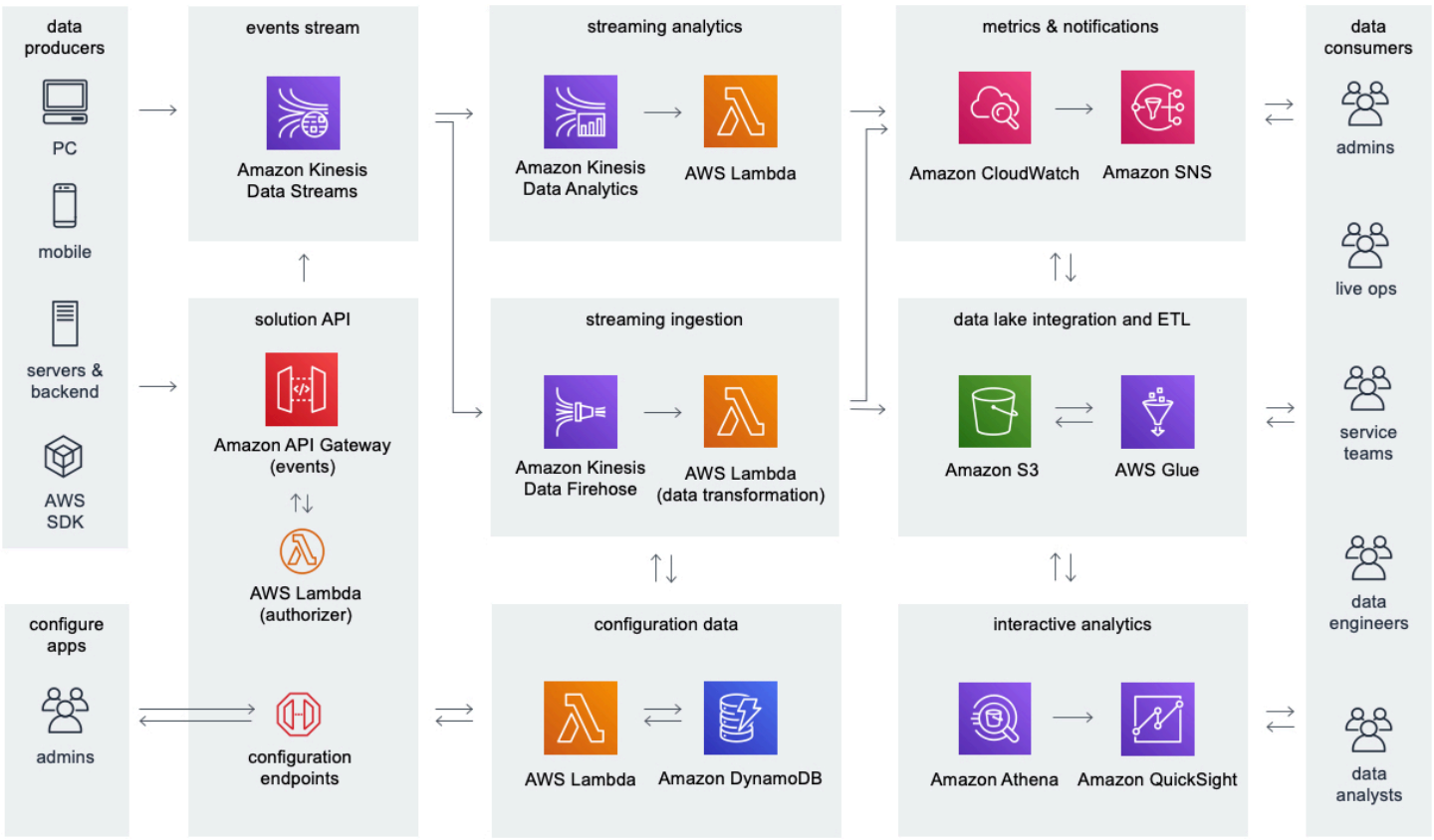
Game analytics pipeline

Game developers are increasingly looking for ways to better understand player behavior so that they can improve the gameplay experience to retain and grow their player base. Game analytics represents the technical infrastructure and processes that is required to understand and analyze the data that is generated from the game and related services. This typically requires the use of an analytics pipeline architecture that can support this end-to-end process, such as the Game Analytics Pipeline solution implementation.

Game analytics architectures have the following characteristics:

- Data sources send data in a common format such as JSON and typically include game servers and game backend services, as well as game clients, including PC, mobile devices, and game consoles.

- A game analytics pipeline automates the entire workflow of ingesting and storing the raw data and processing it into usable output formats so that it can be analyzed efficiently and cost effectively by data consumers, such as end users and analytics applications.
- Game analytics pipelines provide support for ingesting and processing high volumes of real-time data to scale as a game grows.
- Provide support for both real-time and batch reporting use cases. For example, real-time dashboards and alerts are typically used by live ops teams to monitor game infrastructure and player behavior to detect issues. Data analyst teams typically rely on as-necessary and batch reporting to understand trends over time.



Serverless game analytics pipeline for gameplay telemetry

Game data is ingested from game clients, game servers, and other applications. The streaming data is ingested into Amazon S3 for data lake integration and interactive analytics. Streaming analytics processes real-time events and generates metrics. Data consumers analyze metrics data in Amazon CloudWatch and raw events in Amazon S3.

- **Solution API and configuration data:** Use Amazon API Gateway to provide a REST API for administering your game analytics pipeline and storing configuration data in Amazon DynamoDB using Lambda functions. You can build an internal portal on top of this API or a custom command line interface for administration. A REST API also provides server-authentication for ingesting gameplay data from data sources and forwarding the telemetry data to Amazon Kinesis Data Streams for real-time processing and ingestion into storage.
- **Events stream:** Amazon Kinesis Data Streams captures streaming data from your game and allows real-time data processing by Amazon Data Firehose and Amazon Managed Service for Apache Flink.
- **Streaming analytics:** Managed Service for Apache Flink analyzes streaming event data from the Kinesis Data Streams and can generate custom metrics and alerts that are published to CloudWatch using Lambda functions.
- **Metrics and notifications:** Use Amazon CloudWatch to monitor your solution's metrics, logs, and alarms. Use Amazon SNS for sending notifications to on-call engineers and other data consumers.
- **Streaming ingestion:** Use Firehose to consume your streaming data from Kinesis Data Streams and deliver it to your data lake in Amazon S3 for long-term storage, transformation, and integration with other data.
- **Data lake integration and ETL:** Use AWS Glue for ETL processing workflows and to organize your metadata in the AWS AWS Glue Data Catalog, which provides the basis for a data lake for integration with flexible analytics tools.
- **Interactive analytics:** End users can use Amazon Athena to perform ad-hoc interactive queries on the datasets stored in Amazon S3, and Quick can be used to build dashboards.

Refer to the Game Analytics Pipeline for an automated reference implementation of an analytics pipeline that can be deployed into your account using AWS CloudFormation.

Definitions

The AWS Well-Architected Framework is based on six pillars: operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability. AWS provides multiple core components that allow you to design state-of-the-art architectures for your game workloads. In this section, we will present an overview of key definitions.

For the purposes of this paper, a game architecture encompasses the backend technical infrastructure required to build and operate a game. Some games may not have social, multiplayer, or other online features and may not require the use of certain aspects of backend technical infrastructure that are described in this paper. For a detailed discussion of the different types of workloads that are frequently deployed to support a game architecture, see Scenarios.

The AWS Cloud infrastructure is built around Regions and Availability Zones.

- A *Region* is a physical location in the world where we have multiple Availability Zones.
- *Availability Zones* consist of one or more discrete data centers, each with redundant power, networking, and connectivity, housed in separate facilities.

Depending on the characteristics of your game, you may want to deploy certain components of your game architecture into multiple Regions for reasons such as improving performance for players, or to deliver customized experiences for players depending on their location.

There are many different types of games, and the backend technical infrastructure required to support a game differs depending on the type of game being developed. For example, popular types of games may include first person shooters (FPS), role playing games (RPG), massively multiplayer online games (MMOG), battle royales (BR), sports games, puzzle games, and more. There are also different game interaction modes that influence the architecture of the game, such as turn-based and simultaneous play, with different performance characteristics.

Games are developed to be played on one or more gaming systems including desktop, web, mobile, consoles, and newer interaction modes such as augmented reality (AR), virtual reality (VR), and game streaming solutions. Games commonly support cross-system gameplay, which means that players can save their game progression and resume gameplay on other systems, as well as initiate gameplay sessions with players on other systems.

Video game monetization enables game publishers to generate revenue using different strategies like advertising, digital and retail-based game purchases, in-game purchases of downloadable

content (DLC) known as microtransactions, and through required paid subscriptions to play the game. Some of the most common key performance indicators (KPIs) in the games industry include:

- Daily active users (DAU)
- Monthly active users (MAU)
- Concurrent users (CCU)
- Session duration
- Cost per install (CPI)
- Player lifetime value (LTV)
- Average revenue per user (ARPU)

Gaming system

Video games are developed to be played on a gaming system that provides client input controls, graphics, client software (known as the game client) and hardware, and in some cases system-exclusive features to support gameplay.

Gaming systems are generally separated into the following categories:

- **Consoles**: Purpose-built entertainment systems that are designed for playing games, including popular examples such as the Sony PlayStation, Microsoft Xbox, and Nintendo Switch. Consoles provide the ability to play games by installing physical or digitally distributed game content onto the console hardware that is manufactured by the gaming system provider. In this definition, a console may be handheld or stationary and intended to be used in a home entertainment scenario.
- **Personal computer (PC)**: Games that are played using computer software that is installed onto a client machine that can be customized by the player. For this reason, PC gaming is popular among players because of the flexibility and control that it provides.
- **Web**: Games that are designed to be played using a web browser and which usually provide the benefit of enabling a player to access the game irrespective of their operating system.
- **Mobile:** Games that are developed to be played on a mobile phone, usually a smartphone operating system. Mobile games are usually downloaded from a digital app store and installed onto the phone.

In addition to the previously mentioned systems, there are also nascent systems that are still relatively new and growing and have much smaller market share compared to the more predominant systems. Examples of gaming systems in this category include AR, VR, and game streaming, sometimes referred to as cloud gaming.

Game streaming involves rendering the gameplay in the cloud and streaming to thin client, typically a browser. Game streaming allows a player to play a game that is entirely hosted remotely, typically in the cloud by a game streaming service provider. In game streaming, the player connects to a cloud-based game through a browser or a thin client provided by the cloud gaming service provider (gaming system).

Game server

Game servers represent one of the most important aspects of the compute infrastructure for your game. Game servers, sometimes referred to as dedicated game servers, are used when developing a multiplayer game or when server authoritative processing of gameplay events is required. The game server is at the center of the game architecture, serving as the location where the core logic runs, which includes managing player and game state as well as managing the interactions between the connected game clients and the game server. The game server is usually one of the most performance-sensitive aspects of a game architecture because it is responsible for processing the inputs from a player's game client and properly distributing it to other connected players in real-time. A poorly performing game server impacts the overall performance of the game experience. Therefore, you should optimize game server performance and provide sufficient capacity, especially at game launch or peak gameplay periods.

For the purposes of this document, a game server or game server instance refers to the compute, such as a virtual machine, that hosts one or more game server processes. A game server process represents a single instance of your game server build hosting a game session, which is an instance of your running game that players can connect to through a player session. For this reason, we often refer to game server process or game session interchangeably due to the implied one to one relationship between a game session and the game server process that is hosting it. In AWS, there are multiple options for compute to host game servers, which provide access to scalable cloud-based capacity through elastic provisioning of resources.

Amazon EC2 provides cloud-based virtual servers, known as instances, with support for multiple versions of Linux and Windows. You can create instances and manage them directly like another server or virtual machine. Typically, multiple game server processes are deployed to an instance to

improve efficiency and reduce costs. Amazon EC2 is a good choice for game servers if you desire the most control over the compute infrastructure.

Amazon GameLift provides a fully managed solution for dedicated game server hosting in the cloud as well as additional features such as matchmaking with GameLift FlexMatch. GameLift provides a layer of abstraction on top of Amazon EC2 to make game server management straightforward and is available in most AWS Regions so that you can host game servers close to players to reduce latency, achieve high availability, and significantly reduce costs by using Spot Instances. While GameLift can be integrated into existing game backends, it is especially useful for game developers who do not want to develop their own game server management and matchmaking solutions and prefer a solution that is managed by AWS and can scale as their game grows.

Amazon Elastic Container Service (Amazon ECS) is a fully managed container orchestration service that you can use to run Docker-based containers. You can also use Amazon Elastic Kubernetes Service (Amazon EKS) to run Docker-based containers that are built using Kubernetes. Using container technologies such as those provided by Amazon ECS and Amazon EKS can assist you to improve your compute utilization by efficiently packing many game server processes or other game application instances into an EC2 instance.

The use of containers can also improve developer productivity by hosting applications using the same Docker image operating runtime used by developers on their local machines during development. You can further reduce operational overhead by using AWS Fargate, which is a serverless compute solution for running containers and is compatible with both Amazon EKS and Amazon ECS. Fargate is best suited for use cases where you want to run game servers in containers without responsibility for operating the underlying instances that the containers run on.

You can use AWS Outposts to run AWS infrastructure and services in a data center or on-premises facility, which can enable games to run in on-premises environments and AWS using the same infrastructure to support a hybrid cloud adoption strategy. AWS Local Zones serve as extensions of AWS Regions that allow your game servers and other latency-sensitive workloads to run more closely to your players or development teams. Additionally, to reduce global network latency for your game servers, you can use AWS Global Accelerator to improve performance for player traffic to your game servers.

AWS Lambda is a serverless compute service that runs code without provisioning or managing servers, which makes it useful for asynchronous game server use cases such as turn-based games or those that have lightweight compute requirements, a small codebase, and where gameplay functionality can be designed using a stateless microservices architecture. It is important to keep in

mind that Lambda functions run on an event-driven, per-request basis, rather than running a long-running game server process. Lambda provides the most runtime abstraction of the options in this paper because the underlying application is readily available for developers to choose from to host their code.

When selecting your approach for game server hosting, consider various requirements including operational overhead, legacy codebases, performance requirements, and scale. EC2 instances and containers are good options for legacy codebases, as they require the smallest change to move to the cloud, and you can use EC2 instances to dedicate the resources of a compute instance, while containers can make management and high utilization simpler to achieve. Serverless functions offer the highest level of abstraction, which you can use to define code that only runs in response to events, which can reduce costs.

Game client

The *game client* represents the software and hardware device that the player uses for playing a game. The game client provides the software for translating the player's inputs into messages that are sent to a server for processing, and it is responsible for handling the incoming responses from the server and rendering outputs such as graphics to the player. In real-time networked multiplayer games, the game client usually maintains a persistent network connection to a game server for the duration of a gameplay session to reduce network latency and minimize processing time. However, the game client may also interact using REST with a game server or backend services.

Messaging

There are typically three primary categories of messages in games:

- **Player engagement messaging** targeted at a specific user or cohort of users, such as game invites or push notifications
- **Group messaging** between players such as in-game chat
- **Service-to-service** messaging, such as JSON messages used to integrate two or more applications

A common strategy for sending and receiving these types of messages is to use publisher-subscriber and asynchronous processing architecture patterns. AWS provides several services that can assist you to implement messaging in your game.

- **Amazon Simple Notification Service (SNS):** Managed service for delivering messages between publishers and subscribers using a pub/sub architecture pattern. Publishers send messages using an API to Amazon SNS, which delivers the messages asynchronously to subscribing applications and can deliver push notifications directly to mobile clients or desktops with support for some of the most widely used push notification services. Amazon SNS can be used for push notifications to clients as well as service-to-service messaging use cases.
- **Amazon Simple Queue Service (SQS)**: A fully managed queue service that makes it straightforward to integrate game servers and your game regardless of the programming language used in each. Many games tasks can be decoupled and handled in the background such as updating a leaderboard or playtime values in a database. This approach is an effective way to decouple various parts of your game and independently scale the player-facing features from backend processing.
- **Amazon Managed Streaming for Apache Kafka (MSK)**: A fully managed service that simplifies building data streaming and producer or consumer applications using Apache Kafka, a popular open-source solution. Kafka is typically used for ingesting and processing real-time streaming data and can be used for service-to-service messaging.
- **Amazon ElastiCache (Redis OSS):** Provides a fully managed in-memory data store which includes support for the popular pub/sub feature of Redis that is commonly used for developing chat room applications and high-performance service-to-service messaging. Redis also supports rich data types such as lists and sets so that developers can use Redis for high-performance queuing.
- **Amazon Pinpoint:** Provides user engagement messaging through email, SMS, voice, and push notifications. For example, Amazon Pinpoint can be used to deliver user engagement messages to players to invite them back to the game and can be used for transactional use cases such as supporting multi-factor authentication tokens, order confirmations, and password reset emails.

Live game operations (Live Ops)

Live game operations (Live Ops) is a style of game management and operations that treats a game as a live service and continually delivers new features, updates, promotions, in-game events, and improvements to the launched game to improve the experience for the player community.

Traditionally, games were delivered as products rather than services, and new content and features were frequently incorporated into subsequent releases or sequels rather than into the launched product. With a Live Ops approach to game management, a game operations team can launch a

game and maintain an engaged player community through experimentation, promotions, in-game events, and innovation to keep players entertained.

Although this approach has the benefit of unlocking new player engagement strategies and delivering recurring revenue streams, it requires more operational expertise. For example, to implement a successful Live Ops strategy, a developer may need to integrate with cloud services or operate their own backend technical infrastructure. They also need an effective way to identify and respond to issues that arise in the game, or within the player community, that can negatively impact the player experience.

Operational excellence

The operational excellence pillar focuses on best practices for deploying and operating cloud-based games at scale. It is important to focus on operational excellence to maintain a positive player experience and implement preventative measures to prepare for and recover from issues that impact their experience.

Focus areas

Design principles

In addition to the design principles from the Well-Architected Framework whitepaper, the following design principles can assist you to achieve operational excellence in building and operating games:

- **Define measurable and achievable objectives for game operations teams and adapt as necessary:** Due to the hits-driven nature of games, it is difficult to determine ahead of time how many players will play your game when it launches or what expectations players will have for your ongoing game operations. It is important to set ambitious but achievable goals with stakeholders and design an approach that can be scaled up if your game exceeds projections and scaled down while game development teams optimize the player experience. Adequately prepare and test ahead of time to meet these requirements and align your business and technical stakeholders on target objectives for operating the game. With targets defined, the game teams can achieve an appropriate balance between cost and performance during planning, designing, provisioning, testing, deploying, and operating the game backend infrastructure.

- **Use operational runbooks to plan for scaling activities related to game launches and special events:** Game operations teams should coordinate with business stakeholders to model projections for anticipated peak player concurrency for events and perform proactive planning to pre-scale infrastructure capacity ahead of time. Due to the fluctuating nature of player traffic during events, prior planning and pre-scaling activities should augment your existing automated scaling systems to improve your chance of success during an event and verify that you have enough resources to provide a positive player experience. Implement performance engineering practices to develop a baseline of your resources and a data-driven understanding of your system's capacity, which will assist guide pre-scaling activities and automated scaling configurations. Develop operational runbooks to provide consistency in the process. This advance planning and responsiveness to player demand is especially critical for live service games, which must maintain reliable performance and infrastructure to support an active, engaged player base over an extended timeframe.
- **Establish an operating model for receiving, investigating, and responding to player support requests:** After a launch, monitor reports of complaints and issues with the game. Implement appropriate systems to interact with players in a secure and effective manner to adequately resolve player issues and such as community forums, social media, e-mail, ticketing systems, call centers, or automated chat bot solutions. This is especially crucial for live service games, which require ongoing communication with the player base, responsiveness to player feedback to address evolving needs, and maintenance of an engaged community over an extended lifespan.

Live operations

GAMEOPS01: How do you define your game's live operations (Live Ops) strategy?

Develop a live operations (Live Ops) strategy for your game based on defined objectives and performance metrics, in consultation with business stakeholders.

Best practices

- GAMEOPS01-BP01 Use game objectives and business performance metrics to develop your live operations strategy

GAMEOPS01-BP01 Use game objectives and business performance metrics to develop your live operations strategy

Consult business stakeholders, such as game producers and publishing partners, to determine objectives and performance metrics for a game. This can assist you develop plans for how you will manage the game, including defining your maintenance windows, software and infrastructure update schedules, and system reliability and recoverability goals.

Level of risk exposed if this best practice is not established: High

Implementation guidance

These metrics can also assist you determine at which stage of your game's lifecycle you should incorporate a live operation steam (Live Ops) to monitor game health, collect direct game feedback, and build streamlined and automated release processes. For example, a new game might wait until a certain scale is achieved, measured by active player count, revenue, or another set of metrics, before setting up a dedicated live operations team. An established game development studio might already have live operations experience, perhaps for their other games, so they'd only need to onboard the new game.

Implementation steps

- You may define the targets for player concurrency (CCU) and daily and monthly active users (DAU and MAU) that the game infrastructure should be capable to effectively support, your infrastructure budgets, financial targets, and other performance goals, such as the frequency for release of content and features to increase player engagement. These objectives and metrics feed into decisions about the game design, release management, observability, and support that is needed for efficient operations.
- Your game might have an objective to release new content updates at least once each month with no downtime during release. This information assists you to define your release deployment strategy and coordinate the scheduling of required maintenance that may require downtime at other times throughout the month and contribute towards your availability SLA.

Account structure

GAMEOPS02: How do you structure your AWS accounts for hosting your game environments?

Implement a multi-account strategy to isolate different game environments and enhance security, operational efficiency, and scalability. Use AWS Organizations to manage account hierarchies, apply guardrails to accounts, and enforce tag policies and tagging on deployed resources.

Best practices

- GAMEOPS02-BP01 Adopt a multi-account strategy to isolate different games and applications into their own accounts
- GAMEOPS02-BP02 Organize infrastructure resources using resource tagging

GAMEOPS02-BP01 Adopt a multi-account strategy to isolate different games and applications into their own accounts

Design an account structure that would guide the infrastructure deployment to comply to each environment's security, isolation, and operational needs. Environment isolation by restricting access to it and permitting only requisite AWS services to be used in them is essential, with production environments being locked down, while development and testing environments are lenient to permit experimentation. Further isolation of major sub-systems in each environment, and common services that are used by multiple environments to be hosted and managed out of their own AWS accounts is highly recommended.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Adopt a multi-account strategy on AWS by isolating the different environments (like development, test, staging, production, and shared services) to individual AWS accounts, which reduces the scope of incidents. Consider AWS Organizations to centrally manage the hierarchy of your AWS accounts to further simplify operations, as well as define and apply account-level and organizational unit-level (OU-level) policies selectively. By designing an appropriate OU and AWS account structure that is aligned to your development and productional workflow needs, you can optimize your costs and enhance scalability.

- **Adopt a multi-account strategy:** Isolate environments to reduce incident radius and simplify operations.
- **Use AWS Organizations:** Manage accounts hierarchically, apply policies, and enable centralized governance.

- **Plan for Scalability:** Design fine-grained account structures and implement cost-saving measures for future growth.

Implementation steps

A game system deployed in AWS should use multiple accounts that are logically organized to provide proper isolation, which reduces the blast radius of issues and simplifies operations as your game infrastructure scales. AWS accounts that host game infrastructure are typically grouped into the following logical environments:

- **Game development environments** are used by developers for developing the software and systems for the game.
- **Test or quality assurance (QA) environments** are used for performing integration testing, manual QA, and other automated testing that must be conducted.
- **Staging or pre-production environments** are used for hosting completed software so that load and smoke testing can be conducted prior to launching to production.
- **Live or production environments** are used for hosting the live software and infrastructure and serving production traffic from players.
- **Shared services or tools environments** provide access to common systems, software, and tools that are used by many different teams. For example, a central self-hosted source control repository and game build farm might be hosted in a shared services account.
- **Security environments** are used for consolidating centralized logs and security technologies that are used by teams that focus on cloud security.

For game infrastructure on AWS, it is recommended to create separate accounts for each game environment (development, testing, staging, and production), as well as accounts for security, logging, and central shared services.

Typically, smaller game development studios that manage a limited number of infrastructure resources, usually a few hundred servers or less, may create one AWS account for each of these environments (for example, one production account, one development account, and one staging account). However, as your game infrastructure or team size grows over time, this simplified model may not scale well.

When setting up these environments, consider that many AWS services share resource and API-level Service Quotas for an entire account within a particular Region. This must be considered when determining how to logically organize accounts. AWS accounts only incur cost for consuming

services deployed into them. Therefore, this provides a way to effectively reduce resource contention and service quotas, particularly as your game grows and more developers need access to build and manage resources.

Based on our experience working with larger game development studios that typically operate thousands of servers with hundreds of developers accessing resources, we recommend you design a more fine-grained account structure where individual applications supporting your game have their own development, testing, staging, and production accounts. Because it's difficult and time consuming to re-design your AWS multi-account strategy after you have launched your game due to the complexity in planning and migrating live systems, consider your future scaling needs when determining the right multi-account structure.

You can use AWS Organizations to set up a hierarchy and grouping of AWS accounts, and define organizational units (OUs) to apply common OU-level policies to them through service control policies (SCPs). AWS Organizations centrally manages and governs your environment as you grow and scale your resources. You can programmatically create new accounts and allocate resources, group accounts to organize your workflows, apply policies to accounts or groups for governance, and simplify billing by using a single payment method for your accounts. Additionally, Organizations is integrated with other services so that you can define central configurations, security mechanisms, audit requirements, and resource sharing across accounts in your organization.

AWS Control Tower provides a straightforward way to set up and govern a secure, multi-account environment, called a *landing zone*. Control Tower creates your landing zone using AWS Organizations, bringing ongoing account management and governance as well as implementation best practices based on AWS's experience working with thousands of customers as they move to the cloud. AWS Config , AWS Trusted Advisor, and AWS Security Hub CSPM are services that provide an aggregated or centralized view of your account's hygiene.

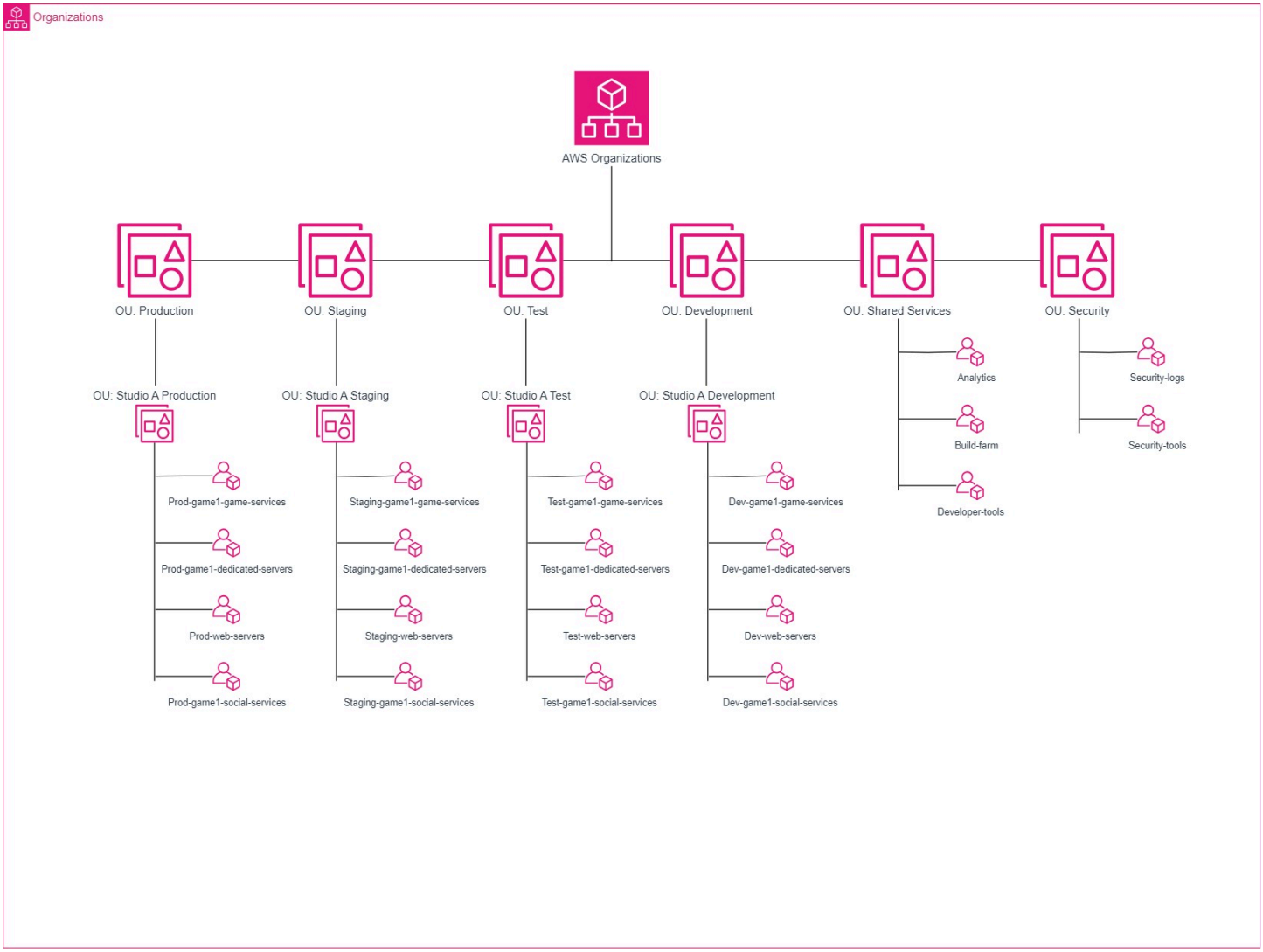
This isolation assists you to set up custom or individual permissions and guardrails to each game environment. Production accounts should have the necessary guardrails, access restrictions, monitoring and alerting, and security tools, while non-production accounts may not require the same level of guardrails and permissions. Non-production environments can be automated to shut down resources after hours and save costs. Separation of accounts at this level of granularity makes it straightforward to monitor infrastructure costs for each of the environments supporting a game.

The following is an example of a multi-account structure for a game company using AWS Organizations and organizational units (OUs) to logically group AWS accounts into separate

environments and studios. In this example, OUs are used to group together accounts based on their environment and then based on the studio that operates the environment. This demonstrates how you can create a nested hierarchy to allow separate applications and games to be deployed into their own accounts within their environment (depicted as OUs), which can be useful if you develop and operate multiple games. Refer to the documentation and whitepapers provided in the resources section of this pillar to learn about additional strategies that you can consider for organizing your multi-account strategy.

Based on the discussion above, the sample diagram below assumes a game studio (Organization) that has a development pipeline comprised of 4 stages (development, testing, staging, and production). For a given game (game1), each of the environments (OU) has individual AWS accounts for game services, dedicated game servers, social services, and web servers. The resources that run in each AWS account are relevant to the respective sub-systems. Typically, every individual game using this kind of development pipeline would replicate this or a similar structure for its AWS accounts.

In addition to these game-centric environment OUs, there are also the shared services OU and security OU. These OUs should be organization-wide, not for each individual game. That way the games would consume the shared services for development tools and data and analytics as in this example. Then, send application and system logs to the AWS account set up for logs in the security OU.



Example of account structure for game environments

GAMEOPS02-BP02 Organize infrastructure resources using resource tagging

To effectively manage and track your infrastructure resources in AWS, use proper resource tagging and grouping to identify each resource's owner, project, application, cost center, and other data. Tagged resources can be grouped together using resource groups, which assists with operational support.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Define tagging policies. Typical strategies include resource tags for identifying the resource owner, such as team name or individual name, the name of the game, application, or project, the studio name, environment (like development or production), and the role of the resource (such as database server, web server, dedicated game server, app server, or cache server). You can add other tags to assist with business and IT needs. AWS Config can also enforce a tagging policy at resource creation and update time. Tags and resource groups are available from the AWS Management Console, the AWS CLI, and API operations.

Implementation steps

- Tag resources to identify their owner, project, app, cost center, and other relevant data.
- Implement tagging policies including tags for owner, project, studio, environment, and resource role.
- Use AWS Config to enforce tagging policies, and manage tags through AWS Management Console, CLI, and API.

Game deployments

GAMEOPS03: How do you manage game deployments?

Manage game deployments by thoroughly validating reused components, conducting regular performance engineering, and implementing regular load testing throughout the development lifecycle.

Best practices

- GAMEOPS03-BP01 Validate and test your existing core game systems and infrastructure before reusing it in your game
- GAMEOPS03-BP02 Conduct performance engineering before every release (or at least for major releases)
- GAMEOPS03-BP03 Load test early and often
- GAMEOPS03-BP04 Adopt a deployment strategy that minimizes impact to players
- GAMEOPS03-BP05 Pre-scale infrastructure required to support peak requirements

GAMEOPS03-BP01 Validate and test your existing core game systems and infrastructure before reusing it in your game

Organizations tend to reuse existing components and source code from previous games to save on development time and cost. These legacy components and code may not be subjected to a thorough review or have detailed integration testing and instead rely on their past performance.

Level of risk exposed if this best practice is not established: High

Implementation guidance

While reuse assists improving productivity, it can also introduce the risk of reintroducing past performance and stability issues into a new project. Therefore, when reusing existing components and source code from previous games, robust testing should be implemented.

Implementation steps

- **Identify reused code and components:** Catalog the source code, libraries, and components being reused from previous games. Clearly distinguish between actively maintained and deprecated code
- **Document original behavior and known issues:** Record the original performance characteristics, functional limitations, and known bugs or production incidents associated with the reused components.
- **Perform a thorough code review:** Conduct a detailed technical review of the reused components, especially those that had issues in the past or are poorly documented.
- **Replace or refactor high risk legacy components:** Prioritize replacing or updating legacy components that have a history of issues or are no longer maintainable, rather than relying on workarounds in production.
- **Conduct integration and compatibility testing:** Validate the reused components within the context of the new game's systems. Verify that they interact properly with new modules, tools, and APIs.

GAMEOPS03-BP02 Conduct performance engineering before every release (or at least for major releases)

Performance engineering is the process of monitoring multiple key operational metrics of an app to discover optimization opportunities that can further improve the application's performance.

This is an iterative process that starts with testing, followed by optimizing code, its dependencies, associated processes, its host operating system, and the underlying infrastructure.

Level of risk exposed if this best practice is not established: High

Implementation guidance

To conduct a deeper analysis of the app's performance, integrate an application performance monitoring (APM) or debugging tool in the app code that can isolate issues and reduce troubleshooting time by tracking its behavior for anomalies across the flows of the app. APM tools are also able to identify slow performing methods and external operations.

AWS X-Ray assists developers with their performance engineering activities, like identifying performance bottlenecks and analyzing and debugging production errors. You can use X-Ray to understand how your application and its underlying services are performing and identify and troubleshoot the root cause of performance issues and errors. Through numerous rounds of load tests, in which the application and its infrastructure is gradually loaded with synthetic player traffic, various system bottlenecks, app errors, exceptions, OS problems, and other issues are identified that may have not been found during other QA tests.

For critical events like game launches, content releases, promotions, and major in-game events, use AWS Countdown, which provides implementation guidance based on playbooks built by games experts to verify operational readiness, mitigate potential risks, and plan for capacity needs. AWS Countdown also has a premium support option that offers enhanced support and options like engineers to optimize your infrastructure.

Implementation steps

- Performance engineering involves evaluating and monitoring key operational metrics to verify that your application's code, processes, operating system, and infrastructure are functioning as expected. Pre-production review also assists to define baseline performance at different levels of simulated usage.
- Discover and track key metrics like utilization, services, I/O, processes and such by using system tools such as sar, top, vmstat, sysstat, netstat, and Performance Monitor.
- Track your application's performance and behavior using APM tools like AWS X-Ray to isolate issues, identify bottlenecks, and debug production errors.
- For critical events like game launches, subscribe to AWS Countdown (IEM) for architectural and operational guidance, on-demand operational support, and to identify risks and plan mitigations.

GAMEOPS03-BP03 Load test early and often

Load testing is the process of simulating real-world traffic on a system to assess its reliability and performance.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Load testing is a key factor in developing a performance baseline for your resources and understanding your system's capacity, which can guide financial forecasting, architecture design, resource allocation, automated scaling configurations, and post-launch pre-scaling activities. Additional benefits include:

- **Optimized infrastructure:** Resources might be over or under-provisioned. Understanding the resources needed will result in lower costs and less infrastructure to manage.
- **Scalability readiness:** Certain mechanisms and features can drive users into a game quickly. Knowing when and how to scale can be the difference between appropriately meeting the increased demand and losing players. Use load test results to prepare runbooks with system thresholds, alert points, and critical alert points at different scaling levels.
- **Higher quality code**: Issues such as excessive crosstalk between services, unbatched database calls, inefficient algorithms, memory leaks, and service degradation issues are sometimes simpler to identify at scale.
- **Behavior validation:** Injecting different kinds of failures into your tests can validate the system's expected behavior or uncover error-handling issues that need to be corrected.

Ideally, developers should perform load testing at multiple points throughout the development process, as each can yield different benefits: Early on, they guide architectural decisions and refactoring efforts while it's cheaper and straightforward to make changes. At the end of each sprint or iteration, they validate the application's performance with the latest features and functionality.

Prior to deploying to production, large-scale load testing simulating expected real-world usage patterns confirms the system's ability to handle the production workload. After the deployment, periodic load tests monitor the system's performance and identify changes or bottlenecks that may arise over time.

To simulate player traffic, you need lightweight clients or bots that emulate the game client flows and transact with the game backend to simulate real-world player behavior. This data is generally captured through game play logs and data generated by human-driven QA tests, as well as through real-world limited scale alpha or beta tests where real players are invited to play an early-access build of the game.

It is important to record the system's behavior in an operational runbook to assist in troubleshooting possible failures in the future and to retain performance metrics that future load tests can be compared against. It is also recommended to have human QA personnel test the game while it is being load tested as they might discover issues that bots fail to identify and metrics do not reflect.

AWS Fault Injection Service is a fully managed service for running fault injection experiments that make it straightforward to improve an application's performance, observability, and resiliency. Fault injection experiments are used in chaos engineering, which is the practice of stressing an application in testing or production environments by creating disruptive events, such as sudden increase in CPU or memory consumption, observing how the system responds, and implementing improvements. Fault injection experiments assist teams to create the real-world conditions needed to uncover the hidden bugs, monitoring blind spots, and performance bottlenecks that are difficult to find in distributed systems.

Implementation steps

- Set up a distributed load testing environment using Guidance for Kubernetes-Bases Game Load Testing.
- Customize and deploy Locust control and worker pods within the EKS cluster using the provided deployment files, enabling scalable and manageable load generation.
- Record system behavior and metrics during load testing in an operational runbook to assist with future troubleshooting and establish performance baselines.
- Use fault injection experiments to simulate real-world disruptions and uncover hidden issues in system performance, observability, and resilience.

GAMEOPS03-BP04 Adopt a deployment strategy that minimizes impact to players

Incorporate a deployment strategy for your game software and infrastructure that minimizes the amount of downtime that keeps players out of your game. While certain types of updates might

require installing new updates to the game client, design the game to minimize or avoid the need for downtime during deployments.

Level of risk exposed if this best practice is not established: High

Implementation guidance

One of the most important steps to consider when developing a game deployment strategy is to determine how your game infrastructure will be managed. Manage your game infrastructure using an infrastructure as code (IaC) tool such as AWS CloudFormation or Terraform by Hashicorp to reduce human errors during environment preparation. Infrastructure templates can be deployed and tested in automated pipelines, which creates consistency in the configuration of different game environments.

There are several deployment strategies that can be used for a game:

Rolling substitution

The primary objective of a rolling substitution for deployment is to perform the release without shutting down the game and without impacting players. It is important that the upgrade or changes that are to be performed are backward compatible and will work adjacent to the previous versions of the system.

In this deployment, the server instances are incrementally replaced (substituted or rolled out) by instances running the updated version. This rolling substitution can be performed in a few different ways. For example, to implement rolling updates to a fleet of dedicated game servers, a typical approach involves creating a new Auto Scaling group of EC2 instances that contain the new game server build version deployed onto them, and then gradually routing players into game sessions hosted on this new fleet of servers. If there is an associated game client update that is required as a prerequisite to use the new game server build, then you must include a validation check to verify that only players that have this new game client update installed are routed into these game sessions.

Server fleets (for example, EC2 Auto Scaling groups) containing the old game server build version are only removed from service after they are drained of active player sessions in a graceful manner, typically by setting up individualized-server metrics that allow game operations teams to automate this process. Alternatively, to reduce the amount of infrastructure and time to conduct a rolling deployment, an alternative approach can be performed where existing production instances are removed from service, updated with the new game server build, and then placed back into the production fleet. This approach reduces the amount of infrastructure that is required, but it also

increases risk since the number of available live game servers for players is reduced as servers are being replaced.

This model can also be used for performing rolling deployments to backend services such as databases, caches, and application servers that don't host gameplay. As long as these services are deployed in a highly-available manner with multiple clustered instances, then the complexity of deployments to these services should be less than deployments to dedicated game servers.

Blue/green deployment

The primary objective of a blue/green deployment in a game is to minimize downtime while also allowing safe rollback to the previous deployment if issues are identified. It is suitable for deployments where two versions of the game backend are compatible and can serve players simultaneously.

In the blue/green deployment strategy, two identical environments (blue and green) are set up. The existing game version is labeled as blue, while the new game version that is the deployment target is labeled as green. When the green environment is ready for migration, you can configure your routing layer to flip the traffic over to the green environment while keeping the old environment (blue) available in case failback is needed. In this scenario, the routing updates might require updating the matchmaking service to configure it to begin sending game sessions to the new fleet, or in the case of game backend services, this could be updating DNS records in Amazon Route 53 for your service or shifting application load balancer weights to send traffic to your new target group.

One of the drawbacks of the blue/green deployment strategy is the inherent cost of the standby environment due to the additional infrastructure required while performing the deployment. An option to mitigate this additional infrastructure cost is to consider adopting a variant of blue/green deployment where new game software is deployed onto the same servers that are already deployed into production. In this scenario, a new green server process can be started with the new software alongside the existing blue server process, with the cutover happening between server processes rather than between separate physical infrastructure. This approach can also speed up game deployments across a large amount of infrastructure by removing the need to wait for new servers to be launched in the cloud. For best practices on this deployment approach, see Blue/Green Deployments on AWS.

Canary deployment

Canary deployment is useful for game developers, as the strategy can be applied to release an early alpha or beta build of a game, or a game feature like a new game mode, map, or challenge to

a restricted or small set of players in-production. Such a deployment is called a *canary*. The release may have additional tracking and reporting, so when real players play that game or feature, their game play telemetry is collected and analyzed for anomalies and issues.

For new features, the players are not consistently notified about this, and the game telemetry is the primary source used to determine if players are experiencing issues and the release should be rolled-back. At the same time, if no significant issues are identified, the feature can then be further rolled out to more players for additional data. If the players are notified, then they can be asked to provide regular feedback about their experience. Such test activity would ideally be coordinated by a live operations team.

As a strategy, canary deployment can also be used for standard releases to gradually make a new feature available to the players. A potential advantage over the standard blue/green environment is that a full-scale second environment is not required. The capacity of the new scaled-down environment determines how many players are to be onboarded to the new feature. Before adding more players, the capacity must be scaled appropriately. Even if this customized blue/green technique is expected to cost comparatively lesser than standard blue/green, it is still estimated to incur cost that may be higher than the rolling substitution technique of canary deployments.

Run only a single canary on a production environment, and focus it for its data and feedback. If multiple canaries are deployed, it complicates troubleshooting and isolating of issues in production and impairs the quality of the datasets and feedback being collected.

A variation in the canary is when one or more experiments (generally UI tests) are run through targeted deployments, where one set of the game backend servers serve one version of a feature and another same-sized set serve another version of the same feature. No additional or special infrastructure is created for this, and only the chosen pockets of backend servers receive these updates. The outcome of the experiments is to observe how players react to each of the versions of the same feature, determine if there is a consensus of overall like or dislike, and observe if there are issues identified with its usability or functionality. Such strategic experiments are also called A/B tests, and the overall process is called *A/B testing*. On completion of these experiments, necessary test data is collected before reverting to the current version of the game backend system on the servers used for the tests.

Legacy traditional deployments

In the traditional style of deployment, during a scheduled maintenance window the game is shut down and connected players are dropped or drained before server instances within the game backend are updated with the latest code builds. This deployment impacts players each time it is

performed, and the players must be notified ahead of the schedule. As a result, this model causes the most player impact and should be avoided whenever possible.

After the game update is deployed, the game can be smoke tested prior to opening the game to the players, who would be waiting for the game to reopen. This can cause a spike of traffic when players try to login and play within a short period of time. Therefore, if the game is not designed to handle such spikes of traffic, you can choose to gradually allow players back into the game in batches.

Alternatively, you can opt to over-provision the infrastructure to sustain the opening spike of traffic, and after the game traffic settles, resources can be scaled down. If necessary, conduct this type of deployment during off-peak hours when the number of players is at its lowest. Frequently scheduled maintenance, as well as extended maintenance, inherently carries a risk of player attrition and potential loss of revenue. Players also expect changes after a new release and can lose trust in the game once returning after a period of downtime.

Implementation steps

- **Minimize downtime:** Implement deployment strategies that reduce downtime and keep players in the game.
- **Infrastructure as code (IaC):** Use tools like AWS CloudFormation or Terraform to manage game infrastructure and reduce human errors.
- **Deployment strategies:** Use one or a combination of rolling substitution, blue/green, and canary deployments to provide smooth updates and reduce player impact.

GAMEOPS03-BP05 Pre-scale infrastructure required to support peak requirements

Scale infrastructure ahead of large-scale game events to make sure that you can handle the sudden increase in player demand.

Level of risk exposed if this best practice is not established: High

Implementation guidance

In addition to new game launches, live games typically run in-game events, promotions, new content, and season releases as examples of ways to sustain and improve player engagement. Such

activities experience a high volume of player traffic for the duration of the event or promotion. The business expects to hit or surpass their intended targets for the event, and the game infrastructure must sustain and support them through it.

Prepare your infrastructure ahead of time to be able to support the anticipated player load that you will experience during large scale events. To prepare, game operations teams should coordinate with stakeholders in sales and marketing to estimate the projected demand that will be generated in an upcoming event by looking at past player concurrency, engagement metrics, and sales data. If the event is for a new game launch, game operations teams should work with these stakeholders to identify realistic projections for what scale they anticipate. While it may be difficult to predict how successful a game will become, it is important that everyone understands what the expectations are for success so that the infrastructure can be scaled and tested to support those goals.

Many games choose to launch in stages, starting with a soft launch by opening the game to a small number of players and then organically scaling the players at every stage, prior to a full public launch. During the soft launch period, monitor, identify, track, and resolve issues while refining your projections for the public launch.

To properly estimate infrastructure requirements, collect data through load and performance tests run against your game backends running on production or a production-like staging environment prior to the game launch. Multiple rounds of these tests should be run to simulate different conditions of the game and validate that the backend can withstand the load under most conditions.

To achieve this, developers can write gameplay bots that traverse various workflows in the game and emulate different conditions. These tests should inspect the different system layers of the game backend so that each layer and component is tested and the details are recorded. Use the data collected from these tests to provision plan for the game launch.

Single points of failure (SPOF) should be identified and removed where possible by making the application highly available and fault tolerant. Use load tests to identify SPOFs by emulating failures at different upstream and downstream layers and verifying game and other component behavior.

Along with the necessary estimated infrastructure to be provisioned for the game launch, in-game event, or promotion preparations, set up the system to automatically scale on-demand. Define, configure, and monitor scaling event thresholds to allow the game backend to scale to sustain a high volume of player traffic. For variable traffic, pre-provisioning is best because there may not be

enough time to scale-out. Manual scaling might be required during initial game launches that drive higher than anticipated demand faster than automated systems can scale resources.

On AWS, organizations should request higher Service Quotas for the services that they use in the game backend. Service Quotas are set up for accounts to safeguard customers from inadvertently standing up or scaling more infrastructure than intended. When a game running in an account hits the upper limit of the configured service quota in that Region, the service throttles the requests beyond the provisioned quota and burst provisions. Throttles can cause unintended or unexpected errors and impair the player experience. Monitor, track, and regularly review service quota thresholds for the services used by the game in-production to avoid throttling. When the usage crosses a tolerable service quota threshold, an increase in the quota can be requested by raising an Support Case from the Console Support Center, after logging in to the affected account, or using the Support API.

For critical events like game launches, content releases, promotions, and major in-game events, use AWS Countdown. Countdown provides implementation guidance based on playbooks built by Games experts to provide operational readiness, mitigate potential risks, and plan for capacity needs. AWS Countdown also has a premium support option that offers enhanced support and options like engineers to optimize your infrastructure.

If you are launching a game hosted on Amazon GameLift, review the pre-launch checklists to prepare.

Implementation steps

- **Scale infrastructure ahead:** Prepare infrastructure in advance for large-scale game events to handle sudden increases in player demand.
- **Estimate demand:** Coordinate with sales and marketing to estimate projected demand using past player data and realistic projections.
- **Load testing and SPOF removal:** Conduct multiple rounds of load tests to validate backend capacity, identify single points of failure, and properly configure automated scaling.

Health monitoring

GAMEOPS04: How do you monitor the health of the game?

Monitor game health by implementing comprehensive instrumentation to detect and track player-impacting issues, including client-side activity logging, backend service monitoring, and error reporting. Use a combination of AWS tools like Amazon CloudWatch and AWS X-Ray, as well as third-party solutions, to help quickly identify and resolve problems.

Best practices

- GAMESOPS04-BP01 Instrument the game to detect and monitor player-impacting issues

GAMESOPS04-BP01 Instrument the game to detect and monitor player-impacting issues

In addition to responding to social media and player reports of issues, instrument your game with monitoring solutions to detect and investigate player-impacting issues.

Level of risk exposed if this best practice is not established: High

Implementation guidance

No amount of testing can identify every issue in a game. Games are usually launched with known issues that are planned to be gradually fixed with the next release of the game. Known and reproducible issues are straightforward to address and fix. To assist with identifying such issues, game clients should implement player activity tracking, app logging, and reporting in various strategic places to assist the backend team identify client-side issues. The ability to find such issues early assists the game developers troubleshoot and fix the issue before it becomes widespread. The data and logs reported by the tracking code should never include personally identifiable information (PII), and they should only contain game specific metadata that assist with debugging.

Implement an observability solution for detecting and responding to issues such as game crashes or bugs. You can use Amazon CloudWatch Synthetics to create canaries that can monitor the health of your player-facing backend game services. You can instrument your backend services with AWS X-Ray to trace requests across distributed services, and send your custom logs and metrics to Amazon CloudWatch.

Third-party solutions, such as Backtrace.io and Sentry, are popular solutions for error reporting in games. Application performance monitoring (APM) solutions from partners such as New Relic, Splunk, Datadog, and Honeycomb.io are also popular.

The game's live operations team and community managers should also monitor various social networks and channels to check for player feedback, complaints, and bug reports in addition to the

official support channels. Review and attempt to reproduce every game-specific complaint, or send them to the QA team for review. If reproducible, escalate the issue to the game developers for their troubleshooting and a fix before it impacts the larger player base.

Implementation steps

- **Implement monitoring solutions:** Use monitoring tools to detect player-impacting issues and respond quickly.
- **Track player activity and logs:** Instrument game clients to log player activity and report issues, and verify that no personally identifiable information (PII) is included.
- **Use third-party and AWS tools:** Use tools like CloudWatch, X-Ray, and third-party solutions for error reporting and performance monitoring, and monitor social media for player feedback and bug reports.

Load testing

GAMEOPS05: What should you consider when load testing a game?

When load testing a game, consider the appropriate testing stage, load-generating architecture, and testing framework to effectively evaluate system performance and scalability. Choose the right combination of timing (early development, sprints, pre-production, or post-deployment), infrastructure (EC2, EKS, Fargate, or Lambda), and testing tool (JMeter, Locust, Grafana K6, or Gatling) to align with your game's unique characteristics and development goals.

Best practices

- GAMEOPS05-BP01 Choose the right stage, architecture, and load testing framework to meet your goals

GAMEOPS05-BP01 Choose the right stage, architecture, and load testing framework to meet your goals

The approach to load testing a game can vary significantly depending on many factors, including the stage of the development process it is performed in, the architecture of the load-generating system itself, and the choice of load testing framework. The timing of when it is conducted,

whether in the early phases, during iterative sprints, prior to production deployment, or post-deployment, will shape the goals and focus of the testing efforts. Different designs of load-generating infrastructure have their own pros and cons, and the selection of the load testing framework greatly influences the capabilities, ease of use, and integrations available for the testing process. By thoughtfully aligning these elements, development teams can tailor the load testing approach to the unique characteristics of the game, extract the most valuable performance insights, and provide a smooth experience for their players.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Load testing in different development stages

Conducting exploratory load testing early in the development phases can validate the underlying system architecture. This assists developers to make informed decisions about the game's infrastructure, database design, and network topology before extensive implementation work is done. Load tests identify risks and create a performance baseline, potentially minimizing the need for costly rework and technical debt later in the development lifecycle. They can also foster a shared understanding of the game's performance requirements among the team, leading to better collaboration and decision-making. Ultimately, load testing during the initial phases builds a strong foundation for a high-performing, scalable, and resilient game, helping enhance the overall player experience.

At the end of each sprint or iteration, load testing can evaluate the performance impact of the new features, bug fixes, and other changes introduced in the latest cycle. This targeted approach allows development teams to quickly identify regressions or performance degradations introduced by the latest updates, enabling them to address these issues before they are propagated further down the pipeline and maintaining a consistent level of quality and performance.

Before deploying to production, robust load testing assists teams validate the system's ability to handle the anticipated real-world traffic and load conditions. They can uncover scalability bottlenecks or resource constraints within the production infrastructure and provide the opportunity to optimize the game's performance, creating a smooth and responsive user experience from day one. The insights gained from pre-launch load testing can mitigate launch-day risks and inform ongoing capacity planning, which lays the foundation for the game's long-term sustainability and scalability.

Load testing a game that is already live in production allows teams to monitor the game's performance and identify performance regressions or degradations that may occur over time.

This enables them to proactively address issues before they impact the player experience and negatively affect user retention. Additionally, load testing in production validates the effectiveness of performance optimization efforts or infrastructure scaling that has been implemented. This process provides a high-quality, responsive, and scalable gaming experience for players even as the game evolves and matures.

Load-generating architectures

The design of the load-generating architecture for game load testing can take various forms, each with its own set of advantages and considerations.

At the most basic level, self-managed Amazon EC2 instances can be provisioned and configured to act as load generators. With a control node and worker nodes approach, you can set up multiple load-generating instances, each running their own test script and overall managed by a single control instance. The architecture can scale up and generate more load without increasing complexity by spinning up additional worker nodes, but this hands-on approach requires teams to handle the provisioning, configuring, and managing of the underlying infrastructure.

For a more scalable and orchestrated approach, you can use Amazon EKS Kubernetes clusters to manage and distribute the load testing workload across a fleet of container-based load agents. Kubernetes automatic scaling features can be used to handle the scaling of the load-generating pods, while teams themselves configure and manage the underlying EC2 instances in the cluster hosting the pods.

Alternatively, the serverless nature of AWS Fargate can speed up and simplify the load testing setup by abstracting away the infrastructure management while still providing the necessary scalability and flexibility. For hybrid solutions where an on-premises, load-generating Kubernetes cluster already exists but additional capacity might be needed, EKS Anywhere can manage both clusters as one from the AWS Management Console.

You can also use AWS Lambda functions depending on your requirements and goals. Lambda functions are relatively straightforward to set up and scale without the need to provision and manage additional resources. They also allow the creation of more complex and dynamic test scenarios due to deep integration with other AWS services. However, Lambda functions do have limits on concurrent functions and runtime (15 minutes), which may constrain the scale and length of load testing that can be achieved. Cold start latencies can also impact the accuracy of the results, and the resource limitations of Lambda may not be suitable for highly demanding load testing workloads.

Studios wishing to use a pre-built solution can use Distributed Load Testing on AWS. This solution uses the Amazon ECS on AWS Fargate to deploy containers that can run simulations of tens of thousands connected users. You can use this to quickly start your load testing infrastructure in IAC fashion using AWS CloudFormation.

Load testing frameworks

No two load testing frameworks are built the same. Some have intuitive graphical interfaces for test creation, while others are entirely command line-based. One tool might be flexible and performant but require time and effort to configure and manage, and another might be serverless but limited in the tests it can create and run. Some enjoy large communities and plenty of tutorials while being unproven in the field, contrasting sharply with others that might be battle-tested in production but lack community support or documentation. Choose the framework that strikes the right balance for you and your team. Some few popular options are:

- **Apache JMeter:** Popular Java-based, open-source load testing framework due to its robust feature set and ease of use. Its ability to simulate complex user scenarios, wide range of supported protocols, comprehensive reporting, and proven track record makes JMeter a reliable choice for load testing.
- **Locust:** Modern, distributed load testing framework built on an event-driven architecture, making it performant while resource-efficient. Tests are written in Python, allowing flexible testing scenarios that take advantage of thousands of powerful third-party libraries, while remaining friendly and simple to read.
- **Grafana K6:** Powerful load testing framework that combines ease of use with advanced capabilities. Its support for distributed load generation, flexible scripting, and seamless integration with Grafana for data visualization make Grafana K6 an attractive choice.
- **Gatling:** Open-source load testing framework known for its performance and scalability. Its Scala-based, domain-specific language (DSL) allows developers to create concise, maintainable load testing scripts, and its robust reporting and analysis capabilities provide detailed insights of the system under test.

Implementation steps

- **Load testing stages:** Conduct load testing at various development stages (early development, sprints, pre-production, and post-deployment) to validate system performance and identify issues.

- **Load-generating architectures:** Choose appropriate load-generating architectures (EC2, EKS, Fargate, or Lambda) based on scalability needs, management preferences, and specific test requirements.
- **Load testing frameworks:** Select a load testing framework (like JMeter, Locust, Grafana K6, or Gatling) that balances ease of use, performance, flexibility, and community support to suit your team's needs.

Optimize over time

GAMEOPS06: How do you optimize your game over time?

Optimize your game over time by monitoring key metrics and telemetry data to identify player trends, system performance, and areas for improvement. Continuously update your game design, infrastructure, and load testing approach based on these insights, while adapting to new technologies and frameworks to provide optimal performance and player experience as your game evolves.

Best practices

- GAMEOPS06-BP01 Monitor key game metrics to identify player trends and patterns, and use the information to improve the game
- GAMEOPS06-BP02 Update and adapt the load testing approach as the game changes

GAMEOPS06-BP01 Monitor key game metrics to identify player trends and patterns, and use the information to improve the game

In addition to game client system usage, app usage, exceptions, and crash data, capture game telemetry data that is sent to a game backend system. This data should represent player activity so that you can understand how players interact with various features in the game.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Depending on its implementation, game clients can collect telemetry data at predefined game features or locations in a game world. The data is sent to the backend ingestion service for

processing. If the backend service is unreachable, the clients can store the data locally on the local device until the backend service is available again. The game designers use this telemetry data to review how players are playing the game, and if there are anomalies in the game.

For example, player movements and interactions with items in a map can be extracted from telemetry data and plotted as a heat map of activities in-game by players over a set window of time. Such data assists the game designers identify the need to balance various elements in the game, such as the power of a weapon, the power of an in-game character, or the complexity of a map. The raw telemetry data is generally stored and then processed to extract analytics that can be visualized by analysts.

The Game AnalyticsPipeline solution implementation assists game developers launch a scalable serverless data pipeline to ingest, store, and analyze telemetry data generated from games and services. The solution supports streaming ingestion of data, allowing users to gain insights from their games and other applications within minutes.

For custom game telemetry data ingestion, storage, processing and analytics, AWS also offers a number of specialized services for big data processing and Analytics.

Implementation steps

- **Capture game telemetry data:** Collect data on player activity, system usage, exceptions, and crashes to understand player interactions and identify issues.
- **Implement telemetry collection:** Use predefined game features or locations to collect telemetry data and send it to backend services, storing locally if the backend is unreachable.
- **Use AWS analytics solutions:** Use AWS services like the Game Analytics Pipeline for scalable data ingestion, storage, and analysis, as well as specialized big data processing and analytics services.

GAMEOPS06-BP02 Update and adapt the load testing approach as the game changes

Optimizing the load testing approach is a continuous process that should evolve alongside the game development cycle. As the game grows in complexity, user base, and feature set, the load testing strategy must adapt to verify that it accurately simulates real-world conditions and provides actionable insights.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Consider the following:

Missing or outdated testing scenarios

As new functionality is added to a game during the development process, create and run new load testing scenarios to validate the performance and scalability of the new features. Similarly, features and functionality are often refactored to improve performance, address player feedback, or align with new design goals, requiring testing scenarios to be continuously updated to keep pace with the changes and truly test and reflect the state of the system.

New load testing frameworks

Developers might need to change load testing frameworks for a variety of reasons:

- The initial framework may no longer be able to adequately simulate the user load or provide the necessary level of insight into the system's performance
- New game features might require load testing support for new protocols, APIs, or integration points
- Developers might desire more advanced features as they become more comfortable with the load testing process
- Preference for frameworks that better align with the team's technical expertise, programming languages, or existing toolchains

By carefully evaluating and adapting over time, developers can align the load testing process with the game's changing requirements and continue to provide the requisite insights to optimize and improve the overall user experience.

Optimizing cost

The ease and convenience of using managed AWS services can be highly beneficial, especially in the early stages of development. These services abstract the underlying infrastructure management, allowing teams to quickly set up their solution and focus solely on crafting load testing scenarios and analyzing the results. However, using managed services can often come at a higher cost because of the additional value and convenience they provide, like provisioning, configuring, and maintaining infrastructure, as well as providing high availability, scaling, and monitoring capabilities.

As teams mature and grow more comfortable and confident with their load testing process, there may come a time when self-managing the infrastructure can provide additional optimization and cost savings. While this hands-on approach increases the operational overhead, having direct control over the compute resources, configurations, scaling behaviors, and resource utilization can unlock new opportunities for fine tuning and reducing cost. For example, it might make sense for teams to start their load testing journey with an AWS Fargate serverless architecture, then move to self-managing the underlying nodes in an Amazon EKS cluster later.

Implementation steps

- **Update testing scenarios:** Continuously create and update load test scenarios to validate new features and refactored functionalities, and verify that they reflect the current state of the game.
- **Evaluate load testing frameworks:** Adapt to new frameworks as needed to simulate user load, support new protocols, and align with the team's expertise and toolchains.
- **Optimize costs:** Start with managed AWS services for ease and convenience, then consider self-managing infrastructure for cost savings as the team grows more comfortable with the load testing process.

Resources

Refer to the following resources to learn more about our best practices related to operational excellence.

Documentation and blogs

- Architecture best practices for Game Tech
- Managing Your Game Studio on AWS pt.1
- Managing Your Game Studio on AWS pt. 2
- Managing Your Game Studio on AWS pt. 3
- Establishing your best practice AWS environment
- multi-account strategy for your Control Tower landing zone
- Game Analytics Pipeline
- Maximize Your Game Data Insights with Game Analytics Pipeline
- Leveraging AWS Glue and Amazon Redshift Spectrum for Player Insights
- How to set up a CI/CD pipeline on

- How Good Job Games Accelerates 43% with AWS Build Pipeline
- Implementing a Build Pipeline for Unity Mobile Apps
- Other relevant CI/CD blogs
- Game DevOps made easy with Game-Server CD Pipeline blog
- Harmony Games Deploys a Fully Custom Game Backend Utilizing AWS Cloud Development Kit (AWS CDK) (AWS CDK)
- GameLift Prepare for launch
- Hybrid game server hosting with Amazon Gamelift Anywhere
- Speed up game server development with Amazon Gamelift Anywhere and the Amazon Gamelift Agent
- How to host your Unreal Engine game for under $1 per player with Amazon Gamelift
- New Solution Guidance for building scalable cross-platform game backends on AWS
- Load testing the Pragma backend game engine to 1 million concurrent users on AWS
- How Code Wizards load tested Heroic Lab's Nakama to two million concurrent players with AWS
- Best practices with Organizational Units
- AWS X-Ray
- AWS Countdown
- AWS for Games Solutions Hub

Partner solutions

- New Relic
- Splunk APM
- Backtrace.io
- Sentry
- Datadog APM
- Honeycomb.io

Whitepapers

- Organizing your Environment using multiple accounts
- Introduction to Scalable Game Development Patterns on AWS

Videos

- YouTube series: Building Games on AWS
- AWS for Games: Boss LEVEL Podcast
- Re:Invent 2023: Scaling on AWS for the first 10 million users
- Re:Invent 2022: How Riot Games processes 20TB of analytics daily on AWS
- Re:Invent 2022: How AWS and Riot Games built a governance reporting engine
- Re:Invent 2023: Implementing distributed design patterns on AWS
- Re:Invent 2023: Scaling a multiplayer game into the millions with Mortal Kombat 1
- Re:Invent 2022: The evolution of chaos engineering at Netflix
- Re:Invent 2023: Best practices for cloud governance
- Re:Invent 2023: Best practices for creating multi-Region architectures on AWS

Training materials

- Curriculum – Getting Started with AWS for Games Part 1

Security

The security pillar includes the ability to protect information, systems, and assets while delivering business value through risk assessments and mitigation. Due to the global visibility and large number of players, games are a desirable target for exploiters, hackers, and others looking for ways to exploit and abuse systems. This can often result in a disappointing player experience and increased costs for the game developer if there isn't a strong security foundation set in place.

As described in the Shared Responsibility Model, it's important to understand which aspects of security are the responsibility of AWS and which aspects are the responsibility of the customer so that you are prepared to maintain a strong security posture. This pillar provides best practice cloud security guidance for you to consider when developing and operating games in the cloud.

Before you architect a system, you must establish a set of security best practices which includes access controls. Additionally, you should be able to identify security incidents and protect your systems and services while maintaining the confidentiality and integrity of data through data protection. You should have a well-defined and practiced process for responding to security incidents. These tools and techniques are important because they support business objectives such as preventing financial loss or complying with regulatory obligations.

Customer example

AnyCompany Games is a fictional game studio that is in the process of improving their security posture. Security can be straightforward to understand when there's an explanation of its direct application. AnyCompany Games is used in this section to contextualize the security best practices described in the pillar

Focus areas

- Design principles
- Security foundations
- Ongoing security
- Identity and access management
- Access control
- Detection
- Infrastructure protection

Design principles

In addition to the design principles from the security pillar of the Well-Architected Framework whitepaper, the following design principle can strengthen the security of your game workload in the cloud:

- **Monitor and moderate player usage behavior:** Capture and analyze usage data to understand how players interact with your game and social features. By analyzing this data, you can detect and respond to abusive and inappropriate behavior that can degrade the player experience.

Security foundations

GAMESEC01: How do you implement security fundamentals for game development?

Game studios require a unique security approach that protects both development environments and live player services. A robust AWS security strategy for game studios requires three interconnected components: a multi-account structure, strong authentication, and a clear authorization strategy using IAM policies. A multi-account AWS structure enables studios to separate different game projects, development stages, and tool environments. This gives studios more granular control for things like access to specific environments or services. Enabling strong authentication verifies team members can securely access development resources whether working in-studio or remotely, while maintaining strict controls over source code, game builds, and proprietary tools. Studios should also have a clear authorization strategy for granting permissions using the principle of least privilege with IAM permissions and roles. Use IAM roles to assign permissions between different development team roles, such as giving dev teams access to low-level AWS services while restricting artists and designers to specific asset management and build systems. This specialized approach verifies game studios can protect their intellectual property,

maintain efficient development workflows, and scale their teams securely while giving developers the appropriate access to iterate quickly on their projects.

Best practices

- GAMESEC01-BP01 Use roles and federated access, rather than the account root user, to perform actions on your AWS environment
- GAMESEC01-BP02 Use AWS Control Tower to quickly set up a multi-account environment on AWS
- GAMESEC01-BP03 Use least privilege role policies that are tailored to specific job functions
- GAMESEC01-BP04 Use roles and federated access policies together with account level access policies to grant access to your AWS resources
- GAMESEC01-BP05 Use a central identity provider

GAMESEC01-BP01 Use roles and federated access, rather than the account root user, to perform actions on your AWS environment

When you first create an AWS account, you begin with an identity known as the root user, which is accessed using the email address and password associated with the account. The root user has complete access to AWS services and resources within that account. In most cases, you should avoid using the root user for day-to-day tasks. When root-level access is required, confirm that it's absolutely necessary and verify that additional logging and guardrails are in place to track its use.

Level of risk exposed if this best practice is not established: High

Implementation guidance

In an AWS Organizations configuration, each account still has its own root user, but day-to-day access should instead be managed through IAM roles and IAM Identity Center users. Create role-based access tailored to your game's lifecycle stages and teams. For example, the live operations team might need permissions to manage in-game events, while developers need access to push updates. When working with third-party services or partners, use federated access to enable secure collaboration without exposing sensitive infrastructure. This approach verifies that each user or partner has only the access they need while maintaining the security of your game's infrastructure and player data.

Customer example

AnyCompany Games implemented role-based access control when developing their new game. By using specific IAM roles for their diverse development teams, they avoid using shared credentials. This setup allows a dev team to assume a role for core game systems, while the content team's role is only able to access asset management services.

Implementation steps

- Do not use the root user after setting up an account unless absolutely necessary. Create the account, secure the root user, and immediately create the required administration IAM roles and assign that role to federated user.
- Only use the root user when you need to perform a limited number of tasks that are only available to the root user. Examples of these tasks include changing your root user email address and changing your AWS support plan.

GAMESEC01-BP02 Use AWS Control Tower to quickly set up a multi-account environment on AWS

If you start using AWS with just a single account, you might find your game studio growing out of it as your game development process advances. For example, with a single AWS account, you might begin to reach service limits, or your costs for different projects and workloads may become more complex. Creating different accounts for different game titles and environments allows teams to experiment with new features, bypass service limits, and maintain security posture and compliance. By implementing a multi-account strategy in AWS, you can benefit from distributing service limits across multiple accounts and gain insights into your AWS costs.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

It is a common misconception that using multiple AWS accounts will automatically be more confusing and time consuming. Rather, using AWS services that are designed to facilitate the governance of multiple accounts can assist your game studio to spend less time managing your accounts.

You can use AWS Control Tower is a service to securely provision a multi-account AWS environment. Control Tower is recommended if you are building a new AWS environment, starting your journey on AWS, or are completely new to AWS. During the short setup process , you can

integrate with other AWS services that are involved with managing accounts and user access, such as AWS Organizations, Service Catalog, and AWS IAM Identity Center.

Customer example

AnyCompany Games initially operated from a single AWS account, and they hit multiple roadblocks when one of their games' development team reached EC2 service limits during a crucial beta test. At the same time, their development team for a different game struggled with resource allocation for their automated testing pipeline. The situation reached a breaking point when AnyCompany Games couldn't accurately separate costs between projects, making it difficult to budget for each game's development.

AnyCompany Games then implemented a multi-account strategy using AWS Control Tower. They created separate accounts for each game project, with distinct development, QA, and production environments. This account level separation isolates each projects data and assets, so teams working on one game can't access or modify resources from another. Through AWS Organizations, they established a centralized billing structure that clearly showed each game's infrastructure costs and also created organization-wide access polices.

Implementation steps

- Use AWS Control tower to set up an automated multi-account environment.
- Organize accounts based on environments (like development, QA, and production).
- Use AWS IAM Identity Center and Service Catalog to centralize user permissions and streamline resource provisioning across accounts.

GAMESEC01-BP03 Use least privilege role policies that are tailored to specific job functions

Configuring IAM policies is an essential part of establishing a strong security foundation. When you set permissions with IAM policies, grant only the permissions required to perform a task. You do this by defining the actions that can be taken on specific resources under specific conditions, also known as least-privilege permissions. For example, QA teams need access to change things in the testing environments but should not have the ability to modify the production environment.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

You might start with broad permissions, like managed policies, while you explore the permissions that are required for your workload or use case. As your use case matures, you can work to reduce the permissions that you grant to work toward least privilege.

Implementation steps

- Follow the practice of least privilege permissions for create IAM roles for users and applications.
- Use AWS-managed policies to quickly provide broad access while identifying the specific permissions teams or applications need to perform their tasks.
- Studios can also use IAM access analyzer policy generation to generate custom IAM policies based on CloudTrail events that identify actions and services used by an IAM entry.
- Regularly review IAM policies and edit overly permissive policies.

GAMESEC01-BP04 Use roles and federated access policies together with account level access policies to grant access to your AWS resources

New AWS users often use IAM policies only when granting access to others. However, if you are using AWS Organizations, consider how to use service control policies together with IAM policies to grant your studio team members and contractors the necessary levels of access.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

You can create IAM policies to allow or deny access to AWS services or API actions that work with AWS Identity and Access Management. They can only be applied to IAM identities, such as users, groups, or roles. For example, an IAM policy might be used to provide a user read-only access to Amazon S3.

Service control policies (SCPs) are guardrails for your AWS accounts. An SCP doesn't grant permissions, they are used to restrict actions on AWS services for individual member accounts. For example, an SCP can deny an AWS account from accessing a particular Region.

When an action is taken, the relevant IAM policy is evaluated in combination with the SCPs. Following up on the previous example, is a role is attempts to run an EC2 instance, IAM indicates is they are permitted ("Allow" for ec2:RunInstances) and the SCPs will determine if their choice of Region is valid ("us-east-1" is permitted, but "us-west-1" is denied by the SCP).

Layering IAM policies and SCPs can verify that anyone who accesses your AWS resources will only be given the appropriate permissions that they need. This is especially important to consider if your AWS accounts and resources span multiple Regions, but not everyone within your game studio needs to access all of them.

You can tailor IAM policies to grant specific teams specific permissions for updating things like game configurations, managing player data, configuring promotional events, and moderating user-generated content. Meanwhile, you can use SCPs to enforce organization-wide controls crucial for game operations. These might include restricting deployment to only approved Regions where the game operates, helping prevent unauthorized access to sensitive player data stores, enforcing compliance requirements, and controlling costs by limiting service usage across development accounts.

Implementation steps

- Use IAM policies to manage permissions for individual users, groups, or roles.
- Use service control policies (SCPs) in AWS Organizations to enforce account-level permissions.
- Combine IAM policies and SCPs to grant only the required access for specific users and accounts.

Resources

- Policies and permissions in AWS IAM
- Service control policies
- AWS managed policies for job functions

GAMESEC01-BP05 Use a central identity provider

A central identity provider acts as a single source for storing and managing user credentials, identities, permissions, and authentication.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Use a central identity provider to streamline your user authentication process, enforce consistent security polices, and simplify your user management across your AWS accounts and applications. Having a centralized approach removes the need to manage user identities and

credentials separately, which reduces the risk of inconsistencies, redundancies, and other security vulnerabilities. Consolidating user identities and authentication into one place also allows for better visibility, control, and auditability for your entire AWS environment.

Customer example

AnyCompany Games faced significant challenges with managing developer access across their rapidly expanding AWS infrastructure. Their development team grew from 50 to 200 people across three major titles. Initially, each project team managed their own AWS access credentials, resulting in inconsistent security practices, delayed onboarding for new developers, and occasional security incidents.

The studio implemented AWS IAM Identity Center as their central identity provider, consolidating user management into a single system. They connected it with their existing corporate directory, enabling developers to use their same company credentials for AWS access. Now developers use their single, existing company login to gain the AWS access they require to complete their work

Implementation steps

- Consider using AWS IAM Identity Center as your central identity provider. This provides consistent access management across your AWS accounts, provides your employees with single sign-on authentication, and simplifies user access auditing to your AWS applications. IAM Identity Center also connects with existing corporate identities from supported identity providers.

Ongoing security

GAMESEC02: How do you achieve, maintain, and monitor ongoing security best practices?

Adhering to best security practices is paramount for businesses across industries, but especially in the gaming industry. The games industry relies on cultivating and sustaining player trust and creating a strong reputation, and even minor security issues can quickly undermine that confidence.

Moreover, the global nature of the gaming industry necessitates compliance with various industry regulations and standards governing data protection, consumer privacy, and security across

the Regions where games are offered. Fair and secure gameplay is another critical aspect that underscores the importance of robust security measures. Cheating, hacking, and other forms of game exploitation can disrupt the gaming experience for legitimate players, which makes strong security controls essential to maintain the integrity of gameplay and foster a level playing field for participants.

Best practices

- GAMESEC02-BP01 Use ready to deploy templates for standard security practices
- GAMESEC02-BP02 Use automated remediation techniques when a security event does arise

GAMESEC02-BP01 Use ready to deploy templates for standard security practices

Ready-to-deploy templates provide a proactive and agile way to assess your security posture in the cloud. Pre-configured templates evaluate your cloud security and implement necessary changes promptly. The templates encompass a wide range of best practices across various technologies and widely accepted security frameworks. Using templates can assist game studios to maintain consistent infrastructure configurations, especially as they may scale and add additional AWS accounts to support new workloads.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

By using AWS services and implementing ready-to-deploy templates, game developers can proactively assess and strengthen their cloud security posture, safeguarding their intellectual property, protecting player data, and fostering a secure gaming landscape through regular security assessments and continuous monitoring to promptly identify and address potential vulnerabilities.

Customer example

AnyCompany Games faced a significant challenge when preparing to launch their next title in the European industry. They realized that their existing data handling practices didn't meet GDPR requirements. They turned to AWS Security Hub CSPM and AWS Config and its ready-to-deploy templates for a solution. The team implemented the GDPR-specific conformance pack in AWS Config, which automatically assessed their existing infrastructure against GDPR standards. This initial scan revealed several critical gaps, such as improper data retention policies and inadequate access controls on where player data was stored. Using the template's predefined

rules, AnyCompany Games rapidly implemented the necessary changes. Moreover, the ongoing automated compliance checks provided by the template allowed the small team to maintain GDPR compliance effortlessly, even as they continued to update and expand the game.

Implementation steps

- Use templates for standard security practices, such as managed rules and conformance packs in AWS Config and standards in AWS Security Hub CSPM.
- Review the details of the Security Hub CSPM standards to determine which ones align most with the security needs of your game studio.

GAMESEC02-BP02 Use automated remediation techniques when a security event does arise

Using automated remediation techniques, game developers can proactively protect and maintain their gaming infrastructure and minimize the potential impact a security incident might have. If a security issue is detected, use a runbook to guide your response to the situation. Automate these responses where possible to remediate issues more quickly and reduce their impact. This improves the player experience by reducing the chance of downtime and disruptions to the game.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Preparing to respond to security issues not only safeguards the players' experience but also to meet the various compliance and regulatory standards. Additionally, using automated security responses scales your security operations as your workloads expand. AWS provides services to help identify and automate a response to these incidents.

Customer example

AnyCompany Games faced a critical security incident when an S3 bucket containing unreleased character models and textures for their upcoming game was accidentally made public during a routine asset pipeline update. The automated security system detected the bucket permission change within minutes of the modification. The system immediately executed its remediation runbook: reverting the bucket to private status, logging access attempts during the exposure window, notifying the security team, and creating a detailed CloudTrail log of the permission changes.

Implementation steps

- Use the Automated Security Response on AWS solution to implement automation runbooks that define the actions that will automatically be taken in response to security events in AWS Security Hub CSPM.

Resources

- AWS for Games Blog — Managing Your Game Studio on AWS: Part 1
- AWS for Games Blog — Managing Your Game Studio on AWS part 2
- Register an existing organizational unit with AWS Control Tower
- AWS account root user
- Tasks that require root user credentials
- Automated Security Response on AWS

Identity and access management

GAMESEC03: How do you manage player identity and access management?

When developing a game, you must determine how you will provide players with access to your game and related services. The following section explores design considerations, including player authentication, authorization, and multi-factor authentication.

Best practices

- GAMESEC03-BP01 Determine your approach to identify and control player access to your game's environment and resources
- GAMESEC03-BP02 Authenticate requests that are sent to your game backend service
- GAMESEC03-BP03 Use your game backend service to validate player requests to join a multiplayer game
- GAMESEC03-BP04 Enforce a strict security policy for player user accounts by requiring a strong password
- GAMESEC03-BP05 Provide an option for players to set up multi-factor authentication (MFA) on their accounts

GAMESEC03-BP01 Determine your approach to identify and control player access to your game's environment and resources

This decision is influenced by your player acquisition and monetization strategy, player experience, and other factors such as the existing capabilities that might be provided by your game publishing partners. For example, a game might require purchases and require a player to create a user profile to associate real-money payment methods with their account.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Alternatively, a game may desire to reduce the barrier to entry for first-time player experiences by removing the need to create a user account before playing the game, thereby improving the chance that a player will try the game for the first time. Typically, games will implement one or more combinations of player identity and access management approaches for their game.

Unauthenticated or anonymous access

This access level is useful in situations where a game does not require a player to create a new user account or link with their identity on social networks and gaming systems. This is the simplest and quickest way for a player to start playing a game and is particularly useful in mobile games where a game developer may want to reduce the barrier to entry for the initial experience.

In this access scenario, if you want to identify usage from the game installation, you can program the game client to generate and store a unique identifier onto the player's device. This unique identifier is used to identify the player across game sessions on their device and allow analytics reporting on usage over time. Later, if a player chooses to create an account, you can associate their new user account with their previously-generated unique identifier. This will link their new player identity to their historical usage, which might include stats and game achievements.

If a player does not eventually create and link an account, the device that the player uses to interact with the game can be uniquely identified, but recoverable information about the player is not collected and stored. Thus, if the player breaks or loses their device, the previous stored data associated with the device is also lost and might not be recoverable.

Authentication with username and password

A game may allow players to create their own user accounts with a username and password that are stored within the game's backend. This might occur when a game developer is collaborating

with a game publisher who already has an existing player account system that the developer can integrate with. Alternatively, a developer who publishes their own games might want to simplify the player experience by allowing players to create a single user account for access across the games that they publish.

Authentication and account linking with third-party social networks and gaming systems

It is common for online games and games with social features to provide third-party identity provider federation to simplify the player experience. Instead of asking players to create a username and password combination to authenticate, you can use identity federation to allow players to authenticate using their third-party accounts with social networks and gaming systems. This login process simplifies the sign-in and registration experience for players. It also provides a convenient alternative to mandatory account creation and a frictionless method for players to access games.

For game developers, a federated login process can offer a streamlined player verification workflow. It may also provide a more reliable way to manage player data that is used for personalization. This is because you do not need to ask players to provide you with certain data that they likely have already provided to the third-party identity provider. Additionally, these systems provide integration with additional social features such as the ability to link players with their friends.

Implementation steps

- Use unauthenticated or anonymous access to reduce barriers for first-time players by generating a unique device identifier to track usage and enabling account linking later.
- Implement username and password authentication for dedicated user accounts, using existing player account systems or creating a unified experience across games.
- Integrate third-party identity providers for federated authentication, simplifying login processes and enabling access to social features and personalization data.

GAMESEC03-BP02 Authenticate requests that are sent to your game backend service

Authenticating requests that are sent to your game backend service can block unwanted requests from succeeding.

Level of risk exposed if this best practice is not established: High

Implementation guidance

You should provide an authentication service for players to log in, which should return secure short-lived tokens, such as a JSON Web Token (JWT), to the game client when a player successfully authenticates.

These tokens can include claim assertions that contain player attributes and other relevant metadata. This relevant metadata can be used in subsequent requests that are sent from the game client to your game backend to authenticate requests and authorize them in the context of the authenticated player.

You have the option to either design and build your own player authentication system, which would require ongoing improvement and maintenance, or you can use the scalable and secure user sign-up, sign-in, and access control features provided by Amazon Cognito.

Amazon Cognito user pools include a user directory for authentication and authorization. A user pool provides APIs that you can integrate into your game for sign-up, sign-in, and password reset workflows, which can be integrated with third-party identity providers. Application Load Balancers and Amazon API Gateway both provide integrations with Cognito to integrate user authentication for requests sent to your custom game backends hosted with these services.

If your game supports anonymous access and you cannot authenticate a player, you can use a client authentication approach to provide a more secure experience when integrating with your game backend. If your game client uses AWS services directly, requests to these services must be signed using credentials. To provide credentials to your game client for unauthenticated users, you can use the AWS SDK to retrieve short-lived credentials from Amazon Cognito identity pools that can be used to sign your requests to AWS services. These credentials can be refreshed from your game client.

In addition to directly integrating with the AWS SDK from the game client, you can also build your own game backend, using a service such as Amazon API Gateway, which supports custom authorization. By designing your own game backend service, you can gain authoritative control over requests with custom server-side logic.

For more information on building a backend service for games hosted using Amazon GameLift, see Design your game client service.

Customer example

AnyCompany Games enhanced the security of their next title by adopting a managed authentication and authorization approach. Instead of maintaining a custom username and

password system, they used Amazon Cognito user pools to handle player sign-up and sign-in, and identity pools to support anonymous access for players trying the training mode before creating an account. They also implemented custom authorization logic within the game to recognize administrator roles defined in Cognito, granting those users access to special in-game management features.

Implementation steps

- Use Amazon Cognito user pools to manage authentication with secure tokens like JWTs, enabling features like sign-up, sign-in, and password resets.
- Retrieve short-lived credentials from Amazon Cognito identity pools for anonymous users to securely interact with AWS services.
- Implement custom game backends using Amazon API Gateway for custom server-side authentication logic.

GAMESEC03-BP03 Use your game backend service to validate player requests to join a multiplayer game

Typically, in multiplayer games, a player will join a game session by selecting an option directly from a list of available sessions, or they will submit a request to find a match. The latter approach places the responsibility on the game developer to locate an eligible game session and provide the connection information (usually an IP address and port number) back to the player's game client. The implementation may vary depending on the genre of game you are developing, but regardless, it is a security best practice to perform server-side validation of a player's request to join a game.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

For example, in a session-based multiplayer game, a request from a player to join a game session should be validated by your game server software with your game backend matchmaking service before authorizing their connection to the server. When a player requests to join a game session, the game server should check the request for a unique identifier, such as a player session ID and server-generated ticket that was previously provided to the game client by your game backend matchmaking service.

Upon initiating the connection to the game server, your server-side software can use this information to verify with the matchmaking service that the player's connection request is valid

and verify that the player is not joining a spot that was previously reserved in the game session for another player.

For games that are hosted on Amazon GameLift, see Game client/server interactions with Amazon GameLift Servers for an example of how this type of server-side validation can be implemented.

Customer example

During AnyCompany Games' initial beta launch, they discovered that players were bypassing their matchmaking system by directly connecting to game servers, leading to serious competitive integrity issues. When highly-ranked players found that they could share server IP addresses with friends, they began circumventing the skill-based matchmaking system, resulting in experienced players joining novice matches and creating a frustrating experience for new players. AnyCompany Games responded by implementing a server-side validation system that generated unique session tickets for each matchmaking request. The system required both the player IDs and matchmaking request tickets and verified connection attempts against their backend matchmaking service.

Implementation steps

- Validate player join requests server-side using unique identifiers like player session IDs and server-generated tickets.
- Confirm the validity of connection requests with the matchmaking service to block unauthorized access.
- Verify that reserved spots in game sessions are not accessed by unauthorized players during the validation process.

GAMESEC03-BP04 Enforce a strict security policy for player user accounts by requiring a strong password

If a game provides players with the ability to create a user account with a password, you should require players' passwords to adhere to strong policies. For example, Amazon Cognito user pools provide you with the ability to define password requirements for user accounts. Establishing a strong password policy can protect your players' accounts from being overtaken through social engineering and brute force attacks.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Customer example

AnyCompany Games faced a crisis when their popular title experienced a wave of account hijackings due to weak password policies. Players who were using simple passwords like "password123" were becoming victims of automated brute force attacks, resulting in lost items and compromised in-game currency. To combat this, AnyCompany Games revamped their login system and mandated that passwords not be previously used, include at least one uppercase letter, one number, one special character, and a minimum length of 15 characters.

Implementation steps

- Require strong password policies for player accounts to enhance security.
- Use Amazon Cognito user pools to define and enforce password requirements.

GAMESEC03-BP05 Provide an option for players to set up multi-factor authentication (MFA) on their accounts

Player accounts can be an asset to bad actors, particularly in games that support in-game currency and purchases. Due to the pervasiveness of player account hacking and social engineering attacks, provide players with the option to enhance the security of their accounts by configuring multi-factor authentication (MFA).

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

When a player attempts to access their account by using MFA, a temporary code is sent to their email address, phone number, or a purpose-built multi-factor authentication mobile app. To successfully authenticate, the player must then enter the code into the login system within a limited time frame.

MFA can also be used to help protect accounts that are attempting to authenticate from a new geo-location, accounts that have been flagged by player support for potential malicious activity, and even for accounts that have not logged into the game for an extended period.

For example, Amazon Cognito user pools can configure multi-factor authentication on user directories.

Implementation steps

- Enable multi-factor authentication (MFA) to enhance player account security.
- Use temporary codes sent via email, phone, or MFA apps to verify account access.
- Apply MFA for new geo-locations, flagged accounts, or accounts with extended inactivity.

Access control

GAMESEC04: How do you block unauthorized access to game content?

Modern games include a significant amount of content, such as downloadable content (DLC), which is an important aspect of player engagement and game monetization. Players expect an ongoing stream of new characters, levels, and challenges, requiring game developers to keep up with a constant demand for fresh content to retain players. The variety and the size of the content can vary greatly depending on the type of the game and the device on which the game is played. Regardless of a game's system, protect the game's content from unauthorized access.

Best practices

- GAMESEC04-BP01 Restrict access of downloadable content to authorized clients and users
- GAMESEC04-BP02 Limit origin access to authorized content delivery networks (CDNs)
- GAMESEC04-BP03 Implement geographic restrictions to limit unauthorized access
- GAMESEC04-BP04 Restrict access to content with digital rights management (DRM) solutions

GAMESEC04-BP01 Restrict access of downloadable content to authorized clients and users

Restrict access to game content by authorized applications and clients. Consider using Amazon S3 as a cost-effective and scalable origin for storing downloadable game content and Amazon CloudFront to provide globally performant content delivery to players. Both services provide built-in mechanisms for restricting access to data that is stored, such as restricting access to authenticated users.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Granting access to content that is stored in Amazon S3

When you need to grant access to content that is stored in S3, there are several factors to consider. By default, only the AWS account that created an S3 bucket can access the objects stored within it. To grant access to your internal applications and to manage content stored in Amazon S3 buckets, use AWS Identity and Access Management (IAM) to create policies that provide appropriate access.

IAM roles can be associated with federated users, systems, or applications hosted in services, such as Amazon EC2, AWS Lambda, and container-based applications hosted in Amazon EKS and Amazon ECS. For example, you might use the AWS SDK or AWS CLI to publish and manage game content assets in S3 buckets. To support this use case, you can create an IAM role with appropriate access to read and write game content to your S3 buckets and associate it with the EC2 instances that host your software and scripts.

Resource-based policies can be defined for your bucket and for specific objects. S3 bucket policies are associated with an S3 bucket and can be used to restrict access to the bucket and objects within it, as well as grant access to your Amazon S3 resources from other accounts. For example, in scenarios where multiple teams or separate game development studios are working on the same game content and require the same access to centrally hosted content in Amazon S3, you can use an S3 bucket policy to define permissions for cross-account access to the S3 resources. Consider using S3 access points, which can simplify managing data access to shared data by creating access points with names and permissions specific to each application or sets of applications. The Amazon S3 documentation contains additional best practices for access control in Amazon S3.

Granting short-term access to your content

When access is only need for a specific limited time, generate temporary URLs that grant short term access to your content. Amazon S3 provides support for generating presigned URLs, which allow object owners to grant time-limited access to objects in Amazon S3 without updating your bucket policy. By doing so, the end user or application that is being granted access is not required to have an account or IAM permissions and instead uses the presigned URL to access the content.

This is a best practice that is commonly used in a variety of games use cases, such as granting authorized players access to downloadable content that they have been entitled to and providing temporary access to limited time game content. Presigned URLs can also be used to provide temporary permissions for uploading content to an S3 bucket. For example, you might consider

using a presigned URL to provide a player with access to upload client logs for assisting your support team with troubleshooting a player support case.

Using a content delivery network to provide access to your content

While your applications, game developers, artists, and other personnel may need direct access to the content in S3 buckets for development and management purposes, use a content delivery network to provide access to content that is publicly available to players or other users over the internet. This approach improves download performance and reduces costs by caching frequently accessed content. Amazon CloudFront can globally distribute your content by caching and delivering it closer to your players while reducing the load on your game's download origin, such as Amazon S3.

Rather than serving your public content directly from S3 buckets, it is recommended to keep this content private and serve it publicly by using CloudFront. CloudFront can be configured to require players to access your private content (such as a new game download for paid players only) by using either signed URLs or signed cookies. You can then develop your application either to create and distribute signed URLs to authenticated users, or to send set-cookie headers that set signed cookies for authenticated users. When you create signed URLs or signed cookies to control access to your files, you can specify an ending date and time, after which the URL and cookies are no longer valid.

Optionally, you can also specify the IP address or range of addresses of the computers that can be used to access your content, which is useful if you want to restrict access to specific game development studio partners or contractor networks. Use signed cookies when you want to provide access to multiple restricted files, or if you don't want to change your current URLs. Use signed URLs when you want to restrict access to individual files or if your users are using a client that doesn't support cookies. Signed URLs take precedence over signed cookies.

Implementation steps

- Use IAM roles and bucket policies to grant appropriate access to S3 buckets for internal applications, teams, or cross-account scenarios.
- Generate presigned URLs for granting short-term access to S3 objects, suitable for downloadable content or temporary uploads like client logs.
- Use Amazon CloudFront with signed URLs or cookies to more securely serve private content to authenticated users

GAMESEC04-BP02 Limit origin access to authorized content delivery networks (CDNs)

Block users from circumventing your content delivery networks to directly access content from your origin, such as your Amazon S3 buckets. It is important to restrict access to your origin to only your authorized CDNs, which reduces data transfer costs from unnecessarily serving content out of the origin. It also improves your security posture by flowing public access to your origin content through the same entry point, where you can deploy edge security controls such as AWS WAF layer 7 filtering, injection and inspection of security-related HTTP request parameters, and distributed denial of service (DDoS) protections.

Level of risk exposed if this best practice is not established: High

Implementation guidance

To implement these controls for an Amazon S3 origin, you can use an Amazon CloudFront origin access identity (OAI), which verifies that requests to your S3 objects are originating from your CloudFront distribution. Associate AWS WAF with your CloudFront distribution to provide layer-7 filtering. However, if you are serving content from additional CDNs, you can configure the CDN to insert one or more custom HTTP headers into origin requests which can be inspected by AWS WAF to verify that the incoming traffic originated from your authorized CDN provider.

This approach is also useful for helping prevent users from circumventing your CDN providers when your origin is hosted behind an Application Load Balancer (ALB). ALBs can be associated with AWS WAF for layer-7 protections. You can configure AWS WAF to insert a custom HTTP header that will be inspected by your ALB to process and inspect incoming traffic to the load balancer by AWS WAF.

Customer example

AnyCompany Games implements origin access restrictions to help protect their game assets, downloadable content, and patch files from unauthorized direct access that could enable players to bypass security checks or obtain premium content without proper authentication. This approach allows them to monitor content access patterns through a centralized point, making it straightforward for them to identify suspicious download behaviors that might indicate the presence of coordinated attacks or unauthorized content redistribution.

Implementation steps

- Use Amazon CloudFront origin access identity (OAI) to restrict direct access to S3 objects

- Associate AWS WAF with CloudFront or ALB to provide layer-7 filtering and help protect against DDoS attacks and malicious requests.
- Configure custom HTTP headers in Cloudfront to verify that incoming traffic originates from authorized sources.

GAMESEC04-BP03 Implement geographic restrictions to limit unauthorized access

When a player requests your content, Amazon CloudFront serves the requested content from the nearest edge location, regardless of where the player is located. However, there may be scenarios in which you need to restrict how your content is accessible by users in specific parts of the world. For example, you may have a rolling game deployment strategy that releases content in phases on a country-by-country basis, or you may have to abide by country-specific access controls.

Level of risk exposed if this best practice is not established: High

Implementation guidance

You can use geographic restrictions, also known as geo blocking, to block players in specific geographic locations from accessing content that you're distributing through a CloudFront distribution. This feature lets you restrict access to files that are associated with a distribution and restrict access at the country level. Alternatively, you can use a third-party geo-location service to restrict access to a subset of the files that are associated with a distribution or to restrict access at a finer granularity than the country level.

By using CloudFront geographic restrictions, you can allow your players to only access your content if they're in one of the countries that are on an allow list of approved countries. You can also block your players from accessing your content if they're in one of the countries that are on a deny list of banned countries. If a request is received from a blocked geographic location, CloudFront will return a 403 Forbidden HTTP status code to the player. It is important to note that this works well for non-sensitive content and should not be used as stand-alone protection for PII or sensitive game artifacts.

Implementation steps

- Use CloudFront geographic restrictions to allow or deny content access based on country-level allow or deny lists.

- Return a 403 Forbidden HTTP status code for requests originating from blocked geographic locations.
- Avoid relying solely on geo restrictions for protecting sensitive content or PII

GAMESEC04-BP04 Restrict access to content with digital rights management (DRM) solutions

Consider restricting access to your game content by using strong encryption tools such as a digital rights management (DRM) solution. This type of solution can be used to encrypt your private content and distribute the decryption keys to authorized players.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

DRM solutions are recommended in situations where you want to allow players to download game content early, but you do not want them to be able to access or play the content until a predetermined time. For example, this is common in situations where players are allowed to pre-order a game and configure their game client to automatically begin downloading the encrypted files early. This strategy verifies that the game is downloaded and ready to be played once the game has been officially released. After the game is released, the player's game client can request decryption keys from the DRM backend solution so that it can decrypt the previously downloaded files and begin playing the game.

DRM systems are also used to block unauthorized re-distribution and manipulation of games after they have been downloaded and installed by an authorized player. DRM systems require integration with the origin for exchanging encryption keys and authorizing players to retrieve the decryption key. Commercial DRM providers offer a range of solutions with features and support for different devices.

Implementation steps

- Use DRM solutions to encrypt private game content and distribute decryption keys to authorized players.
- Enable pre-download of encrypted files for pre-ordered games, unlocking access with decryption keys at release time.
- Integrate DRM systems with the origin to manage encryption keys and block unauthorized redistribution or manipulation of content.

Detection

GAMESEC05: How do you monitor and analyze player usage behavior within your game?

Monitoring and analyzing player usage behavior is essential for game studios because it enables you to detect security threats, cheating, and other forms of abusive behavior that could compromise game integrity and player safety. By tracking patterns such as unusual progression rates, abnormal in-game transactions, or suspicious communication behaviors, you can identify potential cheaters, fraudulent accounts, or coordinated threats before they significantly impact the player experience.

Best practices

- GAMESEC05-BP01 Implement a comprehensive data collection strategy to monitor player behavior
- GAMESEC05-BP02 Collect, store, and analyze player usage logs to detect inappropriate behavior

GAMESEC05-BP01 Implement a comprehensive data collection strategy to monitor player behavior

To maintain a positive player experience, implement a comprehensive data collection and analysis strategy. Capturing, storing, and analyzing relevant data provides insights into how players interact with your game's features and with each other. This data-driven approach can guide decision-making, enhance player engagement and retention, optimize monetization strategies, and ultimately improve the overall player experience.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Implement data collection systems to capture and log relevant player actions, such as gameplay sessions, progress, achievements, purchases, interactions with game elements, and social activities. Collect server-side data like server load, network traffic, and error logs to monitor the technical performance and identify potential issues. Gather player feedback through surveys, forums, support tickets, and social media channels to understand their experiences and preferences.

When storing your game data, establish a centralized data warehouse or data lake to store and organize the collected data and implement pipelines for data cleaning, transformation, and aggregation to prepare the data for efficient analysis.

After storing the data, analyze it to gain insights such as player retention and churn, monetization strategies, and feature usage through data visualization tools.

Implementation steps

- Capture and log player actions, server-side metrics, and feedback to monitor interactions and technical performance.
- Use a centralized data warehouse like Amazon Redshift or S3 data lake to store, clean, transform, and organize game data for analysis.
- Analyze collected data with visualization tools, like Amazon Quicksight, to gain insights into player retention, monetization, and feature usage.

GAMESEC05-BP02 Collect, store, and analyze player usage logs to detect inappropriate behavior

Instrument your game to collect logs to understand how players use the features of your game and how they interact with other players. You can then block unauthorized activities that can degrade the player experience.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Send structured log events to the Game Analytics Pipeline, by using a logging solution such as Amazon CloudWatch Logs or Amazon OpenSearch Service, or through a solution from an AWS Partner such as Datadog, Sumo Logic, New Relic, Honeycomb.io, or Splunk. Structure these player usage logs so that they can be used to detect when specific actions by players need to be investigated.

After you have captured the data, consider implementing tools to detect inappropriate usage behavior. For example, if your game has social features such as in-game player messaging, voice chat, or online forums, save logs from these player engagements in a format that can be analyzed for moderation purposes.

Configure your game's voice chat feature to export recordings to Amazon S3 and use Amazon Transcribe to convert the audio speech to text format which can be stored for processing. Alternatively, you can perform real-time streaming transcription by integrating your game backend voice chat service directly with the Transcribe API to transcribe streaming audio in real time. Moderation teams can manually review the content, and once the content is in a standard format, you can also use AWS AI/ML services to perform moderation automatically. Amazon Comprehend can be used to perform natural language processing (NLP) to uncover information from the unstructured text, which can classify and organize the conversations into relevant topics and identify inappropriate behavior such as profanity.

Implementation steps

- Collect, store, and analyze player usage logs.
- Use AWS services for artificial intelligence and machine learning to more efficiently review and gain insights into your player usage logs.

Infrastructure protection

Refer to the Well-Architected Framework whitepaper for best practices in infrastructure protection for security that apply to games workloads.

GAMESEC06: How do you monitor and respond to infrastructure threats?

Monitoring and responding to infrastructure threats is critical for game studios because their infrastructure represents the backbone that supports millions of concurrent players, processes real-money transactions, and stores valuable player data and proprietary game content. It is essential for game studios to implement monitoring systems and incident response processes that can preserve the integrity of both player experiences and business operations.

Best practices

- GAMESEC06-BP01 Use tools for detecting and responding to threats to your infrastructure
- GAMESEC06-BP02 Use artificial intelligence and machine learning tools to automate aspects of your infrastructure protection strategy
- GAMESEC06-BP03 Use insights from system-level logs to continuously improve your infrastructure protection strategy

GAMESEC06-BP01 Use tools for detecting and responding to threats to your infrastructure

To continuously monitor for malicious activities and unauthorized behaviors within your AWS environment, consider using Amazon GuardDuty. GuardDuty identifies threats by monitoring account behavior, network activity, and data access patterns within your environment. It analyzes events across multiple data sources, such as CloudTrail event logs, Amazon VPC Flow Logs, and DNS logs for potential threats. By integrating with Amazon CloudWatch Events and Lambda, GuardDuty alerts can be automatically forwarded to relevant security teams for further analysis.

Level of risk exposed if this best practice is not established: High

Implementation guidance

AWS Security Hub CSPM provides a comprehensive view of your security state in AWS and check your environment against security industry standards and best practices. Security Hub CSPM collects security data from across AWS accounts, services, and supported third-party partner products and analyzes your security trends and identify the highest priority security issues. The Amazon GuardDuty integration with Security Hub CSPM enables you to send findings from GuardDuty to Security Hub CSPM. Security Hub CSPM can then include those findings in its analysis of your security posture.

It's common for bad actors to employ bots to take over accounts and cheat in games. WAF Bot Control gives you visibility and control over common and pervasive bot traffic that can consume excess resources, skew metrics, cause downtime, or perform other undesired activities.

Ransomware is malicious code designed to gain unauthorized access to systems and datasets and encrypt that data to block access by legitimate players. After ransomware has locked players out of their systems and encrypted their sensitive data, cyber criminals demand a ransom before providing a decryption key to unlock the data. Organizations can be completely shut down by a malicious event, incurring significant costs and loss of business productivity. Refer to Securing your AWS Cloud environment from ransomware for best practices that you can apply to strengthen your ability to fight ransomware before, during, and after an incident takes place.

Your game may provide players with the ability to contact player support agents through a call center such as Amazon Connect or chat bots using Amazon Lex. Amazon Connect provides support for monitoring live and recorded conversations. To analyze interactions between players and player support chat bots built with Amazon Lex, you can store the conversation logs from these

interactions in Amazon CloudWatch Logs which can be exported to Amazon S3 and analyzed as described previously.

Finally, conduct penetration testing exercises as part of your infrastructure protection strategy. Whether you are performing these assessments in-house or through an AWS Partner, adhere to the AWS customer support policies for penetration testing.

Implementation steps

- Use Amazon GuardDuty to monitor account behavior, network activity, and data access patterns for threats, and integrate with Security Hub CSPM for a unified security view.
- Implement AWS WAF Bot Control to help detect and mitigate bot traffic that can harm resources and player experiences.
- Conduct penetration testing exercises regularly, adhering to AWS customer support policies, to assess and strengthen your security posture.

GAMESEC06-BP02 Use artificial intelligence and machine learning tools to automate aspects of your infrastructure protection strategy

Amazon Lookout for Metrics uses machine learning to automatically detect and diagnose anomalies in your business and operational data and monitors the metrics that are most important to your businesses with greater speed and accuracy. The service also makes it straightforward to diagnose the root cause of anomalies, such as a sudden dip in revenue, logins, transactions, or retention. It does not require game developers to have ML experience to set up and can connect to popular data sources including Amazon S3, Amazon CloudWatch, Amazon RDS, Amazon Redshift, as well as many SaaS applications. For example, you can integrate Amazon Lookout for Metrics with the Game Analytics Pipeline and other data sources to begin analyzing behavior to detect anomalies.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Alternatively, you may choose to build, train, and host a custom machine learning model using Amazon SageMaker AI AI to address use cases such as content moderation, toxicity detection, cheat detection, fraud detection, and more.

Customer example

AnyCompany Games uses Amazon Lookout for Metrics to automatically detect unusual patterns in server performance, player login attempts, or transaction volumes that could indicate threats from bad actors. Additionally, they have used Amazon SageMaker AI to develop custom machine learning models that continually analyze network traffic patterns and player behavior to help identify coordinated threats, such as bot networks that are attempting to exploit their virtual economy.

This automated approach allows their security team to focus on investigating and responding to genuine threats rather than manually monitoring thousands of metrics, while making sure that emerging threat patterns are detected and addressed before they can significantly impact game availability or player safety.

Implementation steps

- Use Amazon Lookout for Metrics to help automatically detect and diagnose anomalies in key business and operational data
- Integrate Amazon Lookout for Metrics with data sources like the Game Analytics Pipeline, Amazon S3, or CloudWatch to monitor metrics such as revenue, logins, and retention.
- Use Amazon SageMaker AI to build, train, and host custom machine learning models for advanced use cases like cheat detection, fraud prevention, and content moderation.

GAMESEC06-BP03 Use insights from system-level logs to continuously improve your infrastructure protection strategy

Capture and store system-level logs from relevant services, such as S3 server access logs, CloudFront access logs, and ALB access logs. These logs can be stored in an S3 bucket in your account and are useful for associating your player usage information from within the game with system-level information including connection details such as IP addresses, request headers, and relevant request manipulation and filtering that you may have configured within your game backend. You can send these logs to the same logging solutions mentioned earlier, and you can analyze them using SQL queries with Amazon Athena without requiring the logs to be moved out of Amazon S3.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Access Analyzer for S3 is a feature that monitors your bucket access policies, making sure that the policies provide only the intended access to your Amazon S3 resources. Access Analyzer for S3 evaluates your bucket access policies and allows you to discover and swiftly remediate buckets with potentially unintended access.

Implementation steps

- Use AWS services for threat detection and incident response to automate aspects of your infrastructure protection strategy.
- Gain insights into your infrastructure protection through system-level logs and AWS services for artificial intelligence and machine learning.

Data protection

When developing and architecting your game, consider what type of data your studio is gathering and how you have decided to approach protecting it. Topics to explore within this aspect of security include:

- How you have chosen to identify and classify your data
- How you are protecting data at rest
- How you are protecting data in transit

There are no data protection best practices specific to the Games Lens. Refer to the Well-Architected Framework whitepaper for best practices in data protection for security.

Incident response

GAMESEC07: How are you defining and enforcing policies to respond to player misconduct and abusive behavior?

Player misconduct and abusive behaviors can significantly impact the experience of your players. Bad player behavior can drive away legitimate players, leading to decreased player retention,

reduced revenue from in-game purchases, and negative reviews that can damage a game's reputation and future sales.

Define policies that promote positive actions among your players and determine how you will enforce these policies.

Best practices

- GAMESEC07-BP01 Implement an incident response plan to handle bad actors and abusive behavior
- GAMESEC07-BP02 Ban accounts that are associated with bad actors

GAMESEC07-BP01 Implement an incident response plan to handle bad actors and abusive behavior

Create a plan of action for responding to bad actors and abusive behavior in your game.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Consider factors such as when to temporarily suspend or permanently ban players and how long to disable credentials for temporarily suspended players.

Customer example

AnyCompany Games creates a tiered incident response system in which minor infractions like inappropriate chat messages result in automatic 24-hour account suspensions, while more severe violations such as cheating or harassment trigger immediate 7-day suspensions with mandatory review by human moderators.

Additionally, AnyCompany Games establishes escalation procedures in which repeat offenders face progressively longer suspensions. They create appeal processes that allow falsely flagged players to contest automated actions while maintaining security through identity verification requirements.

GAMESEC07-BP02 Ban accounts that are associated with bad actors

If left unmitigated, abusive behaviors in a game can continue to have a negative impact on the gaming experience for others and should be mitigated as soon as possible. Implement a process to impose bans or other forms of restrictions on bad actors who are confirmed to be in violation of your terms of service.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Typically, the rules and evaluation process for determining the circumstances for imposing these types of restrictions will be determined by personnel such as a player community team or trust and safety team within your organization. After you've flagged bad actors, run a pre-determined workflow to act on the identified players.

For example, AWS Step Functions and AWS Lambda functions can be used to run an automated workflow that accepts a batch of player accounts as input. The workflow then updates entries in an Amazon DynamoDB table called Bans, which can include details about the player account, the ban reason, and duration.

Depending on the design of your game and account management system and the type of abuse that you encounter from bad actors, maintain a banning system of record that is separate from your account management system. You may not want to turn off the player's account from your account management system, opting instead to simply turn off their ability to play your game. This can be useful in situations where the player's account credentials are used to access multiple games with different terms of service or policies.

Implementation steps

- Define and enforce policies for responding to abusive behaviors from bad actors.
- Use AWS services to automate your responses to bad actors.

Resources

- AWS Security Incident Response Technical Guide
- AWS Machine Learning Blog: Detect real and live users and deter bad actors using Amazon Rekognition Face Liveness
- AWS Solutions for Games: Community Health

Application security

GAMESEC08: How do you secure your CI/CD pipeline?

A game development CI/CD pipeline is typically comprised of highly available source control servers and storage, compute resources to run your builds, and software to perform automated testing, along with the proper network connectivity from your development machines. Securing your CI/CD pipeline is important for protecting sensitive information, preserving code integrity, and maintaining trusted releases. Embedding governance and guardrails allows for developer agility while maintaining good security practices.

Since games often handle payment processing, store personal information, and maintain virtual economies that are worth real money, a security breach in the development process could result in significant financial losses, regulatory penalties, and a loss of player trust.

By integrating safeguards, organizations maintain visibility and control over the software delivery process, enabling rapid incident response and promoting a culture of secure coding practices.

Best practices

- GAMESEC08-BP01 Apply security at every stage of the CI/CD pipeline

GAMESEC08-BP01 Apply security at every stage of the CI/CD pipeline

Guardrails such as access controls, separation of duties, and audit trails provide protection against unauthorized access or malicious activities.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Your people, processes, and technology should secure the pipeline as well. The people closest to the code must establish secure coding practices and make sure they follow them. Iterate on your processes continuously to verify that there is consistency in the level of security throughout the pipeline. Lastly, implement technology to verify that best practices and processes are not bypassed.

Customer example

AnyCompany Games implements role-based access controls in which only senior developers can approve changes to their anti-cheat system code, while requiring mandatory code reviews from security team members for components that handle player payment data.

Their CI/CD pipeline automatically runs threat model validation checks, making sure that new features like a player trading marketplace are tested against previously identified attack vectors such as item duplication exploits or fraudulent transaction attempts.

Implementation steps

- Provide users permissions bases on the principle of least privilege.
- Use AWS CloudTrail to audit API calls made across the services used in the pipeline.
- Employ pre-commit hooks to verify that code is following general practices and company policies.

Automate security

GAMESEC09: How do you automate security within your CI/CD pipeline?

Integrate security measures within your CI/CD pipeline to maintain a robust security posture throughout the development life cycle. This process offers many of the same benefits as having security of your pipeline. Having a secure CI/CD pipeline reduces the likelihood of security events that could potentially delay the game development timeline.

Providing security in your CI/CD pipeline involves implementing security best practices and tools at every stage of the development cycle. Having security in the pipeline also allows you to also reduce the time of security reviews.

Automating security within your CI/CD pipeline is particularly crucial to verify that security controls are consistently implemented and tested with every code change. Implementing the proper tools and automation can deliver safe and secure games.

Best practices

- GAMESEC09-BP01 Integrate tooling and automation to reduce the mean time of security reviews

GAMESEC09-BP01 Integrate tooling and automation to reduce the mean time of security reviews

To identify security vulnerabilities, organizations can use a variety of different tools and services like Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST). SAST is a way to review the source code and determine security vulnerability. DAST is a black box way of testing your code which tests your applications without looking at the source code.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

Another tool that organizations can use is Software Composition Analysis (SCA), which assesses the security of your third-party or open source dependencies. For a more manual approach, secure code reviews can be implemented throughout the pipeline.

Customer example

AnyCompany Games uses SAST tools to automatically flag potential security flaws during the development process. They also use DAST tools to simulate threats against running game builds to validate that security controls are working as intended. Additionally, AnyCompany Games integrates dependency scanning tools into their development process to automatically identify known vulnerabilities in third-party libraries and game engines.

Implementation steps

- Use Amazon CodeGuru as a SAST tool.
- Use open-source tools like OWASP Dependency Check, SonarQube, or OWASPZap.

Resources

- Security for Developers

Threat modeling

GAMESEC10: How do you integrate threat modeling into your organization's application development lifecycle?

Threat modeling is the process of identifying and prioritizing potential threats to your application and determining solutions that can be used to mitigate them. This practice has become increasingly important as games have evolved into complex, connected systems that handle sensitive user data and real-money transactions.

Integrate threat modeling as an ongoing exercise for supporting the security of the game, not only in the initial design phase but as the game continues to grow and evolve.

Best practices

- GAMESEC10-BP01 Determine when and how to complete threat modeling exercises throughout your application development lifecycle

GAMESEC10-BP01 Determine when and how to complete threat modeling exercises throughout your application development lifecycle

There is no one single best way to approach threat modeling. Details for when and how to do this will vary based on the unique needs of your game studio. For example, depending on the size of your studio, you may have team members who are involved in one or multiple aspects of the threat modeling process.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

The AWS Security Blog provides an overview for considerations to keep in mind when devising your strategy for threat modeling, such as:

- Which of your team members and personas should be involved in threat modeling
- How to determine the appropriate workflow tools to use
- How to determine ownership of various aspects of threat modeling
- How to identify and evaluate security controls to be used within your workload design

Customer example

AnyCompany Games begins by cataloging valuable assets such as player data, game code and algorithms, in-game currencies, user-generated content, and intellectual property like unreleased content or proprietary engines. They consider different types of potential bad actors such cheaters seeking unfair advantages, bad actors attempting to steal personal or financial data, and malicious users trying to disrupt gameplay.

Throughout the development process, AnyCompany Games uses threat models to guide secure coding practices and influence testing strategies to focus on high-risk areas. Before a game launch, they conduct comprehensive threat modeling reviews to assess readiness for anticipated player loads and unauthorized access attempts, and to prepare incident response procedures.

Implementation steps

- Implement guardrails at every stage of your CI/CD pipeline.
- Use automation and tools to improve the efficiency of your application security reviews.
- Use threat modeling as a process for improving the security of your applications.

Resources

- AWS Security Blog: How to approach threat modeling
- NIST: Guide to Data-Centric System Threat Modeling
- Threat modeling the right way for builders – AWS Skill Builder virtual self-paced training
- Threat modeling for builders – AWS Workshop

Resources

Refer to the following resources to learn more about our best practices related to security.

Related documents:

- Common Amazon Cognito scenarios
- Using signed URLs
- Use channel flows to remove profanity and sensitive content from messages in Amazon Chime SDK messaging
- Security in Amazon GameLift
- Secure content delivery using Amazon CloudFront
- Security Response Guide
- AWS Best Practices for DDoS Resiliency
- Securing your AWS Cloud environment from ransomware

Related partner solutions:

- Datadog
- Sumo Logic
- Splunk

- Honeycomb.io
- New Relic
- AWS Marketplace - DRM Solutions

Related training materials:

- Getting Started with Amazon Cognito
- Security self-paced training

Reliability

The reliability pillar includes the ability of a system to recover from infrastructure or service disruptions, dynamically acquire computing resources to meet demand, and mitigate disruptions such as misconfigurations or transient network issues.

Focus areas

- Design principles
- Foundations
- Workload architecture
- Change management
- Failure management
- Resources

Design principles

In addition to the design principles in the AWS Well-Architected Framework whitepaper, the following are design principles that can increase reliability in the cloud for games workloads:

- **Establish the baseline for the peak player concurrency and system scalability targets required to meet business projections:** Prior to launching a game and during live game operations, develop estimates for the number of concurrent players expected at peak to establish target goals for system scalability to meet these projections. This assists creating a baseline for your game's reliability. Define scaling policies to accommodate changes in demand automatically without impact availability by verifying that your scaling systems gracefully manage active player sessions.
- **Measure your reliability and the impact on player experience:** Define key performance indicators (KPIs) that represent the health of your game. Monitor the impact of changes in infrastructure and game features on your reliability.

Foundations

To achieve reliability, a system must have a well-planned foundation and monitoring in place with mechanisms for handling changes in demand or requirements. The system should be designed to detect failure and automatically heal itself.

There are no foundations best practices specific to the Games Lens. Refer to the Well-Architected Framework whitepaper for best practices in foundations for reliability that apply to games workloads.

Workload architecture

GAMEREL01: Is your game architecture taking advantage of the cloud's resiliency?

AWS infrastructure is built around Regions and Availability Zones. AWS Regions provide multiple physically separated and isolated Availability Zones, which are connected using low-latency, high-throughput, and highly redundant networking. These constructs can be used to architect workloads with reliability goals in focus.

Best practices

- GAMEREL01-BP01 Distribute game infrastructure across multiple Availability Zones and Regions to improve resiliency

GAMEREL01-BP01 Distribute game infrastructure across multiple Availability Zones and Regions to improve resiliency

To minimize the impact of localized infrastructure impairments on your players, you should distribute your infrastructure deployment uniformly across enough independent locations to be able to withstand unexpected impairments while still having enough capacity to meet the needs of your player demand.

Level of risk exposed if this best practice is not established: High

Implementation guidance

When deploying your game infrastructure, it is recommended to uniformly distribute your capacity across multiple Availability Zones in a Region so that you can withstand disruptions to one or more Availability Zones without disrupting the player experience. Game backend services such as web applications should be load balanced across multiple Availability Zones or should be built using managed service such as AWS Lambda and Amazon API Gateway which provide Regional high availability by design. Similarly, components that maintain state such as caches, databases, message queues, and storage solutions should be designed to provide durable persistence of data across multiple Availability Zones, which is provided by design in services such as Amazon S3, DynamoDB, and Amazon SQS, and can be configured in other services.

When designing your game server hosting architecture for resiliency, deploy your fleets of game servers uniformly across the Availability Zones within an AWS Region to maximize your access to available compute capacity in the Region as well as reduce the impact scope of Availability Zone impairments. For example, you can configure Amazon EC2 Auto Scaling to use the Availability Zones. If an EC2 instance becomes unhealthy, EC2 Auto Scaling can replace the instance, as well as launch instances into other Availability Zones if one or more of the Availability Zones becomes unavailable.

For critical infrastructure, such as authentication, provision a minimum number of viable instances running across multiple Availability Zones, and use auto scaling to handle load increases or fault tolerance if there is an impairment with one of the Availability Zones.

Deploy your game infrastructure into multiple Regions to maximize availability. Cross-Regional disaster recovery features like Aurora global databases and redundant infrastructure that can become active with a simple DNS change deployed into a secondary Region can provide service continuity should the primary Region become impaired. While we encourage this for your game backend services to achieve high availability, this recommendation is especially important for your game servers.

For example, in a multiplayer game, your infrastructure capacity for game servers is likely to outpace the capacity needs for your other services, since game servers are used to host game sessions for players. Many games choose to shard players into logical game Regions (like US west and east). To simplify the player experience and make it straightforward to use global infrastructure to host games, consider de-coupling the name of your player-facing game Regions from the underlying cloud provider Region or data center location that is physically hosting the game servers, along with other infrastructure such as Local Zones or your own data centers that are hosting game server instances supporting that player game Region.

When designing your matchmaking service, deploy a multi-Region architecture with separate software deployments across Regions. Decouple your matchmaking service deployment from the fleets that host your game server instances so that you can route players to a game server in Regions regardless of which regional deployment of your matchmaking service handled the matchmaking request.

Design logic in your matchmaking implementation to favor the game server Regions that meet your latency and other rules, with the ability to fallback to routing players to other Regions if your fleets are low on capacity or there are other regional infrastructure disruptions.

Implementation steps

- Distribute game infrastructure uniformly across multiple Availability Zones to provide high availability and resilience.
- Deploy game backend services and stateful components using managed like AWS Lambda, Amazon S3, DynamoDB, and SQS, or configure load balancing and durability for custom solutions.
- Implement multi-Region deployments for critical game services and servers, using disaster recovery solutions like Aurora global databases and logical player-facing Regions decoupled from underlying physical locations.

Resources

- Static stability
- Best practices for Amazon GameLift game session queues
- Amazon GameLift Multi-Region fleets
- Aurora Global Database for Disaster Recovery

Change management

GAMEREL02: How do you scale your stateful games to accommodate changes in demand?

As your player demand fluctuates over time, your game infrastructure should be able to adaptively scale to handle these changing requirements. While it is difficult to predict the popularity of a

game ahead of time, design an architecture approach that allows for addition and removal of infrastructure capacity to accommodate fluctuations in player population.

Best practices

- GAMEREL02-BP01 Implement a scaling strategy that incorporates the state of active player game sessions
- GAMEREL02-BP02 Support the use of multiple EC2 instance types for your game

GAMEREL02-BP01 Implement a scaling strategy that incorporates the state of active player game sessions

Implement a solution for automatically scaling your game infrastructure in a manner that incorporates the stateful nature of your actively connected player sessions and gracefully handles scaling activities without disrupting gameplay.

Level of risk exposed if this best practice is not established: High

Implementation guidance

One of advantages of developing a game in the cloud is the elasticity that can be achieved by automatically scaling server infrastructure as needed to meet demand. While stateless or asynchronous games and backend services can be dynamically scaled using Amazon EC2 Auto Scaling policies, EKS autoscaling, or similar techniques typically adopted for scalable web applications, game developers typically require a more customized approach for scaling stateful or synchronous games to help block disruptions to active player sessions.

Implementation steps

- For stateful games, generate custom metrics that can be used to monitor the state of your player sessions and available game server capacity, which can be reported to Amazon CloudWatch as custom metrics. Exercise features with application monitoring, such as CloudWatch Synthetics, checking the game for impairment of functions that simple up and down health monitoring may not detect.
- Using custom metrics, implement game server scaling software, for example, as a serverless application using AWS Lambda function or AWS Fargate, to manage the fleet of dedicated game server instances by using the AWS SDK to make API calls to update the minimum, maximum, and desired capacity settings for the EC2 Auto Scaling groups hosting your game server build.

- Use Amazon GameLift to host your game servers and use the out-of-the-box game server auto scaling capabilities to manage this scaling process for you.

The automatic scaling capabilities of Amazon GameLift are aware of active player sessions and can be configured to block the termination or scale-in of game server instances that are actively hosting players. For more information, see Monitor Amazon GameLift Servers with Amazon CloudWatch.

GAMEREL02-BP02 Support the use of multiple EC2 instance types for your game

When hosting your game using EC2 instances, or if you use containers hosted on EC2 instances in your AWS account, use multiple instance types in your hosting strategy. By using multiple instance types, you can increase the number of compute options that can be used when your game is scaling to add more servers to support player growth, which improves reliability in case your preferred instance type is unavailable.

Level of risk exposed if this best practice is not established: High

Implementation guidance

When hosting your game using Spot Instances, use multiple instance types since the availability of Spot Instances fluctuates based on customer demand.

Test your game on multiple instance types to meet your cost and performance requirements and determine a prioritized ranking of instance types. Amazon EC2 Auto Scaling supports using multiple instance types and sizes as well as assigning weights to each instance type in your configuration so that you can implement prioritized ranking of compute options.

When hosting your game using Amazon GameLift managed hosting, decide what type of instances your game needs and how to run game server processes on them (using a runtime configuration). When choosing resources for a fleet, consider several factors, including game operating system, instance type (the computing hardware), and whether to use On-Demand Instances, Spot Instances, or both. Hosting costs with Amazon GameLift primarily depend on the type of instances you use. For more information, see Choose compute resources for a managed fleet.

Implementation steps

- Use multiple EC2 instance types to improve reliability and scaling options when hosting your game on EC2 or containers.
- Configure Amazon EC2 Auto Scaling or GameLift fleets with prioritized instance types and weights to optimize cost and performance.
- Test your game on various instance types to verify that performance meets requirements and adjust your hosting strategy accordingly.

Failure management

GAMEREL03: How do you persist game state during infrastructure disruptions?

As your game infrastructure experiences various operational events over time, your game's architecture should be designed to maintain continuity for player experiences and preserve game state during infrastructure events. To handle these events, implement monitoring, graceful shutdowns, and state persistence mechanisms to verify smooth gameplay experiences for your players.

Best practices

- GAMEREL03-BP01 Monitor game server disruptions, and use the data to improve hosting architecture to achieve reliability goals
- GAMEREL03-BP02 Implement loose coupling of game features to handle failures with minimal impact to player experience
- GAMEREL03-BP03 Monitor infrastructure events over time to measure impact on player behavior

GAMEREL03-BP01 Monitor game server disruptions, and use the data to improve hosting architecture to achieve reliability goals

Monitor game server metrics and the impact of failures or degradations in performance, such as increased latency under load, on player behavior over time so that you can adjust your game server hosting strategy to meet your game's reliability requirements. Game server infrastructure to be

degraded should be removed from service immediately if it is impacting players or proactively replaced when there are no active player sessions hosted on the server.

Level of risk exposed if this best practice is not established: High

Implementation guidance

For scenarios where games are hosted as REST APIs, system reliability can be managed like traditional web application architectures, where traffic can be load balanced across multiple servers in a distributed manner to mitigate the risk of server failures.

For real-time synchronous gameplay, a game session is usually hosted on a game server process running on a virtual machine, or game server instance, since gameplay state needs to be maintained in a performant manner and replicated to connected game clients. This implementation means that a player's experience is tightly coupled to the performance and reliability of the game server process that hosts their game session. This type of architecture makes managing the reliability of game servers more complex than traditional approaches.

To mitigate the impact of a game server failure, configure your game to continuously perform asynchronous updates of a player's game state to a highly-available cache or database such as Amazon ElastiCache (Redis OSS) or Amazon MemoryDB. If a server failure occurs, the player's last saved game state can be fetched from the external data store, and their session can be restored on a new game server instance.

However, this approach adds additional cost and complexity to manage this external state, and may not be suitable for fast-paced or competitive games where the state changes are so frequent and happening at such a significant scale that introducing even a performant in-memory cache data store would result in replication lag that is too significant to be useful to restore a session from. For games of this nature, the optimal approach is to accept the loss of the server and send the player back to a game lobby to find another session or you can automatically redirect them into another game session.

Capture as much useful log data about what caused the server disruption so that you can investigate the issue later. Amazon GameLift provides guidance for debugging fleet issues and provides the ability to remotely access Amazon GameLift fleet instances .

Implementation steps

- Monitor game server metrics for performance degradations, and remove or replace degraded servers as necessary to maintain reliability.

- Use Amazon ElastiCache or MemoryDB for asynchronous game state updates to enable session recovery after server failures when feasible.
- Capture detailed log data on server disruptions for investigation and debugging, leveraging tools like Amazon GameLift for fleet monitoring and remote access.

GAMEREL03-BP02 Implement loose coupling of game features to handle failures with minimal impact to player experience

Decoupling components refers to the concept of designing server components so that they can operate as independently as possible. Some aspects of gaming are difficult to decouple since data needs to be as up to date as possible to provide a good in-game experience for players. However, many components and gaming tasks can be decoupled. For example, leaderboards and stats services are not critical to the gameplay experience, and the reads and writes to these services can be performed asynchronously from the game.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Implement graceful degradation for features in your game that can be disabled automatically or by an administrator if issues are detected, as well as configure upstream services that depend on the feature to be able to gracefully handle the failure. For example, if specific player data is not properly loading within your game client, you should consider whether this data is critical to the gameplay experience. If not, configure the game client to gracefully handle this failure without disrupting the experience for the player, optioning to retry fetching this data later when the player revisits the screen.

Employ logic such as timeouts, retries, and backoff to handle errors and failures. Timeouts keep systems from hanging for unreasonably long periods. Retries can provide high availability of transient and random errors.

Define non-critical components which can be loosely coupled to critical components. Loose coupling allows systems to be more resilient since failure in one component does not cascade to others. When game features do not require stateful connections to your game servers or backend, you should implement stateless protocols to scale dynamically and recover from transient failures. Develop your non-critical components where it can be loosely coupled with stateless protocols using an HTTP/JSON API. Implement network calls from the game client to be asynchronous and

non-blocking to minimize the impact to players from slow-performing game features or other dependent services.

To further improve resiliency through loose coupling, use a messaging service such as a queuing, streaming, or a topic-based system between components that can be handled asynchronously. This model is suitable for an interaction that does not require an immediate response or where an acknowledgment that a request has been registered is sufficient. This solution involves one component that generates events and another that consumes them. The two components will not integrate through direct point-to-point interaction but through an intermediate such as a durable storage or queuing layer. This also assists to improve system's reliability by preserving messages when processing fails.

Research and select an appropriate messaging mechanism, as various messaging services have different characteristics, such as ordering and delivery mechanisms. Design operations to be idempotent so the chosen message system delivers messages at least once. As an example, consider a typical game use case where your game needs to track player playtime, stats, or other relevant data which can lead to a high write-throughput use case at times of peak player concurrency.

To implement a reliable architecture, consider whether the use case requires read-after-write consistency as perceived by the player. Typically, scenarios such as these are suitable for asynchronous processing and can be achieved by implementing a write-queueing pattern where the requests are ingested into a scalable and durable message queue such as Amazon SQS and can be inserted into your backend database in batches using a consumer service, such as a Lambda function. This approach is more reliable than synchronous communication between multiple distributed components including the player's game client, your backend web and application servers, and your internal database system. It also reduces costs because the backend database does not need to be scaled to meet peak write throughput since the consumer processing from the write queue can be used to slow down this ingestion rate as needed.

Implementation steps

- Decouple non-critical components like leaderboards and stats services from critical gameplay features to allow asynchronous operations and enhance resiliency.
- Implement graceful degradation for non-critical features with logic for timeouts, retries, and backoff, and verify that the game client handles failures without disrupting the player experience.

- Use messaging systems like Amazon SQS for asynchronous communication between components, enabling scalable, durable, and reliable processing of high throughput use cases.

Resources

- Build highly scalable and reliable workloads using microservice architecture
- Integrating microservices by using AWS serverless services
- Understanding asynchronous messaging for microservices
- Introduction to Scalable Game Development Patterns on AWS
- Implementing Graceful Degradation

GAMEREL03-BP03 Monitor infrastructure events over time to measure impact on player behavior

Monitor your game server process, game server instance metrics, and game experience metrics to determine the root cause of issues. In addition to monitoring CPU and memory, you can also set up monitoring for network metrics related to network limitations of EC2 instances to alert you of issues such as exceeding bandwidth, packets-per-second, or other network-level issues that may indicate your server resources are under provisioned.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Use CloudWatch Synthetics to check critical path application functionality for player experience, such as being unable to login or other service impacting issues. For game servers hosted using Amazon GameLift, consider monitoring metrics like:

- GameServerInterruptions and InstanceInterruptions, which can assist in understanding how limitations in Spot instance availability are impacting your game servers deployed using Spot.
- ServerProcessAbnormalTerminations, which can be used to detect abnormal terminations in your game server processes.

It is recommended to maintain historical metrics data of your game server reliability. Use this historical data for reporting purposes and join it with other datasets to uncover potential trends and assess the impact on player behavior over time that may be due to game server issues.

Amazon CloudWatch does not retain metrics indefinitely, and the storage resolution of metrics is increased over time, so consider exporting these metrics to cost-effective long-term storage such as Amazon S3. You can configure CloudWatch Metric Streams to automatically deliver your metrics from CloudWatch Regions to your own S3 bucket, where they can be stored long-term in a storage tier such as S3 Intelligent-Tiering and eventually archived using Amazon Glacier. By placing your metrics in Amazon S3, they are readily available to be joined with other datasets in your data lake for interactive querying with Amazon Athena.

Implementation steps

- Monitor game server, instance, and network metrics, including bandwidth and packet-per-second limits, using Amazon CloudWatch and CloudWatch Synthetics for critical path functionality checks.
- Track GameLift-specific metrics like *GameServerInterruptions* and *ServerProcessAbnormalTerminations* to assess the impact of Spot instance availability and detect abnormal server terminations.
- Export CloudWatch metrics to Amazon S3 for long-term storage, use cost-effective tiers like S3 Intelligent-Tiering or Glacier, and analyze trends with tools like Amazon Athena.

Resources

- Amazon EC2 instance-level network performance metrics uncover new insights
- CloudWatch Metric Streams – Send AWS Metrics to Partners and to Your Apps in Real Time

Resources

Refer to the following resources to learn more about our best practices related to reliability.

Related documents:

- Practicing Continuous Integration and Continuous Delivery on AWS
- Autoscaling Asynchronous Job Queues
- Design Your WorkloadService Architecture
- Timeouts, retries, and backoff with jitter
- Well-Architected Framework - Reliability Pillar
- Architecting for Reliable Scalability

- The Amazon Builder's Library
- Massive Scale Real-Time Messaging for Multiplayer Games
- Introduction to Scalable Game Development Patterns on AWS
- Running Containerized Microservices on AWS
- Web Application Hostingin the Cloud
- Building a Scalable and Secure Multi-VPC Network Infrastructure

Related videos:

- re:Invent 2020: Ubisoft: Building a multi-platform multiplayer game on AWS
- re:Invent 2018: Supercell- Scaling Mobile Games
- re:Invent 2019: How CAPCOM builds fun games with containers, data, and ML
- re:Invent 2018: Globalizing Player Accounts at Riot Games While Maintaining Availability
- re:Invent 2020: GameLoft - A zero downtime data lake migration deep dive

Related training:

- Using Amazon GameLiftFleetIQ for Game Servers
- Game Server Hosting with Amazon EC2

Performance efficiency

The performance efficiency pillar focuses on the efficient use of computing resources to meet requirements and maintaining that efficiency as demand changes and technologies evolve.

Take a data-driven approach to selecting a high-performance architecture. Gather comprehensive data on the architecture, from the high-level design to the selection and configuration of resource types. Consider your architectural choices on a cyclical basis to take advantage of the continually evolving set of services and solutions. Metrics assist with understanding deviation from expected performance so you can take action. A data driven approach assists with making tradeoffs with your architecture to improve performance, reduce cost, or improve the developer experience.

Focus areas

Design principles

In addition to the design principles in the AWS Well-Architected Framework whitepaper, the following design principles can achieve performance efficiency for your games:

- **Measure game performance from end-to-end:** It is important to measure performance as it is perceived from the perspective of your players. This means you should measure the performance of the game client, your game infrastructure, and the internet connectivity that connects your players to the infrastructure. This will assist in understanding where you can make performance improvements within your architecture.

- **Optimize your architecture to improve metrics that reflect actual player experience:** As you adapt and evolve your architecture over time, think about how these improvements and changes will impact player experience. Games workloads should be able to withstand and minimize the impact of failures to block widespread disruptions to gameplay. Game features and systems that aren't critically dependent on each other should be de-coupled to reduce the blast radius of failures and isolate player impacting issues.
- **Use technologies that simplify game operations and increase development speed:** Prioritize the adoption of technologies that can improve developer efficiency. Operational overhead during pre-production stages of development can be a distraction from improving gameplay. By leveraging managed services from AWS or AWS Partners, engineering can be reduced, which enables game developers to focus on the core game loop and player experience. Architecture and performance requirements may change and evolve throughout the game development life cycle and technology trade-offs should be considered at each phase.
- **Design infrastructure to meet peak player concurrency and dynamically scale as needed:** Infrastructure should be designed to scale to meet player demand. Metrics, such as player session concurrency and number of logins, can be used to preemptively scale before systems become overloaded. Reactive system utilization metrics, such as CPU and memory consumption, can be used to scale after systems become overloaded. By dynamically scaling your infrastructure, you can reduce the costs of operating your game.

Architecture selection

GAMEPERF01: How do you select the appropriate hosting option for your game servers?

Selection of the appropriate hosting option for your game servers is foundational to game server performance. Deciding to use EC2 instances, a container solution, or a fully managed service is one of the first decisions to make when architecting for production. Each hosting option will have varying capabilities and considerations for performance tuning, scaling, operations and integration.

Best practices

- GAMEPERF01-BP01 Evaluate game server resource requirements and scalability needs
- GAMEPERF01-BP02 Consider operational overhead for scaling game servers

- GAMEPERF01-BP03 Evaluate integration with other AWS services, development environments, target CPU architectures, and features

GAMEPERF01-BP01 Evaluate game server resource requirements and scalability needs

Evaluate server requirements against your scalability needs to verify that you are selecting a hosting option that both meets your requirements and provides optimal performance.

Level of risk exposed if this best practice is not established: High

Implementation guidance

When selecting the appropriate hosting option for your game servers, consider the following factors:

Game server resource requirements

Assess the CPU, memory, network, and storage requirements of your game server processes to determine what your game consumes. Do not overlook networking; each frame requires CPU cycles to receive player actions, update the state of the game, and send it back to the player. Offloading packet processing can free up CPU for core game functions. Networking is the foundation for smooth and responsive game play so testing it early in the process defines a baseline performance profile for a game.

A first person shooter game might have high actions per second which the CPU needs to quickly move off to the network which may favor compute optimized C family instances, while a turn-based strategy game which can spend more CPU cycles processing per turn may need increased memory from R family instances to locally store and update the state of the game on the server before sending it back to players. Use a data-driven approach like the Utilization Saturation and Errors (USE) Method to make well informed architectural choices.

Scalability and elasticity

Evaluate how quickly and smoothly each hosting option can scale to meet player demand without compromising performance. Consider the level of automation and flexibility required for your game's workload to maintain a smooth gaming experience during peak times. A game server might scale quickly by increasing utilization through adding additional game server processes on the same instance, where a game backend may scale slower based on the rising active user count and games being played. Your fleet should scale with demand to minimize cost while facilitating

minimal wait time for the players to get into game. Review Amazon EC2 Spot Instance Advisor to gain insight into cost effective available capacity for game server fleets.

Implementation steps

- Evaluate game server resource requirements for CPU, memory, network, and storage to select suitable instance types, considering game-specific performance needs such as high network throughput for FPS games or memory optimization for turn-based strategy games.
- Compare different hosting options such as containers, instances, bare-metal, and managed services by analyzing performance data using frameworks like the USE method. Use these insights to make better decisions about your system architecture.
- Design fleets for scalability and elasticity, leveraging tools like EC2 Spot Instance Advisor to optimize costs while facilitating quick scaling to meet player demand during peak times.

GAMEPERF01-BP02 Consider operational overhead for scaling game servers

Consider the management and operational overhead associated with each hosting option.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Operational overhead

Self-hosted solutions on EC2 or containers can provide more control but also will require more management. Container orchestrators like ECS or EKS can reduce launch times for containerized servers while also increasing networking complexity and maintenance orchestration overhead.

As an example, EKS managed node groups can automate provisioning and lifecycle management of your game servers but do not respect pod disruption budgets when terminating a node, if your game requires longer than the 15 minute termination period to safely complete games, you may need to create lifecycle hooks or consider self-managed nodes with custom controllers to block game interruption.

Managed services like Amazon Game Lift may handle most of the operational overhead but reduce the amount of visibility and control over special requirements for low level networking and security configuration. Choosing a game server solution is a trade-off between the level of customization, control and responsibility you will have for tuning game server performance and scaling behavior.

Implementation steps

- Assess operational overhead for hosting options, balancing control and management effort between self-hosted solutions like EC2, ECS, or EKS and managed services like Amazon Game Lift.
- Use EKS managed node groups for automation but implement lifecycle hooks or custom controllers if your game servers require longer termination periods than the default.
- Weigh the trade-offs between customization, visibility, and operational responsibility when selecting a game server solution.

GAMEPERF01-BP03 Evaluate integration with other AWS services, development environments, target CPU architectures, and features

Evaluate how well each hosting option integrates with other AWS services your game relies on, such as databases, analytics, or content delivery services.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Integration with other AWS services

Seamless integration between services provides operational benefits like improved performance monitoring and efficient secure data delivery between game components, game servers, game backend services and observability solutions.

For example, Coordinating traffic shifts for live games can be complex. Amazon Route 53 will help keep your DNS records up to date which simplifies coordinated traffic shifts. AWS Global Accelerator traffic dials enable you to send a percentage of traffic to another Region and keep your game running during maintenance.

Development environment and tools

Consider the development tools, frameworks, and environments supported by each architecture option. Verify that your chosen option aligns with your game development solution and programming languages, as this can impact your team's ability to optimize and maintain game server performance. Delivering a game across mobile, console, and PC will increase tooling and testing complexity. Cross-system support is particularly important for multi-game studios where centralized services can standardize development best practices across titles.

Target CPU architecture and features

Consider the performance profile of your game engine and game server processes and the level of ARM support available. Evaluate if you can benefit from improved price performance of ARM based Graviton or x86 based AMD64 processors. Do you need to use Intel features like AES-NI encryption, AVX or Turbo Boost? Review Dedicated Host types to identify single versus multi-socket instance families. When using a multi-socket instance family, consider NUMA pinning and L3 cache sharing in your game server processes. Use C-state and P-state configuration to get the best performance for your game by tuning frequency clock and reducing sleep levels.

Implementation steps

- Select hosting options with seamless integration with AWS services like AWS Secrets Manager, ACM, and others to help streamline performance monitoring, secure data delivery, and reduce manual operational tasks.
- Verify compatibility between your hosting option and your development environment, frameworks, and programming languages to optimize and maintain server performance effectively.
- Evaluate CPU architecture requirements, leveraging Graviton for price-performance or x86 for specific features like AES-NI, AVX, and Turbo Boost, and optimize server performance with NUMA pinning and C-state/P-state tuning.

Region selection

GAMEPERF02: How do you determine which geographic Regions to host your game infrastructure?

Choosing the ideal location for your game infrastructure can improve networking performance to players and backend. Considering where your player base is connecting from and how your communities or servers are built is important for long term growth and sustainability in a geographic Region. Deploying decoupled game server infrastructure and backend services can benefit your overall operational efficiency and improve resiliency by using multiple Regions, local zones, and outposts to host your game.

Best practices

- GAMEPERF02-BP01 Select a home Region that is near your players
- GAMEPERF02-BP02 Design an approach that supports placing latency-sensitive game infrastructure close to players to improve performance

GAMEPERF02-BP01 Select a home Region that is near your players

For an initial game launch, you should determine where to deploy infrastructure based on discussions with your business stakeholders, such as publishing teams who determine where the game is expected to be made available to players, and where they are focusing their pre-launch marketing and advertising efforts.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Your business stakeholders should also have mechanisms to stimulate demand to gain a better understanding of player reception and viability. For example, these teams will have mechanisms such as game pre-orders, marketing events and campaigns, public email lists for players to register interest before launch, and other approaches to establish relevant signals to determine where the game will likely have the most players at launch. The game may also use a regional roll out strategy that includes play test and soft-launch phases to determine regional player demand.

Select a home Region that is near your player base and your developers and has the AWS services and features you need to host your game. The home RSegion will be where the game backend services will run, and it may also run game servers. Evaluate a home Region based on services supported, connectivity to edge locations, proximity to failover Regions, and number of Availability Zones. If you are using a Local Zone, consider the parent Region is sometimes located in a different geographic area. As an example: Santiago, Chile Local Zone us-east-1-scl-1a has N. Virginia us-east-1 as its parent Region even though it is geographically closer to Sao Paulo sa-east-1.

Implementation steps

- Identify deployment Regions based on player demand signals from pre-launch activities like pre-orders, marketing campaigns, and interest registrations.
- Choose a home Region close to the primary player base and developers, making sure it supports required AWS services, edge locations, and failover Regions.
- Evaluate Local Zones carefully, considering that the parent Region may differ geographically from the location of the Local Zone.

GAMEPERF02-BP02 Design an approach that supports placing latency-sensitive game infrastructure close to players to improve performance

Separate placement for latency sensitive infrastructure like game servers minimizes the impact of long network routes. Repeatable deployments can make it simple to maintain multiple locations that are more performant for your players. Ping is a common metric that is surfaced in game UI and low ping can be a differentiating capability.

Level of risk exposed if this best practice is not established: High

Implementation guidance

When first launching a game, you may not yet have enough information about your player base to adequately know where best to deploy infrastructure closest to the players that are most interested in playing your game. This is a common challenge, and you should prepare for this scenario by designing an architecture that allows you to rapidly adjust your hosting placement strategy to deploy servers where they are needed closer to players. It is typical for game developers to regularly assess their game infrastructure deployment as a recurring post-launch analysis to incrementally invest in improvements over time with an iterative approach.

A best practice is to use infrastructure-as-code templates, such as AWS CloudFormation or Terraform by Hashicorp, for the configuration of your infrastructure such as VPCs, subnet configurations, and dependencies required to launch critical game services so that you can refer to these templates, quickly customize them if needed, and deploy them into locations where additional infrastructure is needed to support your players.

You should also make sure you understand how your current deployment strategy could be evolved to allow future expansion. IaC templates are repeatable but are not a substitute for network planning. IPAM manages your VPCs. Subnet sizing, Availability Zone selection, and IP inventory and cross-account Availability Zone alignment. The network is important to consider and can be disruptive to players when changed. Game servers deployed across multiple geographic locations will connect to your game backend, which is more common to be hosted in a single or multiple home Regions which can require additional configuration to support private connectivity. These considerations should be continuously evaluated over time so that you can make changes to your game hosting strategy as your game's requirements evolve or your player requirements change.

When determining how many game hosting locations to use for your game, consider the following factors:

- **Quality of player experience improvement:** How much of a player experience improvement can you introduce by adding additional game hosting locations? What is the incremental performance gain that you can achieve by doing so? How will you measure this performance improvement?
- **Which player populations to prioritize:** How many players can you improve the experience for if you add additional game hosting locations? Which player populations, or geographic locations, will you prioritize?
- **Downstream impacts of change:** If you change your game hosting strategy, how will this influence your matchmaking wait times for players? Can the match sizes, skill balance or number of players in the player pool accommodate a game hosting location strategy change? Supporting more locations can potentially fragment the player pool and add increased cost and complexity.

Each of these considerations should be evaluated as you determine where you add or remove game hosting locations. For example, you may choose to prioritize improving the experience for players in geographic locations with the least performant gameplay experience, or for players who express the most vocal public feedback. You may also choose to factor the player monetization into your priorities, for example by focusing attention on improving the experience for players in geographic locations that generate a significant source of revenue for your game or have the potential to generate incremental revenue if you introduce performance improvements.

In addition to hosting infrastructure in AWS Regions, you can use Local Zones, which are an extension of an AWS Region, to host your game servers and other latency sensitive applications such as voice chat servers closer to your players. You might also choose to run game development infrastructure in Local Zones to improve the experience for your game development teams. For example, you can use Local Zones to address use cases such as hosting replicas of your self-managed source control servers closer to your game developers, and to offer game development virtual workstations and content storage to users using Amazon EC2 instances, EBS volumes, and Amazon FSx file systems deployed into one or more Local Zones near your development studios without requiring you to host the infrastructure on-premises.

Outposts are a good choice when Regions or Local Zones are not available in the same geographic area. Connectivity from your data center to AWS should be considered to enable game server to backend system reliability. AWS Outposts and Outpost Servers are purpose-built to run AWS in your datacenter using the same services and APIs to help create a consistent deployment model wherever you run your game. Multiple racks can be combined into a logical Outpost, and the infrastructure can be shared across AWS accounts. The hardware lifecycle is managed by AWS and the lead time can be as short as 3 months.

If you are building games using containers and want the flexibility to adopt a hybrid deployment architecture using open-source software that can be deployed on your own on-premise infrastructure, you can use ECS Anywhere, or EKS Anywhere as an alternative to AWS Outposts or Local Zones. If you host with Amazon GameLift; Amazon GameLift Anywhere can be used to run your server build on local hardware which can speed up your development process, enabling you to use Local Zones or register your own metal as part of the your fleet.

Implementation steps

- Use infrastructure-as-code tools like AWS CloudFormation or Terraform for repeatable deployments, enabling quick customization and scaling of game hosting locations based on player needs.
- Evaluate player experience improvements, player population priorities, and downstream impacts such as matchmaking times when adding or removing game hosting locations.
- Use AWS Local Zones, Outposts, or hybrid options like ECS Anywhere, EKS Anywhere, or GameLift Anywhere to optimize latency-sensitive infrastructure and support diverse deployment needs.

Iterative development

GAMEPERF03: How can you use Amazon GameLift for iterative development performance efficiency?

Amazon GameLift provides an end-to-end workflow for development and performance testing of your game in a local test environment.

Best practices

- GAMEPERF03-BP01 Use Amazon GameLift Anywhere and a GameLift testing toolkit
- GAMEPERF03-BP02 Test performance and scalability of game servers
- GAMEPERF03-BP03 Optimize resource utilization of GameLift containers

GAMEPERF03-BP01 Use Amazon GameLift Anywhere and a GameLift testing toolkit

To enhance performance efficiency through an iterative development process, utilize Amazon GameLift Anywhere along with the Amazon GameLift Testing Toolkit to establish a comprehensive testing environment.

Level of risk exposed if this best practice is not established: High

Implementation guidance

This approach allows rapid iteration, efficient data collection, and detailed performance analysis. Key steps include:

Create a test environment

Use Amazon GameLift Anywhere to set up a local or cloud-based test environment. This setup removes the need to upload each game server build iteration to a managed fleet, reducing the activation time.

Integrate Amazon GameLift Testing Toolkit

Incorporate the Amazon GameLift Testing Toolkit into your development workflow. The toolkit provides scripts, tools, and libraries to visualize Amazon GameLift infrastructure, launch virtual players, and iterate upon FlexMatch rule sets with the FlexMatch simulator. It simplifies the integration and management of Amazon GameLift resources, allowing you to automate common tasks and gather necessary data for performance analysis.

Rapid build and test cycles

Quickly update the test fleet with new builds, start it, and commence testing. This facilitates a fast build-test-repeat cycle, enabling developers to validate various aspects of the game's player experience, including multiplayer interactions.

Comprehensive testing

Test your game server integration with the Amazon GameLift server SDK, backend service interactions, matchmaking configurations, and other GameLift hosting features. Utilize the GameLift Testing Toolkit to automate testing and gather detailed performance metrics, making sure that game components work seamlessly together.

Analyze performance data

Use the data collected by the GameLift Testing Toolkit to analyze performance bottlenecks and optimize your game server. The toolkit helps track key metrics, identify issues, and make data-driven decisions to improve performance efficiency.

By incorporating Amazon GameLift Anywhere and the GameLift Testing Toolkit into your iterative development process, you can significantly enhance performance efficiency through rapid testing, comprehensive integration checks, and detailed performance analysis.

Implementation steps

- Use Amazon GameLift Anywhere to create a test environment, reducing activation time for game server builds and enabling rapid iteration.
- Integrate the Amazon GameLift Testing Toolkit to automate testing tasks, simulate players, and validate FlexMatch configurations during development.
- Collect and analyze performance data with the GameLift Testing Toolkit to identify bottlenecks, optimize game servers, and enhance performance efficiency.

GAMEPERF03-BP02 Test performance and scalability of game servers

To test the performance and scalability of your game servers, implement a robust testing framework using the Amazon GameLift features and the GameLift Testing Toolkit.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Key practices include:

Iterative testing

Use an Amazon GameLift Anywhere fleet to create a cloud-based hosted environment where you can iteratively build and test game components. This environment should mirror real-world hosting conditions, enabling realistic performance and scalability testing.

Game server integration testing

Test the integration of your game server with the Amazon GameLift server SDK, including starting new game sessions and tracking game session events using AWS CLI or GameLift Testing Toolkit. This verifies that the game server functions correctly within the GameLift environment.

Use the GameLift Testing Toolkit to automate testing and gather detailed performance metrics. The toolkit allows you to visualize GameLift infrastructure, launch virtual players for load testing, and iterate on FlexMatch rule sets with the FlexMatch simulator. It is particularly useful for scaling ECS Fargate tasks, which simulate player sessions by creating numerous concurrent game sessions to stress test the server infrastructure.

Scalability testing

Experiment with game session queue designs, multi-location fleets, Spot and On-Demand fleets, and multiple instance types. Test game session placement options, latency policies, and fleet prioritization settings. Configure capacity scaling to meet player demand and validate that the system can handle the expected load under different conditions.

Implementation steps

- Use Amazon GameLift Anywhere to set up a realistic test environment for iterative performance and scalability testing.
- Test game server integration with the GameLift server SDK, facilitating correct session management and event tracking within the GameLift environment.
- Perform scalability testing with the GameLift Testing Toolkit, simulating player load, testing session queues, and validating fleet scaling, latency policies, and prioritization settings.

GAMEPERF03-BP03 Optimize resource utilization of GameLift containers

To optimize resource utilization of GameLift containers, design your container fleet effectively and set precise resource limits.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Key guidelines include:

- **Container group design:** Organize your software into container groups. The primary container should bundle your game server application and the Amazon GameLift Agent. Use sidecar containers for additional software to manage dependencies and set container-specific limits for memory and CPU usage.

- **Set resource limits:** For each container group, determine the required memory and CPU resources. Set optional limits for individual containers to verify they have reserved resources but can also exceed these limits if additional resources are available. This helps prevent resource contention and potential container failures.
- **Daemon container group:** Consider using a daemon container group for background or monitoring processes that do not need to scale with the primary container group. This verifies that essential background tasks are handled efficiently without impacting the primary game server processes.

Implementation steps

- Design container groups with a primary container for the game server and GameLift Agent, and sidecars for managing dependencies, with specific memory and CPU limits.
- Set resource limits for each container group to reserve required resources while allowing controlled resource usage to avoid contention.
- Use a daemon container group for background or monitoring tasks, making sure they operate efficiently without affecting primary game server processes.

Compute and hardware

GAMEPERF04: How do you block game sessions from impacting players running on the same game server instance?

After your game server is running in AWS, you need to monitor its performance to provide a quality player experience regardless of resource utilization, underlying compute, or saturation.

Best practices

- GAMEPERF04-BP01 Monitor game server processes to detect issues
- GAMEPERF04-BP02 Performance test your game server with simulated and real gameplay scenarios

GAMEPERF04-BP01 Monitor game server processes to detect issues

You might run multiple game server processes per instance to efficiently utilize the resources on your game server instances. If so, design your architecture so that an individual game server process hosting a game session cannot cause adverse impact to other game sessions hosted on the same instance. Use metrics to understand how game placement and game mode type can impact the performance of game server instances. Incorporate a mix of low load (lobby, shop, or single player tutorial) and high load (ranked, multi-player, or high skill gameplay) processes to avoid hot spotting the game server instance.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Monitor the player experience through client side and server-side metrics by collecting telemetry for ping time and jitter, frame drops, API response time, errors and successful game loop completion. Correlate time stamps for these events with player support issues and server logs to identify performance bottlenecks. Tools like Dtrace, ftrace, uperf, and eBPF can be used for deep investigation and analysis of system performance.

Implement monitoring of the limited resources available to your game server instances so that you can generate alerts when individual game server processes are breaching pre-determined resource budget thresholds. When thresholds are breached, you may want to configure your game server software to dump relevant system and game server logs out to durable storage, such as a central logging solution, so that your game server engineers can investigate this behavior. Additionally, your game server instance should be configured to report metrics from each of the game server processes running on the instance so that you can monitor these individual game server processes in addition to the overall metrics for the game server instance.

For example, GameLift provides metrics for monitoring game sessions, which can be augmented with custom game-specific metrics and logs collected using the Amazon CloudWatch Agent which you can configure on your game server instance. Your metrics can be viewed in CloudWatch or exported to other tools such as Amazon Managed Grafana which is integrated with Single Sign-On to make it straightforward to access metrics by users who may not have access to the Management Console. Refer to the following best practices for managing logs and metrics using Amazon GameLift, which also provides support for viewing individual game session logs.

Implementation steps

- Run multiple game server processes per instance and mix low and high-load game modes to avoid hot spotting and verify balanced resource utilization.
- Monitor client-side and server-side metrics like ping, jitter, frame drops, and API response times, and correlate these with server logs and issues reported by players to identify bottlenecks.
- Configure resource monitoring for each game server process, generate alerts for threshold breaches, and store logs in durable storage for analysis using tools like CloudWatch and Amazon Managed Grafana.

GAMEPERF04-BP02 Performance test your game server with simulated and real gameplay scenarios

Conduct performance testing and evaluate various gameplay scenarios to determine whether the game server process handles the utilization of fixed resources appropriately, such as EC2 instance memory, CPU, and network bandwidth.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Creating simulated gameplay tests with bots that can mirror common gameplay paths and behaviors of your players can determine how your game server processes handle this under different usage scenarios. For example, you can implement a solution, such as Distributed Load Testing on AWS that you can customize to run game client simulations or game client builds to generate gameplay scenarios. Run internal play tests and use QA teams to stress test the various features of your game so that you can develop confidence that your game is designed to perform optimally. AWS Device Farm can be used to perform mobile and web testing for your iOS, Android, and browser games on multiple device types.

Implementation steps

- Conduct performance testing with bots simulating common player behaviors to evaluate game server resource utilization under different scenarios.
- Use solutions like Distributed Load Testing on AWS to customize and simulate gameplay scenarios for stress testing.
- Perform internal playtests and use tools like AWS Device Farm for mobile and browser game testing on various devices.

Compute selection

GAMEPERF05: How do you select the appropriate compute solution for your game?

Compute performance varies across instance sizes and families. It is beneficial to use multiple compute options that are from separate capacity pools. Develop a fleet composition strategy that gives preference to performance but includes enough diversity to avoid insufficient capacity errors.

Best practices

- GAMEPERF05-BP01 Benchmark your game performance across multiple compute types
- GAMEPERF05-BP02 Move non-latency-sensitive compute tasks to asynchronous workflows

GAMEPERF05-BP01 Benchmark your game performance across multiple compute types

For game server workloads, there is no singular approach to identifying the optimal compute solution for hosting your game server. A common strategy for benchmarking game servers is to start with compute-optimized EC2 'c' instances, because this instance family provides high performance for workloads that are computationally intensive. Alternatively, if your game requires a significant amount of memory to implement specific features, the memory-optimized instances may be most suitable.

Level of risk exposed if this best practice is not established: High

Implementation guidance

If your workload utilizes significant network resources, consider implementing instances that are network-optimized which is typically indicated using an 'n' in the instance name, avoid burstable instance types 't' as after credits are exhausted performance will decrease. Games are sensitive to latency and dropped packets, so it is recommended to use EC2 enhanced networking to help improve the network performance of your game servers. Enhanced networking uses single root I/O virtualization (SR-IOV) to provide high-performance networking capabilities on supported instance types . SR-IOV is a method of device virtualization that provides higher I/O performance and lower CPU utilization when compared to traditional virtualized network interfaces. Enhanced networking provides higher bandwidth, higher packet per second (PPS) performance, and consistently lower

inter-instance latencies. Enhanced networking with Elastic Network Adapter is available for most recent EC2 instance types and is important to regularly update to benefit from performance enhancements from newer instances and improvements to the AWS Nitro hypervisor.

If your game performs similarly across multiple EC2 instance types, then you should consider using multiple instance types to host your game servers. Monitor performance over time and perform further optimization after you have hosted enough production game sessions to be able to identify performance trends. Remember that your compute requirements may change as you add new features into your game that require different allocation of resources. You can configure EC2 Auto Scaling groups to use multiple instance types, or you can use separate Auto Scaling groups to host game server instances that run separate instance types which may make it simpler to manage correlation and aggregation of metrics.

Evaluate how your game performs on different types of processors such as Intel-based instances, AMD-based instances, and ARM-based Graviton instances. Unreal Engine 5.1.1 or newer can compile game servers for Graviton and can improve price performance for your game. Perform sweep and saturation testing at various sizes within each family to determine the sweet spot where utilization and performance are consistent.

You should also benchmark how your game performance is impacted when it is hosted using containers and Lambda functions. For use cases where long-lived game server processes are not required, such as asynchronous games and for game backend services, consider using a serverless architecture with Lambda which can simplify management and operations for game operations teams, as well as allow you to more quickly deploy your game globally to many AWS Regions. For serverless best practices, refer to the Serverless Applications Lens - Well-Architected Framework.

Implementation steps

- Benchmark game servers on compute-optimized 'c' instances for CPU intensive workloads, memory-optimized instances for memory heavy task, and network-optimized 'n' instances for high network throughput.
- Use enhanced networking with Elastic Network Adapter (ENA) on supported instances to improve network performance, reduce latency, and increase packet processing rates.
- Evaluate and test multiple instance types, processors (Intel, AMD, Graviton), and container or Lambda hosting options, adjusting compute solutions as game features evolve.

For more information, see Choose the right compute strategy for your global game servers.

GAMEPERF05-BP02 Move non-latency-sensitive compute tasks to asynchronous workflows

When you are optimizing the performance for your game, it is important to keep in mind that only some of the interactions between the client and the game backend must be performed in a synchronous manner. You should consider each feature from the perspective of the player experience and determine whether certain interactions require synchronous communications, which are blocking and resource intensive, or whether those features can be implemented in an asynchronous manner. When you implement network calls, use an asynchronous, non-blocking approach. Additionally, your game backend should also be configured to perform work in an efficient manner by offloading tasks to queues and prioritizing fast responses to clients where possible.

Level of risk exposed if this best practice is not established: High

Implementation guidance

For example, updating a leader board at the end of a player session can be implemented asynchronously so that the client does not need to wait for the leader board update to complete. Instead, implement this asynchronously on the game client, and consider designing your backend service to push these types of operations into queues, such as Amazon SQS. With this architecture, configure your backend to accept the request, enqueue it in SQS which helps durably store messages for asynchronous processing, and promptly reply to the client. When the leader board update is completed, the backend can send an update to the game client so that the player's view of the leader board is updated.

Alternatively, the player can simply visit your game's leader board screen to retrieve the latest data, which can issue a web request to your backend to retrieve the latest data from cache.

Implementation steps

- Determine if client-backend interactions require synchronous communication; implement asynchronous, non-blocking approaches where possible to optimize resource usage.
- Use Amazon SQS to offload non-critical tasks like leaderboard updates.
- Allow the client to fetch updated data asynchronously, such as retrieving the latest leaderboard data on demand or via background updates.

Resources

- Understanding asynchronous messaging for microservices
- Lambda - Using service integrations and asynchronous processing

Data management

GAMEPERF06: How do you efficiently manage and analyze game server logs and store different types of game data for optimal performance?

Games can have player data, game logs and server logs which should be decoupled from each other where possible. Centralizing log ingestion and lifecycle management can benefit your game teams by providing insights into what's happening in game and on the server.

Best practices

- GAMEPERF06-BP01 Centralize log collection and storage
- GAMEPERF06-BP02 Categorize and store game data based on access patterns
- GAMEPERF06-BP03 Enable efficient log formatting and batching
- GAMEPERF06-BP04 Implement log rotation and retention policies
- GAMEPERF06-BP05 Use monitoring and visualization tools

GAMEPERF06-BP01 Centralize log collection and storage

Implement a centralized log collection and storage solution to gather logs from game server instances and GameLift.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Use services like Amazon CloudWatch Logs to collect, monitor, and store log data from your game servers and GameLift instances. CloudWatch Logs provides a scalable and fully managed solution for log management, facilitating efficient storage and retrieval of log data without impacting game server performance. If you are running the CloudWatch Logs agent, consider the various install

types and configuration options like batch size, buffer duration to minimize impact to the game server. Consider the game server instances ephemeral and reduce dependency on localized logging where possible. Establish a centralized policy for implementation of Logging best practices.

Implementation steps

- Use Amazon CloudWatch Logs to collect, monitor, and store log data from game server instances and GameLift, facilitating centralized and scalable log management.

GAMEPERF06-BP02 Categorize and store game data based on access patterns

Categorize your game data into different types based on their access patterns and storage requirements.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Common categories include player data, game saves, persistent world storage, and analytics data.

Implementation steps

Use appropriate storage solutions for each data type to optimize performance and cost-efficiency:

- **Player data:** Use Amazon DynamoDB, a fast and scalable NoSQL database, to store player profiles, preferences, and progression data. The low-latency access and automatic scaling capabilities of DynamoDB provide efficient retrieval and update of player data.
- **Game saves:** Use Amazon S3 to store game saves and checkpoints. S3 provides high durability and scalability for storing large amounts of game save data. Consider using S3 Transfer Acceleration or Amazon CloudFront for faster uploads and downloads of game saves.
- **Persistent world storage:** For games with persistent world states or shared game data, consider using Amazon DynamoDB, Amazon ElastiCache or Amazon MemoryDB. ElastiCache and MemoryDB provide in-memory key-value store while DynamoDB is an SSD backed NoSQL database. These services provide fast access to stored data, reducing the time it takes for the game server process to save game state which improves overall process performance.
- **Analytics data:** Use Amazon Managed Streaming for Apache Kafka or Kinesis Data Streams to ingest data streams from your game data producers. Amazon Managed Service for Apache Flink

can be used for real-time transformation and analysis and sent to Amazon Data Firehose for processing and delivery into backend data lakes, warehouses and analytics services. Guidance for Game Analytics Pipeline on AWS illustrates how the services work together to provide near real-time and batch analytics.

GAMEPERF06-BP03 Enable efficient log formatting and batching

Configure your game server processes to generate logs in a structured and in a format that can be parsed, such as JSON.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Implement log batching techniques to minimize the frequency of log data transfers from your game servers to the centralized log storage. Batching logs reduces network overhead and improves game server performance. Use verbose or debug level logs as an exception and not a default, as they can incur a performance and cost penalty that should be avoided when possible.

GAMEPERF06-BP04 Implement log rotation and retention policies

Establish log rotation and retention policies to manage the growth of log data and optimize storage utilization.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Configure your game servers to automatically rotate logs based on size or time intervals. Define log retention policies in Amazon CloudWatch Logs to automatically archive or delete older log data that is no longer needed for active analysis or troubleshooting.

GAMEPERF06-BP05 Use monitoring and visualization tools

Use monitoring and visualization tools to gain insights into your game server performance and identify optimization opportunities.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Use Amazon CloudWatch to monitor key metrics and set up alarms for proactive notifications. Utilize tools like Amazon Managed Service for Prometheus and Amazon Managed Grafana to collect, query, and visualize metrics from your game servers and infrastructure. Create informative dashboards to track performance, identify bottlenecks, and make data-driven optimizations.

Networking and content delivery

GAMEPERF07: How do you design your matchmaking service to optimize performance?

Player skill, internet service provider (ISP) quality, and player population distribution are important to consider as a dimension of performance tuning. Game sessions can be placed on servers that are strategically located to level the playing field and host a fair game.

Best practices

- GAMEPERF07-BP01 Define network latency thresholds for your game
- GAMEPERF07-BP02 Run a separate matchmaking service for each gameplay mode and game hosting Region
- GAMEPERF07-BP03 Regularly monitor matchmaking performance
- GAMEPERF07-BP04 Regularly monitor networking performance
- GAMEPERF07-BP05 Use network acceleration technology to improve performance across the internet

GAMEPERF07-BP01 Define network latency thresholds for your game

When developing a multiplayer game, verify that your game infrastructure does not introduce unnecessary latency for players. If your game is sensitive to network latency, then you should set latency thresholds in your matchmaking logic to prioritize placing players on game server sessions that are hosted in nearby game server locations or AWS Regions that meet your objective for ideal player experience.

Level of risk exposed if this best practice is not established: High

Implementation guidance

In many latency-sensitive games it is common to instrument the game clients to ping each of the game's infrastructure locations to gather performance data such as network latency, jitter, and packet loss, and report this data to the metrics collection backend so that it can be analyzed. When matching players into game sessions, you can configure your game to incorporate the game client's perceived network latency to your game server infrastructure as one of the inputs used in your matchmaking service logic.

GAMEPERF07-BP02 Run a separate matchmaking service for each gameplay mode and game hosting Region

If your game offers multiple gameplay modes for players to choose from, you should separate the matchmaking systems for each of them so that you can independently tune the performance for each gameplay mode based on its unique requirements and reduce resource contention. Each gameplay mode will likely have unique requirements for acceptable latency, match size, as well as other customize game-specific matchmaking logic. They will also likely attract different types of players. Run each game mode's matchmaking service as a separate software deployment so that you can performance test and operate the game modes independently.

Level of risk exposed if this best practice is not established: High

Implementation guidance

For example, you might run these as separate Lambda functions for each game mode, or you might operate them as separate container-based service deployments.

Deploy your matchmaking services to multiple Regions near your game server locations. Player traffic will take many routes, so it is important for the matchmaking service to maintain an up-to-date latency profile across multiple ISPs to improve the efficiency of low latency game session placement. GameLift FlexMatch provides additional guidance for selecting Regions for matchmakers, and includes the ability to integrate your matchmakers with multi-Region game session queues.

GAMEPERF07-BP03 Regularly monitor matchmaking performance

One of the most noticeable ways to optimize the performance of a game for players is to reduce the time that they must wait before they can enter a game session. Long wait times can cause

players to lose interest and lead to attrition, so it is important to consider this when designing your matchmaking solution.

Level of risk exposed if this best practice is not established: High

Implementation guidance

When you are designing your matchmaking configuration for your game, create rules that determine the conditions that are applied to form a match. You should consider the impact that these rules will have on the performance of the system, particularly the wait times for players. Before deploying changes to your matchmaking implementation, such as the addition of new matchmaking conditions or filters, test this beforehand or consider releasing this change gradually to a small sample population of players as a canary or A/B test to gather performance metrics before introducing this change to the entire player population.

Configure your matchmaking service to generate detailed logs to understand the conditions or rules that were applied to each matchmaking request. This assists in the review and to adjust matchmaking implementation as necessary.

For example, Amazon GameLift FlexMatch provides a fully managed matchmaking service which can be used as a standalone service with your own game server hosting or used with game servers hosted on Amazon GameLift. FlexMatch can generate event notifications to Amazon EventBridge, see Set up FlexMatch event notifications. Use Amazon Simple Notification Service (Amazon SNS) to receive matchmaking data in JSON format, allowing you to automatically process and store this information for analysis to improve matchmaking performance.

Set up metrics to track how long your matchmaking service takes to find a suitable game session for players. Review matchmaking duration metrics regularly and correlate these times with player behavior and community sentiment. Use this data to develop suitable thresholds for matchmaking timeouts that can be included in your matchmaking rule configuration.

For example, Amazon GameLift FlexMatch provides support for defining matchmaking request timeouts as well as creating matchmaking rules that can allow requirements to relax over time. This feature allows you to create matchmaking that can adapt to make it straightforward to create matches and place players into game sessions when matches are difficult to find.

GAMEPERF07-BP04 Regularly monitor networking performance

For competitive games, it is important to have a consistent player experience.

Level of risk exposed if this best practice is not established: High

Implementation guidance

A game that is reliably 50ms for a larger player base is fairer and more fun than a match where one player has 10ms ping and another who has 70ms ping. ISP routing changes may impact part of the player population, and your matchmaking system will need to adapt. Amazon CloudWatch Network Monitoring assists in determining whether the issue is with your game or the player internet provider.

Implementation steps

- Use Amazon Cloudwatch Network Monitoring to track network performance and identify routing issues.
- Use VPC Flow Logs to identify abnormal traffic patterns or dropped packets, which can indicate network congestion, ISP issues, or misconfigurations impacting player latency.

GAMEPERF07-BP05 Use network acceleration technology to improve performance across the internet

In addition to physically placing latency-sensitive game infrastructure closer to players, you can also improve the player experience by optimizing the network performance for your game. AWS uses the BGP protocol to influence internet routing to use the fastest path to our border network from Internet Service Providers. If you operate your own network and need more control and observability over routing behavior and BGP advertisement, you can use private Peering or Direct Connect to route traffic from the internet to your game running on AWS.

Level of risk exposed if this best practice is not established: High

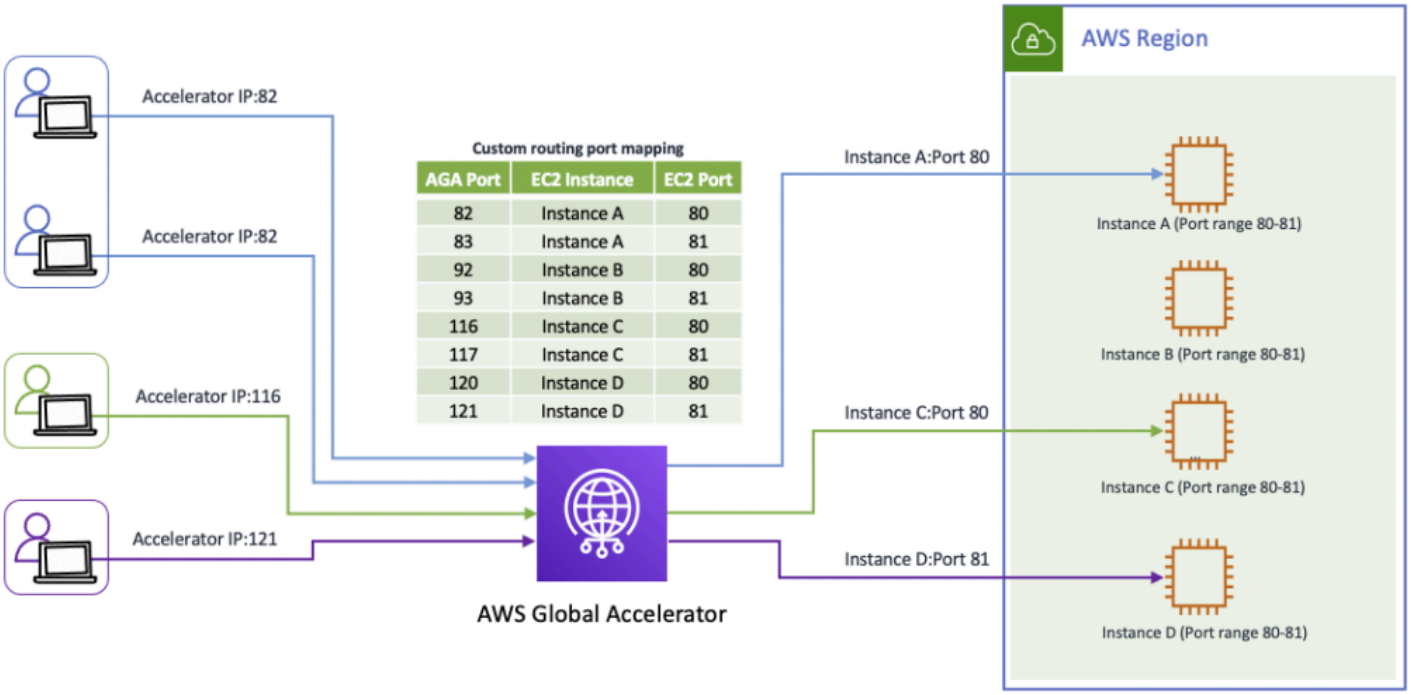
Implementation guidance

Consider the following reference architecture to support improved internet performance and responsiveness.

Enhanced network performance for gaming using Global Accelerator

For a fully managed solution to network routing, AWS Global Accelerator improves your application's network performance using the AWS global network, which can be used to accelerate your gameplay traffic, voice chat, and real-time messaging traffic, as well as other latency-sensitive

applications while providing fast failover to your game servers. Global Accelerator custom routing accelerators can be integrated with your matchmaking service to provide deterministic routing of multiple players to the same game session using static anycast IP addresses and ports.



Your game development teams may be distributed around the world and require performant access to shared content or assets. To improve the performance for shared content stored in Amazon S3 buckets, you can setup bi-directional replication of your data across Regions using S3 Cross-Region Replication so that users can access data from buckets closer to them. To simplify this access pattern, use S3 Multi- Region Access Points which accelerates requests to S3 over the global network using Global Accelerator.

For more information, refer to Improving the Player Experience by Leveraging AWS Global Accelerator and Amazon GameLift FleetIQ.

Implementation steps

- Use AWS Global Accelerator to help improve network performance for gameplay traffic, voice chat, and real-time messaging, while facilitating fast failover to game servers.
- Configure Global Accelerator custom routing accelerators to integrate with your matchmaking service, enabling deterministic routing of players to game sessions using static anycast IPs.
- Enable S3 Cross-Region Replication to replicate shared content across Regions for distributed game development teams.

- Use S3 Multi-Region Access Points to accelerate S3 data access over the AWS global network for globally distributed users.

Process and culture

GAMEPERF08: How do you align the performance bar for your game with player and developer expectations?

Knowing your player and developer is one of the most important aspects of improving performance efficiency. Delivering a performant game with low operational overhead is one of the best ways to show players and developers that you care about their experience and can differentiate your game and studio.

Best practices

- GAMEPERF08-BP01 Inform and include the player in your process
- GAMEPERF08-BP02 Align solution selection with engineering team skills and expertise

GAMEPERF08-BP01 Inform and include the player in your process

Provide an option to display in game metrics like latency, frames per second and dropped packets. Surface infrastructure issues and maintenance downtime through player facing communication like status pages. Celebrate new game locations with player comms including dev blogs and set expectations for expected player experience improvements.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Include the player

Provide a simple diagnostic submission process that collects relevant files and attaches them to a player support ticket from your game client. Enable a support forum where players can help each other and become a part of improving the game experience

Consider trade-offs versus player expectations

Moving backend systems for cost efficiency may not be noticeable to players but moving game servers can change ping time. Be consistent and fair to players with reasoning for expansion and reduction of your game hosting locations.

Player communities and geographies will have their own characteristics that may impact expectations of your game. For example, South Korea has some of the fastest internet on the planet and the expectation for gameplay is single digit latency which drives highly competitive play. Casual gameplay on mobile devices creates a different performance profile and use pattern in comparison to console and PC session play.

Login and lobby are a part of the experience and should feel responsive, even if the server is offline for maintenance. Raid night planning or hanging out in the lobby is part of the player experience and is important to consider when choosing focus areas for performance efficiency. Players may leave your game client open for months, sometimes they may just log in occasionally to read the patch notes. A Live Ops game needs to keep the entire player experience in mind as a part of engineering process and culture.

Implementation steps

- Provide in-game metrics such as latency, FPS, and packet loss, and communicate infrastructure issues and maintenance schedules via status pages and player-facing updates.
- Implement a diagnostic dump and submission feature in the game client and create a support forum to foster community-driven troubleshooting and improvement.
- Tailor performance optimizations to player community expectations, such as low latency for competitive Regions or responsive login/lobby experiences for casual and long-session players.
- Design Live Ops workflows to account for the entire player experience, from active gameplay to idle client behavior, facilitating seamless engagement.

GAMEPERF08-BP02 Align solution selection with engineering team skills and expertise

Assess your team's skills and expertise in managing and optimizing game server performance when choosing your hosting option. Self-hosted solutions like EC2 and containers require more knowledge of infrastructure management, performance tuning, and scaling. If your team lacks these skills, a managed service like GameLift may be more suitable, as it abstracts away many of the complexities and allows your team to focus on game-specific optimizations.

Level of risk exposed if this best practice is not established: High

Implementation guidance

By evaluating these factors and conducting performance tests across different hosting options, you can select the most appropriate solution that meets your game's specific requirements while optimizing for performance efficiency.

Resources

Learn more about our best practices related to performance efficiency.

Related documents:

- AWS Architecture Center
- Performance Efficiency Pillar – AWS Well-Architected Framework
- Comparing your on-premises storage patterns with AWS Storage services
- Instance store temporary block storage for EC2 instances
- Collect metrics, logs, and traces using the CloudWatch agent
- CloudWatch Agent
- How do I turn on and configure enhanced networking on my EC2 instances?
- Improving the Player Experience by Leveraging AWS Global Accelerator and Amazon GameLift FleetIQ
- Riot Games Technology Blog: Scalability and Load Testing For Valorant
- Hyper-scale online games with a hybrid AWS Solution
- Utilization Saturation and Errors (USE) Method
- Amazon EC2 Spot Instances
- Performance at Scale with Amazon ElastiCache
- Database Caching Strategies Using Redis
- Amazon Virtual Private Cloud Connectivity Options
- Best Practice Design Patterns: Optimizing Amazon S3 Performance

Related benchmarks:

- Benchmark Amazon EBS volumes
- Amazon EC@ instance network bandwidth

Related tools:

- Unreal Engine: Testing and Optimizing Your Content
- Unity Profiler
- Open 3D Engine (O3DE) Quality System
- Monitoring Amazon GameLift Servers
- Amazon GameLift Testing Toolkit

Related videos:

- AWS re:Invent 2019: [REPEAT 2] Amazon EC2 foundations (CMP211-R2)
- AWS re:Invent 2019: Powering next-gen Amazon EC2: Deep dive into the Nitro system (CMP303-R2)
- Getting Started with Amazon GameLift FleetIQ – AWS Online Tech Talks
- Zach Blitz of Riot Games on Using AWS to Improve Gaming
- AWS re:Invent 2023 - AWS Graviton: The best price performance for your AWS workloads (CMP334)

Related training:

- Game Server Hosting on AWS
- Using Amazon GameLiftFleetIQ for Game Servers
- Getting Started with AWS for Games – Part I
- Game Server Hosting on AWS
- AWS re:Invent 2023 – AWS Graviton: The Best price performance for your AWS workloads (CMP334)

Cost optimization

The cost optimization pillar includes the continual process of refinement and improvement of a system over its entire lifecycle. This process spans from the initial design of your first proof of concept to the ongoing operation of production workloads. By adopting the practices outlines in this paper, you can build and operate cost-aware systems that achieve desired business outcomes at a lowest price point. Implementing these cost optimization practices can allow your business to maximize the value of your cloud investment.

Games are unique creative projects that must compete for player attention and playtime. Prior to launch, game developers often lack a clear understanding of how popular or long-lasting their game will become. Depending on the game's monetization strategy, business priorities, and lifecycle stage, developers will need to make tradeoffs when evaluating cost optimization decisions.

For example, during the pre-launch phase of a highly anticipated new game, the focus is typically on speed-to-market, feature development, and performance. The priority is verifying the infrastructure can scale to meet peak player demand. Conversely, if a game is not successful or development is slowing down, the focus may shift to reducing costs as much as possible to continue operating the game for existing players.

Many game developers also operate multiple games simultaneously, which requires additional considerations. Resources like infrastructure, software, and staff may be shared across multiple live games, allowing losses from one game to be offset by profits from another. In this scenario, a focus on cost optimization can improve the financials across the entire game portfolio.

Given the unique business models, scale, and unpredictability of games, the following key questions can guide cost optimization decisions:

- How can I measure the infrastructure cost per player, system, and game feature?
- What is the right balance between cost optimization and player experience for my game's current lifecycle stage?
- How can I maximize return on investment using the right pricing model for my AWS resources?

Applying these best practices and asking the right questions can assist game developers build and operate cost-aware systems that achieve business outcomes while minimizing costs.

Focus areas

Design principles

In addition to the design principles from the cost optimization pillar of the Well-Architected Framework, the following design principles optimize the costs of running your game workload in the cloud.

- **Measure the infrastructure cost per player, system, and game feature:** Understand and track the infrastructure costs required for specific player experiences and features across game systems. This can identify areas of your architecture that may need cost optimization.
- **Assess the trade-off of cost optimization versus player experience:** Evaluate what stage your game is in to determine the right focus - player experience or cost optimization. Typically, once a game reaches critical mass and player population stabilizes, it is time to focus on optimizing operating costs. The key is to balance providing a great player experience with running your game infrastructure in the most cost-effective manner. Implementing these design principles maximize the return on your game investments.

Practice Cloud Financial Management

There are no Cloud Financial Management best practices specific to the Games Lens. Refer to the Well-Architected Framework Cost Optimization Pillar for guidance on cloud financial management.

Expenditure and usage awareness

GAMECOST01: How do you measure the cost of your game environments?

Understand the cost per player, game feature, and environment so that you can manage and forecast your spend as the number of players changes over time and features are added and improved. Consider the following best practices to manage your costs of your different game environments.

Best practices

- GAMECOST01-BP01 Implement attribution of cost per player, game feature, and environment
- GAMECOST01-BP02 Discover opportunities for optimization

GAMECOST01-BP01 Implement attribution of cost per player, game feature, and environment

Cost attribution for game servers is usually simpler to perform than game backend services because a game server is usually optimized to be able to host a specific number of concurrent players per instance which can be amortized across the cost of running the instance.

Level of risk exposed if this best practice is not established: High

Implementation guidance

For game backend services, it is recommended to de-couple the components of your game into distinct features that can be managed as separate logical or physical resources to make it straightforward to analyze costs.

For example, although it may seem straightforward to implement a single monolithic application to host game backend services, this pattern makes it hard to derive the total cost per player and game feature over time as you add more features because the compute, networking, and storage costs of resources are shared across the features. Consider adopting a serverless architecture for your game backend services with services such as Amazon API Gateway and AWS Lambda or AWS Fargate for compute, Amazon SQS and Amazon SNS for messaging, Amazon S3 for object storage, and Amazon DynamoDB for database storage. These services are just a few examples of products that offer pricing that is usage-based and primarily driven by request volume so that costs can be visualized with granularity. Individual resources such as Lambda functions, Fargate services, DynamoDB tables, and S3 buckets can be associated with cost allocation tags so that you can attribute the costs of these services with game feature names that make it straightforward for you to understand the costs for each of your services.

It is also recommended to separately manage each of your game development environments so that you can attribute costs for the different environments. Typically, game developers will manage separate environments for development, test, staging and production environments, as described in the operations pillar of this games industry lens. Each environment usually has different scalability, performance, and usage requirements and may be managed by separate teams. To control costs, organize these environments so that you can properly monitor and attribute the costs of each environment.

For more information, refer to the following documentation:

- Building a serverless multi-player game that scales
- Standalone game session servers with a WebSockets-based backend
- Standalone game session servers with a serverless backend

Implementation steps

- De-couple game backend services into distinct features using serverless or containerized architectures like AWS Lambda, Amazon API Gateway, and AWS Fargate to enable granular cost attribution per feature.
- Apply cost allocation tags to individual resources (for example, Lambda functions, DynamoDB tables, and S3 buckets) to associate costs with specific game features for better cost analysis.
- Manage separate environments for development, testing, staging, and production, organizing and monitoring their costs independently to align with scalability and usage requirements.

GAMECOST01-BP02 Discover opportunities for optimization

Game developers and publishers can use AWS FinOps practices to help optimize their cloud costs and gain better visibility into their cloud spending. By doing so, game producers can align the average cost required to maintain infrastructure for the players with the financial results delivered by the game.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

AWS offers a ready to use solution guidance for Cloud Financial Management to manage and optimize your expenses for cloud services. This capability includes granular visibility and cost and usage analysis to support decision-making for topics such as spend dashboards, optimization,

spend limits, charge back, and anomaly detection and response. The solution guidance for Cloud Financial Management includes budget and forecasting features, giving you a defined, cost-optimized architecture for your workloads so you can select the right pricing model and attribute resource costs relevant to your teams. This activates tracking, notification, and cost optimization techniques across your environment and resources. You can centrally manage expense information and give critical stakeholders access as needed for targeted visibility and to support decision-making.

Another key FinOps tool is the Cost Optimization Hub, which provides a centralized view of cost optimization recommendations and opportunities across your AWS accounts and AWS Regions, so that you can get the most out of your AWS spend. You can use Cost Optimization Hub to identify, filter, and aggregate AWS cost optimization recommendations across your AWS accounts and AWS Regions. It makes recommendations on resource rightsizing, idle resource deletion, Savings Plans, and Reserved Instances. With a single dashboard, you avoid having to go to multiple AWS products to identify cost optimization opportunities.

If your games teams are using shared AWS accounts the myApplications in AWS Management Console Home, can be used to view application resource costs for individual workloads. This granular view allows you to identify the specific cost trends within your game infrastructure, enabling you to make informed decisions about resource allocation and optimization.

Additionally, regularly reviewing your billing and cost management data with AWS Data Exports uncovers hidden cost savings opportunities. This detailed report provides a comprehensive breakdown of your cloud spending, allowing you to identify areas of overspending, unutilized resources, and opportunities to take advantage of more cost-effective services or pricing models.

By embracing FinOps principles and leveraging the tools provided by AWS, game developers and publishers can make the most efficient use of their cloud resources, ultimately enhancing their bottom line and freeing up funds for further game development and innovation.

Implementation steps

- Use AWS Cloud Financial Management tools for granular and detailed visibility, spend dashboards, anomaly detection, and cost attribution to optimize and track cloud expenses effectively.
- Use the Cost Optimization Hub to centralize rightsizing, Savings Plans, and Reserved Instance recommendations across AWS accounts and Regions.
- Regularly review AWS billing data using Data Exports and MyApplication on AWS to help analyze workload-specific costs, uncover savings opportunities, and optimize resource allocation.

Cost-effective resources

GAMECOST02: How are you choosing the right compute solution for your game servers?

One of the most unique aspects of a game workload, compared to other types of workloads, is the game server. The game server is critical to the player experience, as players connect to it from their game client to play a game session.

The game server is also one of the biggest drivers of cost for operating a multiplayer game. Therefore, it is important to optimize how you utilize the compute infrastructure for your game servers to reduce costs.

Best practices

- GAMECOST02-BP01 Optimize the cost of data transfer across the internet
- GAMECOST02-BP02 Optimize the number of game sessions hosted on each game server instance to optimize costs
- GAMECOST02-BP03 Select the appropriate compute pricing option to reduce costs

GAMECOST02-BP01 Optimize the cost of data transfer across the internet

While AWS primarily charges for outbound (egress) data transfer from your AWS resources to the internet, game companies can face high costs related to data transfer through AWS Direct Connect or AWS Gateway load balancers, which may charge for both inbound (ingress) and outbound data. Implement solutions that reduce the overall cost of transferring data from your game's AWS backend to your players, focusing on minimizing egress charges from your AWS resources as well as evaluating options to manage ingress and egress fees through AWS connectivity services.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Use Amazon CloudFront to reduce the cost of content delivery and public-facing web applications.

Game content and assets that are stored in the cloud are typically stored in Amazon S3 and delivered to the game client either directly from S3 or from web servers hosted in Amazon EC2

that retrieve the content from Amazon S3 and deliver it to clients. To reduce the data transfer costs of content downloads, consider using Amazon CloudFront in front of your cloud storage to deliver content to users.

Using CloudFront can reduce the cost of data transfer because it costs less to deliver your content from CloudFront points-of-presence than directly from Regions, and CloudFront does not charge origin retrieval fees for AWS-based origins, such as Amazon EC2 and Amazon S3. If your content is static and does not change often, you can use CloudFront to cache that data closer to end-users, which can further reduce costs.

CloudFront also improves the cost efficiency of front-facing public-facing web applications and services, even if caching is not used, since the cost of data transfer between your servers and clients can be reduced by routing traffic through the AWS network.

Amazon CloudWatch can be used to monitor your Amazon CloudFront usage. For use cases where you use multiple content delivery networks (CDNs), Amazon CloudFront Origin Shield can provide an additional layer of caching to consolidate and reduce the number of origin requests from different providers.

To understand your game network traffic, you can enable VPC Flow Logs and Amazon CloudWatch Internet Monitor to have end-to-end visibility on player or game backend connections. That approach can identify the causes for high data transfer cost and perform architectural changes to optimize data transfer spend.

Implementation steps

- Use Amazon CloudFront in front of Amazon S3 or EC2-based content origins to reduce data transfer costs by leveraging lower-cost delivery from CloudFront points-of-presence and removing origin retrieval fees.
- Enable VPC Flow Logs and Amazon CloudWatch Internet Monitor to analyze network traffic and identify architectural changes to optimize data transfer costs.
- Implement CloudFront Origin Shield to consolidate and reduce origin requests when using multiple CDNs for additional cost efficiency.

For more best practices for content delivery, see the Content Delivery for Games whitepaper.

GAMECOST02-BP02 Optimize the number of game sessions hosted on each game server instance to optimize costs

Optimize the number of game sessions hosted per server instance to achieve better compute utilization and reduce compute infrastructure costs.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

To optimize costs, game developers should maximize the number of game sessions hosted on the same physical or virtual server, also known as the packing density of their game servers. This is achieved by increasing the number of game server processes that can be simultaneously hosted on an instance.

A single game server process should not usually require the use of the entire resources available on the EC2 instance. This is one of the most important ways to reduce compute costs for a game and requires the use of software that can spawn and manage multiple server processes on the EC2 Instance on separate ports.

For example, Amazon GameLift has a quota on the maximum number of game server processes per instance, which you should strive to utilize so that you can reduce hosting costs. For more information, see Amazon GameLift Servers endpoints and quotas for details on the current quota for maximum game server processes per instance.

As an alternative to deploying game server processes on virtual machines such as EC2 instances, it is becoming popular for game developers to run their game servers as container-based applications using container orchestration solutions. Game developers can use Amazon Elastic Container Service (Amazon ECS) or Guidance for Game Server Hosting Using Agones and Open Match on Amazon EKS. Another option is Game Server Hosting on AWS Fargate, a serverless compute engine that works with both ECS and EKS, enabling you to focus on your game without having to manage the underlying infrastructure.

Container solutions provide job scheduling functionality that can automatically find an available container instance in the cluster to host your game server container based on resource requirements and other placement logic that you specify. However, it is important to consider how you will manage the scaling and player placement behavior in a way that doesn't disrupt active player sessions.

Implementation steps

- Increase packing density by running multiple game server processes per EC2 instance using separate ports and process management software.
- Use Amazon GameLift or container solutions like ECS, EKS, or AWS Fargate to manage game server processes efficiently and reduce infrastructure costs.
- Continuously monitor resource utilization to refine packing density and maintain cost-efficiency without compromising player experience.

GAMECOST02-BP03 Select the appropriate compute pricing option to reduce costs

Run performance tests of your game server software across a variety of instance types and compute options to determine which option is most cost-effective for your game.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

In addition to efficiently utilizing the right EC2 instance types for your workload, consider which of the available compute pricing options is most suitable for your cost optimization goals. There are several pricing options available, including On-Demand Instances, Spot Instances, Reserved Instances, and Savings Plans.

Savings Plans (SPs) provide discounts for compute by making usage commitments and are ideal for scenarios when you cannot forecast your expected usage for a 1-year or 3-year period. They provide discounts like Reserved Instances with the flexibility to apply these discounts across Regions, instance family, operating system, tenancy. They can also be applied to AWS Fargate, who can be a game server hosting option for casual games or AWS Lambda who is used as a great option for turn-based game that do not require game servers. For more information, see Building a serverless multi-player game that scales.

Savings Plans are introduced during game launches to save costs for game servers workloads that are contributing to EC2 instances spend when the game is released to the audience. Savings Plans can also be introduced post-launch when game operations team have a better understanding of the player traffic after the game has been running in production for an extended period.

Since Savings Plans provide regional flexibility, they are particularly ideal to optimize game servers spend for games with unpredictable usage across geographies.

For example, if your daily player usage pattern requires at least 20 servers to support your player base, but periodically requires up to 40 servers, then consider purchasing Savings Plan commitments to cover the 20 servers' baseline, because that usage demand is predictable and consistent, and will result in maximum utilization of the usage commitment that you have purchased.

Maximize the utilization of Savings Plans and augment them with other purchase options that provide more flexibility for unpredictable game server usage spikes, such as on-demand and Spot Instances to achieve optimal savings.

Spot Instances are ideal for running game servers because they offer the largest compute discounts, do not require usage commitments, and they provide flexibility for unpredictable and spiky workload types. However, Spot Instances can be interrupted, so they are best suited for game server workloads with short game session duration or situations where the tolerance for interruption is higher.

For more information on guidance for running game servers using Kubernetes on Amazon EKS with EC2 Spot Instances, see How to run massively multiplayer games with EC2 Spot using Aurora Serverless.

Use Amazon EC2 Spot Instances to determine pools with the least chance of interruption that will deliver maximum savings compared to on-demand rates.

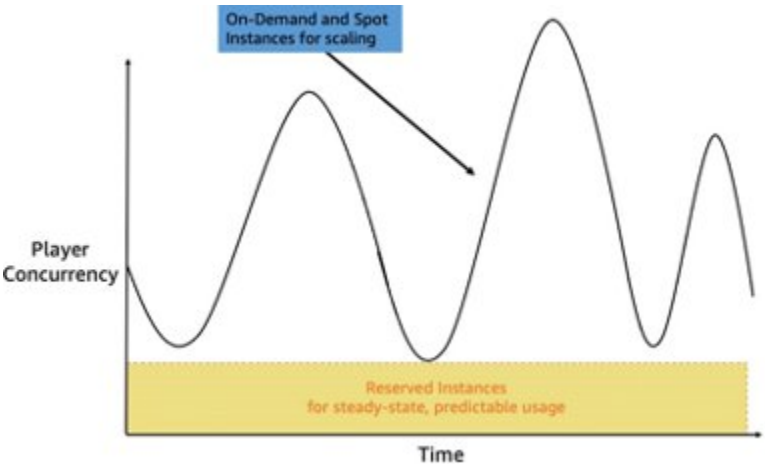
When using Spot, it is also recommended to run game server workloads across multiple EC2 instance types and Availability Zones in an AWS Region to diversify your usage of capacity and reduce interruption risk.

Consider using Spot Instances in combination with On-Demand Instances to minimize the impact of potential disruptions to active game sessions and use capacity optimized allocations strategy to further reduce the risk of interruption.

Refer to the Best practices for Amazon EC2 Spot for additional best practices. Capacity Rebalancing in Auto Scaling to replace at-risk Spot Instances can be used to proactively monitor and add additional capacity when Spot Instances are at increased risk of interruption.

Amazon GameLift FleetIQ integrates with Spot Instances to optimize the use of low-cost Spot Instances while reducing the risk of interruptions. If you are hosting your game using GameLift, review the GameLift documentation for choosing computing resources. For more information, see Choose compute resources for a managed fleet.

The following diagram provides an example to illustrate the use of multiple compute pricing options for game server workloads:



Hosting game servers with multiple EC2 pricing options

In the diagram, the player concurrency fluctuates over time which makes it difficult to manage utilization and achieve cost optimization. To address this fluctuation, consider adopting a mixture of different compute pricing options, using Savings Plans for EC2 to meet the needs of your minimum usage requirements while relying on EC2 On-Demand and EC2 Spot Instances to meet the needs of your player demand.

Implementation steps

- Use Savings Plans for predictable baseline usage, combining them with Spot and On-Demand Instances for flexibility and cost optimization during usage spikes.
- Use Spot Instances for game servers with short session durations or higher interruption tolerance, diversifying across instance types and Availability Zones to minimize risk.
- Implement tools like EC2 Spot Instances Advisor, Capacity Rebalancing, and GameLift FleetIQ to optimize Spot Instance usage and proactively manage interruptions.

Data transfer costs

GAMECOST03: How are you optimizing the data transfer costs for your game infrastructure?

Games can transfer a significant amount of data across the internet between your players' game client devices and your game infrastructure to provide the gameplay experience, as well as between the components of your game infrastructure.

For example, data transfer occurs when players download game content updates to their game clients, save their game progress state to the cloud, engage in real-time multiplayer game sessions with their friends, and when your game infrastructure transfers data between Regions and Availability Zones. It is important to understand where the data transfer occurs in your game workload to optimize your architecture choices to reduce this data transfer cost.

To optimize the data transfer costs for your game workload, consider the following best practices:

Best practices

- GAMECOST03-BP01 Choose the appropriate type of storage for user generated content to reduce costs
- GAMECOST03-BP02 Optimize databases for game backends

GAMECOST03-BP01 Choose the appropriate type of storage for user generated content to reduce costs

Each type of data generated and stored in your game has unique characteristics that you should consider when determining the right storage solution for your workload.

Level of risk exposed if this best practice is not established: Low

Implementation guidance

Use Amazon S3 Object Lifecycle Management to store object data in the most cost-effective storage class. Amazon S3 provides multiple storage classes and object lifecycle management to make it straightforward to set up simple and fine-grained policies to automatically transition data between storage tiers to reduce costs. Instead of simply storing data in the S3 standard storage class by default, consider setting up a lifecycle configuration to transition data between tiers automatically over time, or use S3 Intelligent-Tiering storage class for unknown or changing access patterns.

Alternatively, S3 Intelligent-Tiering can cost-effectively and automatically transition data between tiers and is recommended as a default storage class since it provides cost optimization without

the need to manually setup lifecycle policies, and is now the best choice for small and short-lived objects. For more information, see Amazon S3 Intelligent-Tiering – Improved Cost Optimizations for Short-Lived and Small Objects.

Common use cases for Amazon S3 include storage of game assets, static content, game logs, data lake storage, and backups. For use cases where file systems are required, such as attaching shared file systems to workstations during development, consider using Amazon Elastic File System (Amazon EFS), which provides different storage classes and automatically grows and shrinks as you add and remove files with no need for manage the infrastructure.

Amazon S3 One Zone-IA is an ideal storage option for transient data related to in-game sessions, matchmaking, or other ephemeral information that can be re-created as needed. That type of game data does not require redundancy across multiple Availability Zones (AZs). This lower-cost storage class is well-suited for records of player actions, game events, and other telemetry data used for analytics or debugging.

The key cost optimization benefit of using S3 Express One Zone for such game data is the significant cost savings compared to the standard S3 storage class, with up to a 20% reduction in storage costs. This can be particularly advantageous for games with large volumes of data that do not require the same level of durability and availability as mission-critical application data. By leveraging S3 One Zone, game developers and publishers can optimize their cloud storage costs without compromising the overall player experience.

Implementation steps

- Configure Amazon S3 lifecycle policies to transition data between storage classes or use S3 Intelligent-Tiering as a default for automatic cost optimization with changing access patterns.
- Use S3 One Zone-Infrequent Access for transient game session data, such as telemetry and matchmaking records, to reduce storage costs by up to 20% while maintaining sufficient availability.
- For shared file system needs during development, use Amazon EFS to simplify storage management with elastic capacity and multiple storage classes.

GAMECOST03-BP02 Optimize databases for game backends

Games rely heavily on databases to store a wide range of critical data, from player profiles and inventories to in-game micro transactions and progression metrics. Databases also play a crucial

role in managing the social aspects of games, such as creating and maintaining player groups, parties, and enforcing moderation policies. As the player base of a game grows, the associated database costs will inevitably rise to accommodate the increasing data and usage demands.

Level of risk exposed if this best practice is not established: Medium

Implementation guidance

For game backends running on Amazon Aurora, there are several cost optimization strategies that can be employed. One key recommendation is to auto scale your read replicas based on usage patterns, dynamically scaling the number of replicas up or down to handle fluctuations in traffic. This means that you are paying for the resources you truly need. Another optimization tactic is to replace read replicas used for games analytics with DB snapshots exports to Amazon S3, as the S3 storage service is generally more affordable than provisioned Aurora database instances. For more information, see Exporting DB snapshot data to Amazon S3 for Amazon RDS.

Exploring the use of Reserved DB instances for Amazon Aurora for your core database instances and transitioning to the Aurora Serverless configuration can also lead to substantial long-term cost savings by providing more flexibility and granular control over your resource utilization.

Similarly, for game backends that use Amazon DynamoDB, employing the DynamoDB on-demand capacity mode can be an effective choice, especially for new or unpredictable workloads, as it allows you to pay only for the resources you consume without the need to over-provision. As your game traffic patterns become more stable and predictable over time, you can then transition to the DynamoDB provisioned capacity mode, which can offer cost savings through better capacity planning. Activating auto-scaling on your DynamoDB tables is another key optimization, allowing the service to dynamically adjust the provisioned capacity based on fluctuations in traffic. Test your game's data structure in a development environment before launch to find and remove unnecessary local secondary indexes (LSIs) and global secondary indexes (GSIs). This can lead to substantial cost savings for game data storage and operations. Removing inefficient Scan operations from your game backend code in favor of more targeted queries, purchasing Amazon DynamoDB reserved capacity, and leveraging DynamoDB Streams with AWS Lambda triggers to process game backend events can further optimize your DynamoDB costs. For more information, see Best practices for querying and scanning data in DynamoDB.

By implementing these cost optimization strategies for both Amazon Aurora and DynamoDB, game developers and publishers can significantly reduce their game backend databases spend.

Implementation steps

- Use Aurora read replica auto-scaling and DB snapshot exports to Amazon S3 for cost-efficient handling of fluctuating traffic and analytics needs.
- Optimize DynamoDB costs by starting with on-demand capacity for new workloads, transitioning to provisioned capacity with auto-scaling for predictable traffic, and removing unused LSIs and GSIs.
- Avoid inefficient Scan operations in favor of targeted queries, use Reserved Instances or Reserved Capacity, and use DynamoDB Streams with AWS Lambda for event processing.

Managing demand and supply resources

There are no managing demand and supply resources best practices specific to the Games Lens.

For more information about managing demand and supplying resources, see Cost Optimization Pillar - AWS Well-Architected Framework.

Optimizing over time

There are no optimizing over time best practices specific to the Games Lens.

For more information about optimizing costs over time, see Cost Optimization Pillar - AWS Well-Architected Framework.

Resources

Refer to the following resources to learn more about best practices for cost optimization:

Related documents:

- How do I reduce data transfer charges for my NAT gateway in Amazon VPC?
- Introducing the Amazon GameLift FleetIQ adapter for Agones
- How do I find the top contributors to NAT gateway traffic in my Amazon VPC?
- Choose the right compute strategy for your global game servers
- AWS Well-Architected Labs – Cost effective resources
- Amazon VPC CNI plugin increases pods per node limits

- Architecture Best Practices for Cost Optimization
- Reducing player wait time and right sizing compute allocation using Amazon SageMaker AI RL and Amazon EKS
- AWS Compute Optimizer
- Electronic Arts optimizes storage costs and operations using Amazon S3 Intelligent-Tiering and Amazon Glacier
- Escape unfriendly licensing practices by migrating Windows workloads to Linux
- Overview of Data Transfer Costs for Common Architectures
- AWS and Kubecost collaborate to deliver cost monitoring for EKS customers
- Laying the Foundation: Setting Up Your Environment for Cost Optimization
- Overview of Amazon EC2 Spot Instances

Sustainability

The sustainability pillar provides design principles, operational guidance, best-practices, and improvement plans to help meet sustainability targets for your AWS workloads.

You can find implementation guidance on implementation in the Sustainability Pillar - AWS Well-Architected Framework whitepaper.

Focus areas

- Design principles
- Region selection
- Alignment to demand
- Software and architecture
- Data management
- Hardware and services
- Resources

Design principles

Sustainability for the games industry is evolving at diverse rates in different Regions of the world. With sustainable power in large swaths of North America and sustainability mandates in EU and UK, architects have several approaches to achieving greener workloads in the near future. The Well-Architected sustainability pillar can be used to achieve these measures for a wide variety of workloads.

This section of the lens describes several best practices that may be used for games workloads.

- Select the appropriate storage type for games user data.
- Be aware of data lifecycle policies that will de-duplicate data and purge unneeded data from your workloads.
- Be selective about right-sizing the deployment of compute resources.
- Use serverless for brief and transactional processes.

Region selection

There are no Region selection best practices specific to the Games Lens. For more detail, see Sustainability Pillar - AWS Well-Architected Framework.

Alignment to demand

There are no alignment to demand best practices specific to the Games Lens. For more detail, see Sustainability Pillar - AWS Well-Architected Framework.

Software and architecture

There are no software and architecture best practices specific to the Games Lens. For more detail, see Sustainability Pillar - AWS Well-Architected Framework.

Data management

GAMESUS01: How do you manage user and game data in your game system?

Develop a data lifecycle strategy that optimizes data storage and relevance by only retaining critical historical data.

Best practices

- GAMESUS01-BP01 Use storage technologies that fit the patterns adapted to user content, subscriber information, and in-game purchases
- GAMESUS01-BP02 Use lifecycle policies or TTL expiration to delete unnecessary games user data, log files, or deprecated assets

GAMESUS01-BP01 Use storage technologies that fit the patterns adapted to user content, subscriber information, and in-game purchases

You should classify your data by type, retention need and frequency of access. This enables you to select the most optimized storage solution for the myriad data types of your game or backend

services produce. Fast changing data should be stored in key-value or in-memory database services. Transactional data should be store in relational database services. Large files, game assets, or user-generated content should be stored in object storage services.

Level of risk exposed if this best practice is not established: High

Implementation guidance

Games produce and consume a large variety of data types that require storage solutions optimized for frequency of access, latency, and cost. Data stored should be classified using tags to differentiate data that can be removed or needs to be stored long-term.

The following services work well for a variety of Games use-cases:

Amazon Aurora (compatible with MySQL and PostgreSQL) offers high availability, low-latency, and automatic scaling, making it an excellent choice for handling large amounts of transactional data, such as player account management and authentication, in-game economies, leaderboards and player rankings, game state persistence, event and campaign management, and multi-Region and high-availability deployment.

Amazon DynamoDB is a fully managed NoSQL database known for its low-latency, high throughput, and seamless scalability, which makes it ideal for handling real-time player data, session management, inventory, in-game economy, real-time multiplayer game state, matchmaking, event logging, and scaling for global audience.

Amazon DocumentDB (compatible with MongoDB) provides a scalable, low-latency document-oriented database service, perfect for storing flexible, semi-structured data, such as inventory system, player profiles and customization's, game worlds and procedurally generated content, social and player interactions, analytics and behavior tracking, and in-game metadata and configurations.

Amazon ElastiCache supports in-memory caching with Redis or Memcached, offering rapid data access and reduced response times, which is critical for real-time multiplayer games where speed and performance are essential for a smooth user experience. ElastiCache is utilized in gaming for real-time leaderboards, session management, caching game metadata, in-game chat and messaging, matchmaking, real-time analytics and telemetry, and scaling for high-traffic events.

Amazon Simple Storage Service (S3) can be used to store objects like game assets, videos, pictures, text log files and more. S3 is an object storage service offering industry-leading scalability, data availability, security, and performance.

If it offers multiple storage classes that support frequent and in-frequent data access, and cost-effective archive storage. For data that is frequently accessed throughout development, studios should store objects in S3 Standard for low latency and high throughput performance. For data that frequently goes from hot to cold or vice versa, studios should investigate S3 Intelligent-Tiering. Intelligent-Tiering monitors the access patterns of your data and automatically moves data to the most cost-effective access tier.

For studios that need high throughput, low latency and are ok with it living in a single Availability Zone use S3 Express One Zone. This replicates data to a single AZ and can improve data access speeds compared to S3 standard. For deep archive needs of historical data Amazon also offers Amazon Glacier. The Amazon Glacier storage classes are purpose-built for data archiving, providing you with high performance, retrieval flexibility, and low cost archive storage in the cloud.

Amazon Elastic Block Store can be used to store game servers' binaries, executable files and configurations your game servers or asset repositories need to function. You should snapshot and delete unused volumes that are not attached to an EC2 instance. This alleviates you from storage charges incurred while lowering the usage of unneeded services and hardware.

Implementation steps

- Classify game data by type, retention needs, and access frequency, tagging data to distinguish between short-term and long-term storage requirements.
- Use Amazon Aurora for transactional data, DynamoDB for real-time player data, DocumentDB for semi-structured data, and ElastiCache for low-latency caching of time-critical game information.
- Store game assets, logs, and user-generated content in Amazon S3, selecting appropriate storage classes (for example, Intelligent-Tiering, One Zone, and Glacier) based on access patterns and archive needs, and use EBS for game server binaries and configurations with regular snapshot management.

GAMESUS01-BP02 Use lifecycle policies or TTL expiration to delete unnecessary games user data, log files, or deprecated assets

You can use tags and data type to create lifecycle policies or TTL's to move data to archival storage or remove completely from the service. This may include temporary configurations, expired archived content, and historical logs that are no longer needed. Most services support tagging.

Level of risk exposed if this best practice is not established: High

Implementation guidance

For data stored in S3 you can use lifecycle policies to move the data to infrequent access and archival tiers of storage. In an S3 Lifecycle configuration, you can define rules to transition objects from one storage class to another to save on storage costs. When you don't know the access patterns of your objects, or if your access patterns are changing over time, you can transition the objects to the S3 Intelligent-Tiering storage class for automatic cost savings.

Amazon S3 supports a waterfall model for transitioning between storage classes, as shown in the following diagram.

You can add transition actions to an S3 Lifecycle configuration to tell Amazon S3 to delete objects at the end of their lifetime. When an object reaches the end of its lifetime based on its lifecycle configuration, Amazon S3 takes an Expiration action based on which S3 Versioning state the bucket is in:

- **Non-versioned bucket:** Amazon S3 queues the object for removal and removes it asynchronously, permanently removing the object.
- **Versioning-enabled bucket:** If the current object version is not a delete marker, Amazon S3 adds a delete marker with a unique version ID. This makes the current version noncurrent, and the delete marker the current version.
- **Versioning-suspended bucket:** Amazon S3 creates a delete marker with null as the version ID. This delete marker replaces an object version with a null version ID in the version hierarchy, which effectively deletes the object.
- When you add a Lifecycle configuration to a bucket, the configuration rules apply to both existing objects and objects that you add later. For example, if you add a Lifecycle configuration rule today with an expiration action that causes objects with a specific prefix to expire 30 days after creation, Amazon S3 will queue for removal existing objects that are more than 30 days old and that have the specified prefix.

Time To Live (TTL) for DynamoDB is a cost-effective method for deleting items that are no longer relevant. TTL allows you to define a per-item expiration timestamp that indicates when an item is no longer needed. DynamoDB automatically deletes expired items within a few days of their expiration time, without consuming write throughput.

- To use TTL, first enable it on a table and then define a specific attribute to store the TTL expiration timestamp. The timestamp must be stored in Unix epoch time format at the seconds

granularity. Each time an item is created or updated, you can compute the expiration time and save it in the TTL attribute.

- Items with valid, expired TTL attributes may be deleted by the system, typically within a few days of their expiration. You can still update the expired items that are pending deletion, including changing or removing their TTL attributes. While updating an expired item, we recommended that you use a condition expression to make sure the item has not been subsequently deleted. Use filter expressions to remove expired items from Scan and Query results.
- Deleted items work similarly to those deleted through typical delete operations. Once deleted, items go into DynamoDB Streams as service deletions instead of user deletes and are removed from local secondary indexes and global secondary indexes just like other delete operations.

With ElastiCache for Redis you can control the freshness of your cached data by using TTLs or expiration on cached keys. After the set time has passed, the key is deleted from the cache, and access to the origin data store is required along with reaching the updated data.

- Two principles determine the appropriate TTLs to apply and the types of caching patterns to implement. First, it's important that you understand the rate of change of the underlying data. Second, it's important that you evaluate the risk of outdated data being returned to your application instead of its updated counterpart.
- With dynamic data that changes often, you might want to apply lower TTLs that expire the data at a rate of change that matches that of the primary database. This lowers the risk of returning outdated data while still providing a buffer to offload database requests.
- It's also important to recognize that, even if you are only caching data for minutes or seconds versus longer durations, appropriately applying TTLs to your cached keys can result in a performance boost and an overall better player experience with your game.

Implementation steps

- Use Amazon S3 Lifecycle policies to transition objects to infrequent access or archival tiers and configure expiration actions to delete unnecessary objects based on lifecycle rules.
- Enable Time to Live (TTL) in DynamoDB tables to automatically delete expired items without consuming write throughput, defining the expiration timestamp in Unix epoch time.
- Set appropriate TTLs for ElastiCache keys based on data change rates and risk tolerance for outdated data, facilitating cached data freshness and improved player experience.

Hardware and services

GAMESUS02: How do you manage the use of compute resources in your games backend?

Studios should develop a compute strategy that uses a mix of different compute types, managed services, and savings plan to optimize your usage. You should also optimize how game servers and backend services are packed on to compute instances to reduce the number of unneeded resources.

Best practices

- GAMESUS02-BP01 Select managed services for appropriate compute workloads
- GAMESUS02-BP02 Right-size your compute and deploy GPU performance only where needed

GAMESUS02-BP01 Select managed services for appropriate compute workloads

Architect your game backend services to use managed services for event driven or highly variable traffic workloads. Managed services shift the management of infrastructure to AWS and distributes the environmental impact across multiple users because of the multi-tenanted control planes.

Level of risk exposed if this best practice is not established: High

Implementation guidance

AWS services like AWS Lambda, AWS Fargate (Containers), and Amazon Gamelift (Game server orchestration) can run code, containers, or orchestrate your game servers without having to manage the underlying infrastructure. These services automatically scale based on player demand and you're only charged for the resources you consume. Because the underlying infrastructure is managed on your behalf, you are able to focus solely on your games and backend services' requirements.

You can use AWS Lambda to run code without provisioning or managing servers. Lambda runs your code on a high-availability compute infrastructure and performs the administration of the compute resources, including server and operating system maintenance, capacity provisioning and automatic scaling, and logging. With Lambda, you need to supply your code in one of the language runtimes that Lambda supports. Lambda is useful for processing game events, player

authentication, in-game purchase processing, and matchmaking requests. Lambda automatically scales based on the number of events and can handle unexpected spikes in traffic.

AWS Fargate is a serverless compute engine for containers that works with both Amazon Elastic Container Service (ECS) and Amazon Elastic Kubernetes Service (EKS). AWS Fargate makes it straightforward to focus on building your applications by alleviating the need to provision and manage servers, lets you specify and pay for resources per application, and improves security through application isolation by design. Fargate is ideal for backend services that handle player profiles, state management and matchmaking.

Amazon GameLift is a managed service for deploying, operating, and scaling dedicated game servers for session-based multiplayer games. You can deploy your first game server in the cloud in just minutes, saving up to thousands of engineering hours in upfront software development and lowering the technical risks that often cause developers to cut multiplayer features from their designs.

Implementation steps

- Use AWS Lambda for event-driven workloads like processing game events, player authentication, in-game purchases, and matchmaking requests, leveraging its automatic scaling and serverless management.
- Deploy AWS Fargate with ECS or EKS for backend services such as player profiles, state management, and matchmaking, removing server management and improving application isolation.
- Use Amazon GameLift to deploy and scale dedicated game servers for session-based multiplayer games, reducing development time and operational complexity.

GAMESUS02-BP02 Right-size your compute and deploy GPU performance only where needed

Architect your game servers and backend to efficiently utilize compute resources. Over-provisioning compute can lead to unnecessary costs and minimizes the amount of idle or under-utilized resources. GPU instances should be used to support specific development efforts like HLOD rebuilds in Unreal, or if you game servers require them by design. This greatly reduces the environmental impact and costs of your workloads.

Level of risk exposed if this best practice is not established: High

Implementation guidance

You should optimize your game servers and backend services to use multiple EC2 instances types and the least number of instances needed. This increases the number of instances available to meet your needs during development or for your games launch. You should also match the instance type to the specific workload you're deploying. Compute Optimized instances support a wide range of use-cases including game servers and backend services like matchmaking. Memory Optimized instances are designed to deliver fast performance for workloads that process large data sets in memory. Use GPU instances as required for high performance requirements, but not for general computing tasks. If able, architect your services or game servers to run on ARM with AWS Graviton instances. Graviton is the most performant to energy efficient instance type available on AWS. They also offer improved performance and costs when compared to x86 instance types.

Use the AWS Compute Optimizer to identify the optimal AWS resource configurations, such as Amazon Elastic Compute Cloud (EC2) instance types, Amazon Elastic Block Store (EBS) volume configurations, task sizes of Amazon Elastic Container Service (ECS) services on AWS Fargate, commercial software licenses, AWS Lambda function memory sizes, and Amazon Relational Database Service (RDS) DB instance classes, using machine learning to analyze historical utilization metrics. Compute Optimizer provides a set of APIs and a console experience to reduce costs and increase workload performance by recommending the optimal AWS resources for your AWS workloads.

Implementation steps

- Match compute resources to specific workloads by using Compute Optimized instances for game servers, Memory Optimized instances for large data sets, and GPU instances only for tasks like HLOD rebuilds or GPU-dependent game servers.
- Optimize compute utilization by deploying AWS Graviton instances where possible for energy efficiency, better performance, and cost savings compared to x86 instances.
- Use AWS Compute Optimizer to analyze historical utilization and recommend the most efficient configurations for EC2, AWS ECS, AWS Lambda, and Amazon RDS workloads to reduce costs and improve performance.

Resources

Refer to the following resources to learn more about our best practices related to sustainability.

- UN: Gaming industry spotlights threats to the planet

- Medium: Environmental Sustainability in Game Development: Playing Responsibly for a Greener Future

Key AWS services

- Amazon Aurora
- Amazon DynamoDB
- Amazon DocumentDB (with MongoDB compatibility)
- Amazon ElastiCache
- Amazon S3
- Amazon S3 Storage Classes
- Amazon S3 Intelligent-Tiering storage class
- Amazon S3 Express One Zone storage class
- Amazon Elastic Block Store
- AWS Lambda
- Amazon Elastic Container Service
- Amazon Elastic Kubernetes Service
- Amazon GameLift
- AWS Compute Optimizer

Conclusion

Games are designed to deliver entertainment experiences to a global audience of players and have usage characteristics that are typically unpredictable and variable. The Games Industry Lens describes the common types of scenarios that typically comprise a game architecture and provides a set of questions and best practices to consider when you are building and operating games in the cloud. By applying this framework to your game architecture, you can build reliable, secure, efficient, and cost-effective games in the cloud.

Contributors

The following individuals contributed to this document:

- Adam Hatfield, Senior Solutions Architect, Amazon Web Services
- Brady Webb, Technical Account Manager, Amazon Web Services
- Bruce Ross – Senior Solutions Architect, Well-Architected Lens Leader, Amazon Web Services
- Caleb Cecil, Associate Solutions Architect, Amazon Web Services
- Carlos Perez, Cloud Optimization Success SA, Amazon Web Services
- Chase Herrington, ESL Technical Account Manager, Amazon Web Services.
- Chris Blackwell, Sr. Solutions Architect, Amazon Web Services
- Corey Ouderkirk, Sr. Technical Account Manager, Amazon Web Services
- Derek Villavicencio, Prototyping SA, Amazon Web Services
- Erik Ynigo Becerril, Senior Solutions Architect, Amazon Web Services
- Grzegorz Ochmanski, Sr. Solutions Architect, Amazon Web Services
- Hadrian Baron, Sr. Technical Account Manager, Amazon Web Services
- Ian Armbruster, Sr. Customer Solutions Manager, Amazon Web Services
- Jed Obray, Sr. Technical Account Manager, Amazon Web Services
- Khurram Khokhar, Sr. Technical Account Manager, Amazon Web Services
- Kyle Somers, Sr. Manager, Solutions Architecture, Amazon Web Services
- Madhuri Srinivasan, Senior Technical Writer, Well-Architected, Amazon Web Services
- Matthew Wygant, Sr. TPM Guidance, Well-Architected, Amazon Web Services
- Nataliya Godunok, Cloud Optimization Success SA, Amazon Web Services
- Nirav Doshi, Principal Solutions Architect, Amazon Web Services
- Olivia Liddell, Solutions Architect, Amazon Web Services
- Randy James, Principal Technical Account Manager, Amazon Web Services
- Reou Ando, Game Solutions Architect, Amazon Web Services
- Richard Raseley, Sr. Technical Account Manager, Amazon Web Services
- Sam Patzer, Senior Solutions Architect, Amazon Web Services
- Scott Selinger, Sr. Solutions Architect, Amazon Web Services
- Sean Allen, Sr. Solutions Architect, Amazon Web Services

- Serge Poueme, Senior Solutions Architect, Amazon Web Services
- Stewart Matzek, Senior Technical Writer, Well-Architected, Amazon Web Services
- Trenton Potgieter, Sr Solutions Architect AI/ML/Analytics, Amazon Web Services

Document revisions

To be notified about updates to this whitepaper, subscribe to the RSS feed.

Change	Description	Date
New lens version	Entire lens updated with new best practice guidance.	December 9, 2025
Initial publication	Whitepaper first published.	November 19, 2021

AWS Glossary

For the latest AWS terminology, see the AWS glossary in the *AWS Glossary Reference*.